CS5486
Intelligent Systems

Prof. Jun Wang
Department of Computer Science
Tel: 3442 9701
Email: jwang.cs@cityu.edu.hk

Aims

This course aims to equip students with the skills of problem solving using artificial intelligence (AI) techniques through a demonstrable knowledge in a range problem solving methods and the associated knowledge representation and machine learning techniques.

Keywords

Artificial intelligence vs. computational intelligence. Neural networks. Knowledge representations. Machine learning. Rule-based systems. Fuzzy Systems. Evolutionary computation.

Syllabus

- An introduction to the goals and objectives of AI as a discipline and its milestones. Approaches in AI. Major components in intelligent systems.
- Methods of knowledge acquisition and representations. Associative memory. Techniques on machine learning such as supervised learning, unsupervised learning, reinforcement learning, and deep learning. Generalization.

Syllabus (cont'd)

- Basic concepts of graph and tree search. Optimization methods such as stochastic annealing, neurodynamic optimization, genetic algorithm, particle swarm optimization, ant colony optimization, and differential evolution.

References

- R. Rojas, *Neural Networks: A Systematic Introduction*, Springer, 1996.
- S. Haykin, *Neural Networks and Learning Machines* (3rd Ed), Prentice-Hall, 2009.
- S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. 3rd Ed. Prentice-Hall (2009)
- C.-T. Lin and C.S. G. Lee, *Neural Fuzzy Systems*, Prentice-Hall, 1996.

Intelligence

"Intelligence is a mental quality that consists of the abilities to learn from experience, adapt to new situations, understand and handle abstract concepts, and use knowledge to manipulate one's environment."

Britannica

Definition of Intelligent Systems

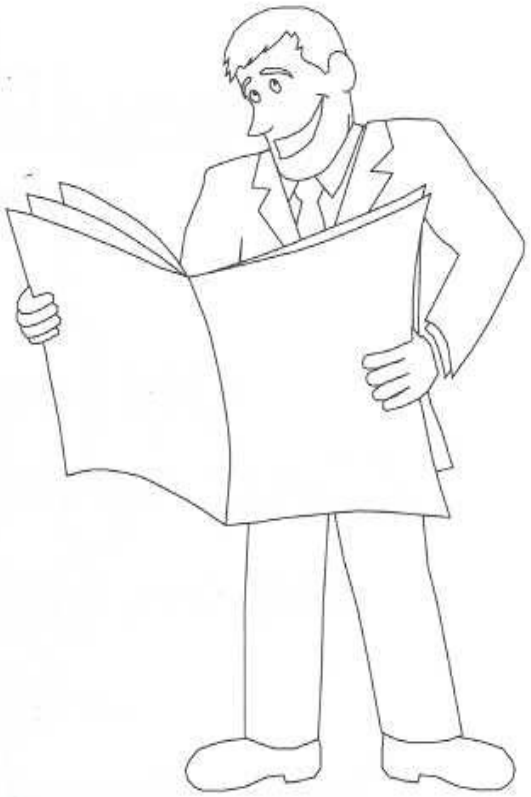
A system is an intelligent system if it exhibits some intelligent behaviors.

For example, neural networks, fuzzy systems, simulated annealing, genetic algorithms, and expert systems.

Intelligent Behaviors

- Inference: Deduction vs. Induction (generalization); e.g., judgment and pattern recognition
- Learning and adaptation: Evolutionary processes; e.g., learning from examples
- Creativity: e.g., planning and design

How far have we got?



An Electric Brain Capable of Translating Foreign Languages is Being Built..

An headline from the popular press of 1950

Today's most intelligent machine is far from being able to do what a child easily does, such as eating food with a knife and fork, peeling an orange, etc

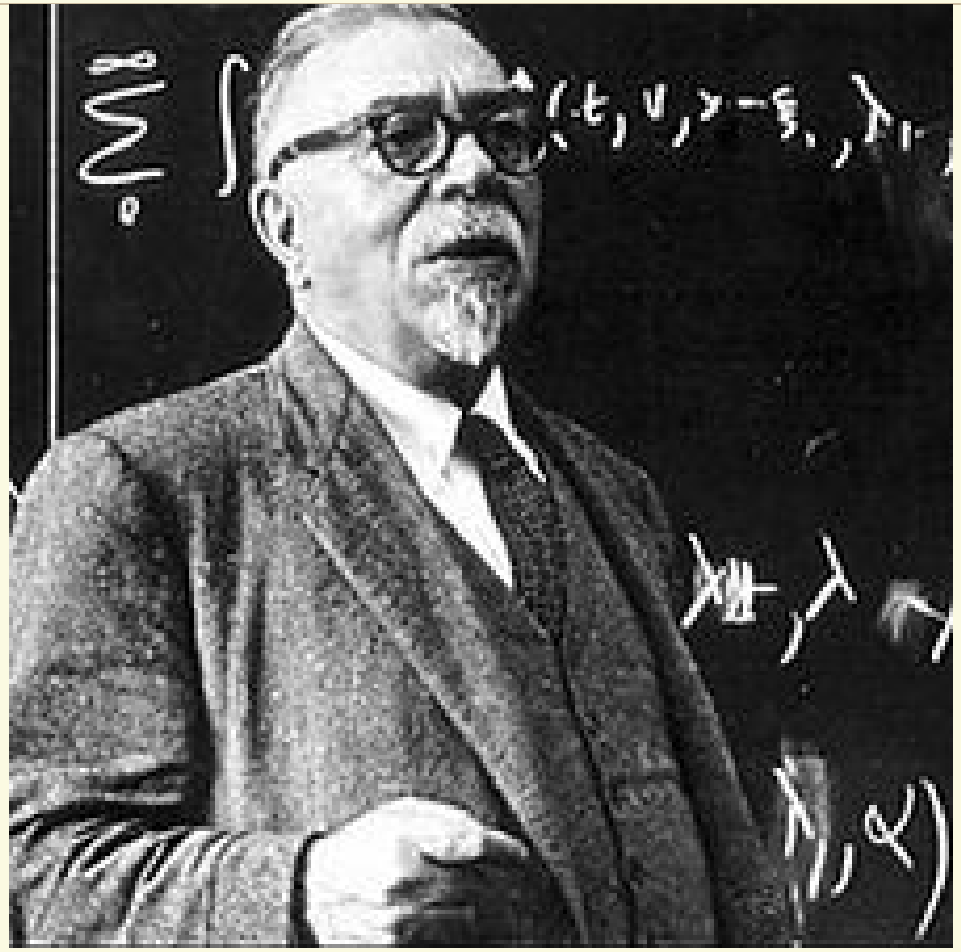
Milestones of Intelligent System Development

- 1940's: Cybernetics by Wiener
- 1943: Threshold logic networks by McCulloch and Pitts
- 1950's-1960's: Perceptrons by Rosenblatt
- 1960's Adaline by Widrow
- 1970's: AI and Expert systems
- 1970's: Fuzzy logic by L. Zadeh
- 1974: Back propagation algorithm by P. Werbos
- 1970's: Adaptive resonance theory by S. Grossberg
- 1970's: Self-organizing map by T. Kohonen
- 1980's: Hopfield networks by J. Hopfield
- 1980's: Genetic algorithms by J. Holland
- 1980's: Simulated annealing by Kirkpatrick et al.
- 2006-: Deep learning by Hinton et al.

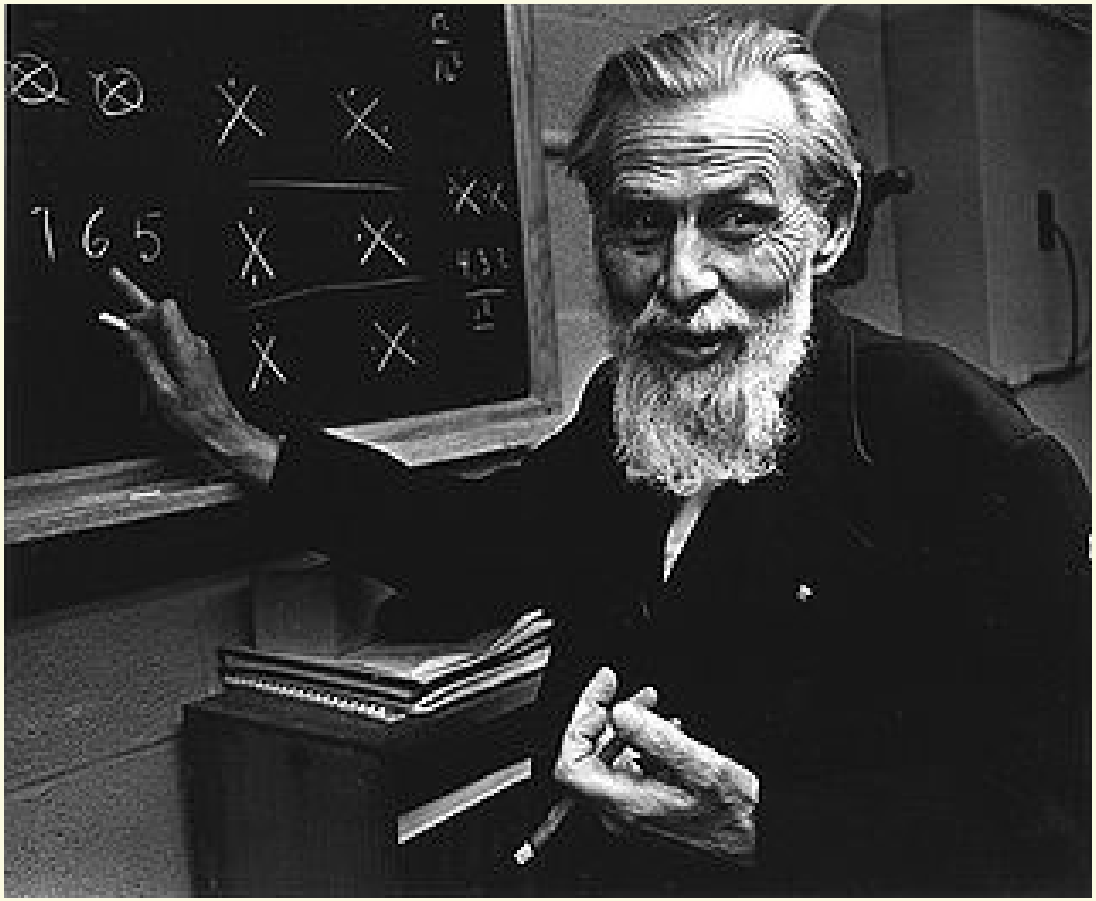
Milestones (cont'd)

- Deep Blue (IBM), 1997
  defeated chess world champion Garry Kasparov
- Alpha Go (Google), 2016
  defeated go nine dan Sedol Lee

Professor Norbert Wiener

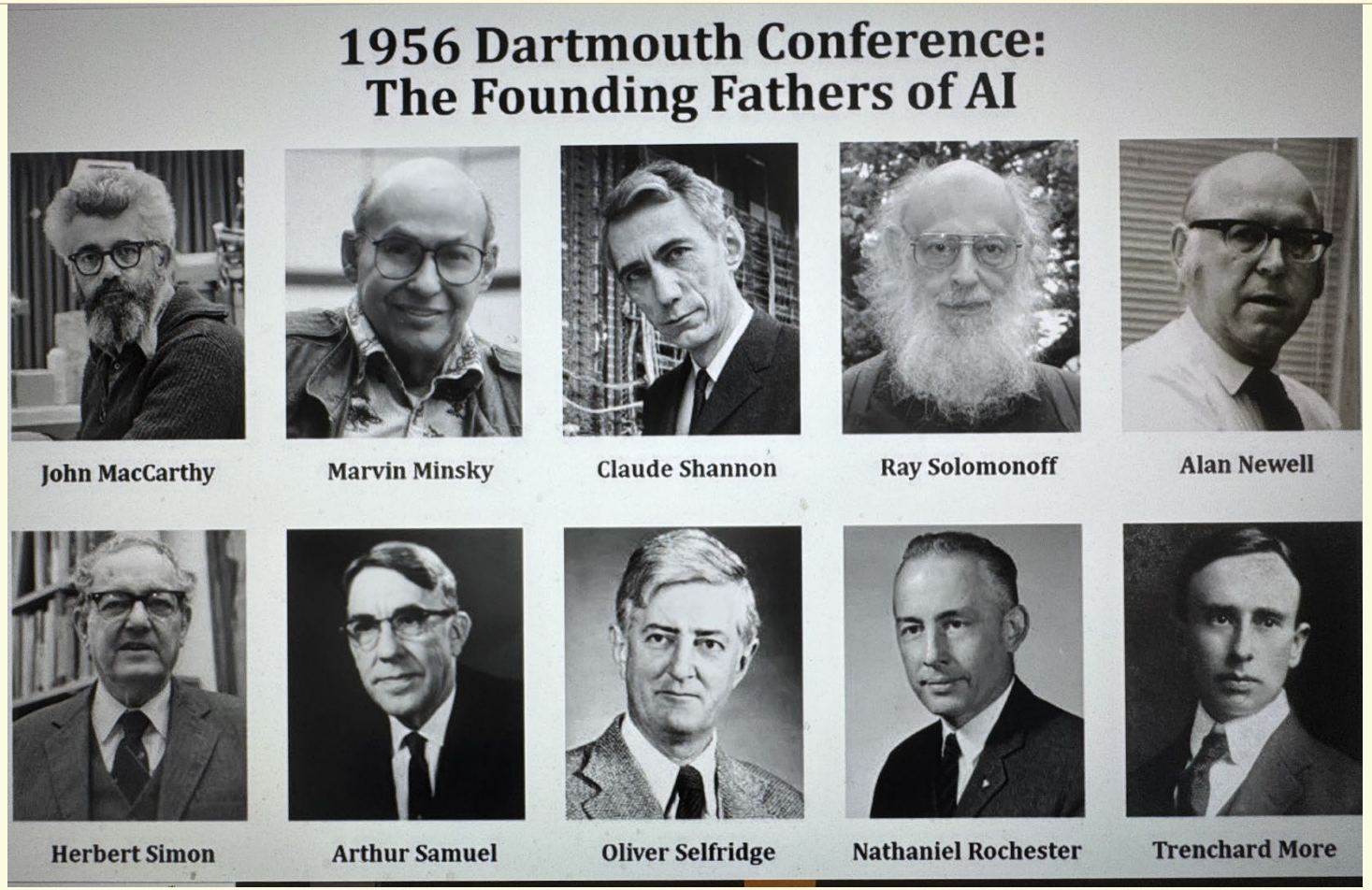


Professor Warren S. McCulloch



0_8MW8iP2QC_WNhmiW.webp

The Birth of "AI"

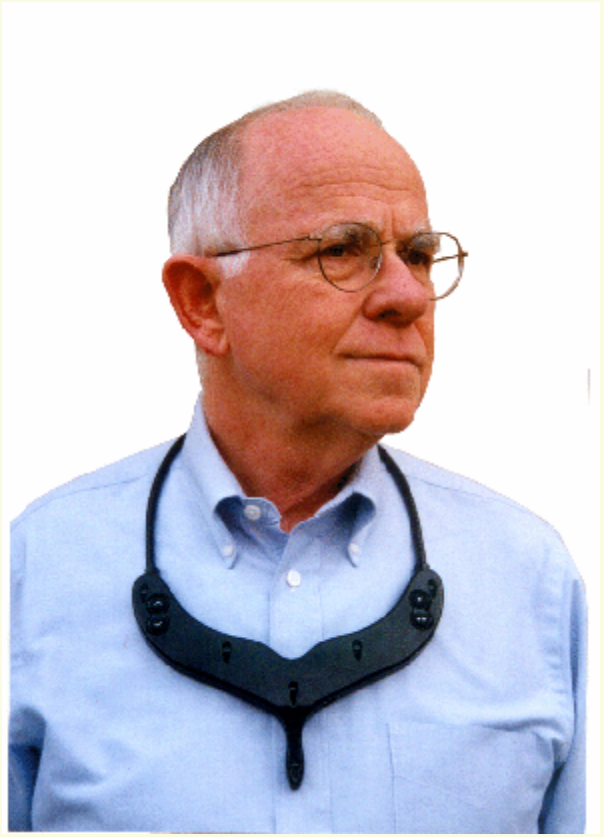


Professor Frank Rosenblatt





Professor Bernard Widrow



Professor Lofti A. Zadeh



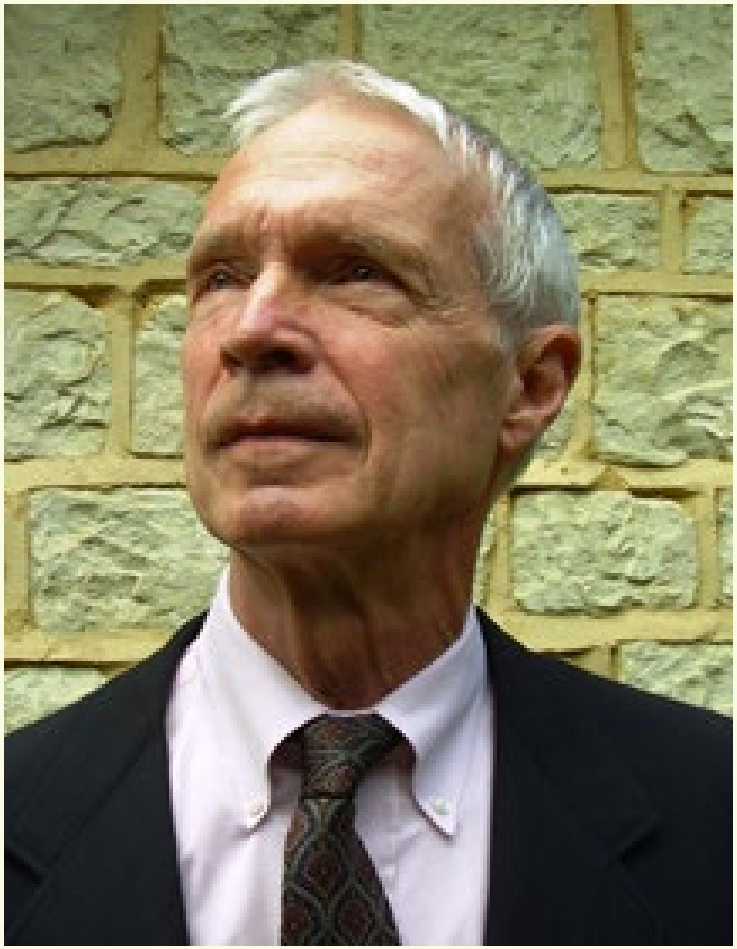
Dr. Paul J. Werbos



Professor Teuvo Kohonen



Professor John J. Hopfield

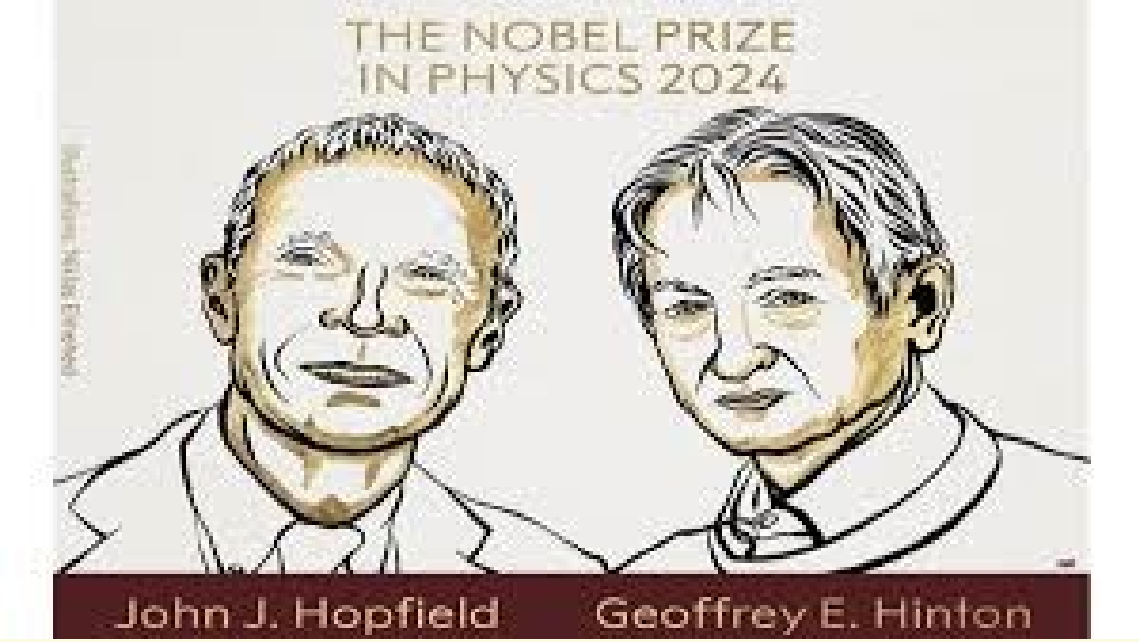


Professor Jeffrey E. Hinton



2024 Nobel Prize in Physics

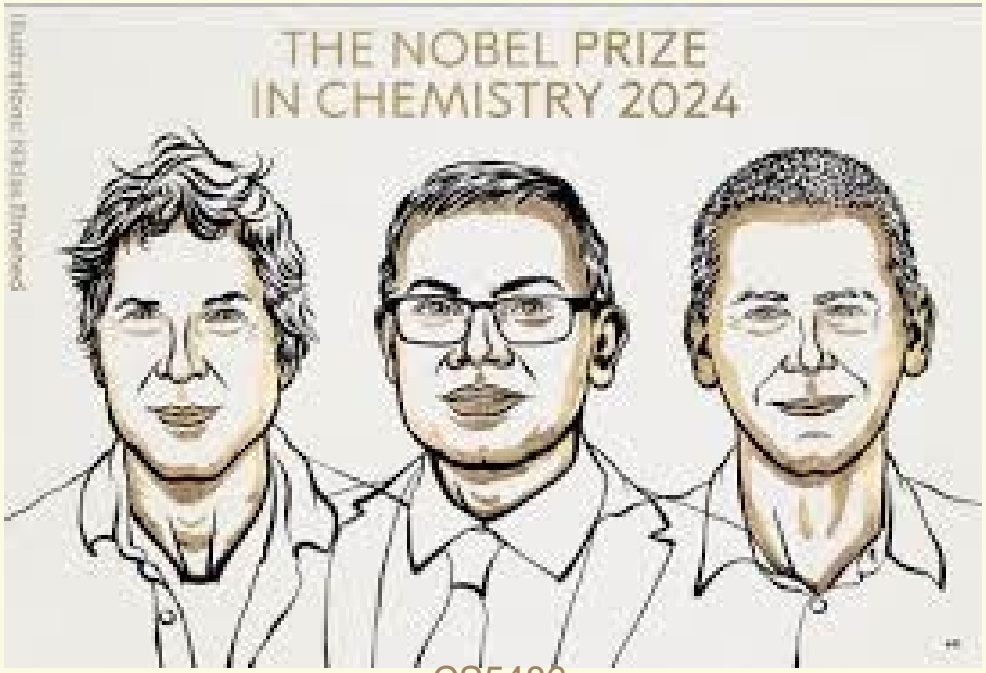
"…foundational discoveries and inventions that enable machine learning with artificial neural networks".



2024 Nobel Prize in Chemstry

D. Baker, D. Hassabis, and J. Jumper

for their contribution to “computational protein design” and “protein structure prediction”.



Applications of Intelligent Systems

- ✓ Pattern recognition: e.g., image processing, pattern analysis, speech recognition, etc.
- ✓ Control and robotics: e.g., modeling and estimation
- ✓ Associative memory (content-addressable memory)
- ✓ Forecasting: e.g., in financial engineering

Generative AI





We're introducing Claude Fable 5.1 and Claude Mythos 5.1. They're the world's most advanced models for coding and knowledge work—and their research capabilities offer an early glimpse of how AI models will contribute to scientific progress.

Claude Fable 5.1 and Claude Mythos 5.1 are the same model, but with different levels of safeguards. Fable 5.1 is generally available, while Mythos 5.1 is available only through our trusted access programs; its safeguards are specifically designed to support work in cybersecurity and the life sciences.

Alongside its increased capabilities, Fable 5.1 takes important steps towards addressing the feedback we've received from customers on price, data retention, and safeguards.

Price. Fable 5.1 will cost an estimated 25% less than Fable 5 for typical workloads, wherever usage is billed by token. This is because we're reducing our pricing on cache reads (where the model reads inputs that have already been processed and stored). For highly agentic work, the savings will often be much larger—up to approximately 45%.

Data retention. Our new system of Enterprise Frontier Safeguards (EFS) gives customers complete privacy (the same as a zero data retention policy) while still being state-of-the-art at preventing adversarial use. EFS works by storing data in cloud infrastructure controlled entirely by the customer, not Anthropic. It will be made available to enterprise customers in phases, beginning later this fall. Until EFS is available, eligible customers will be able to use Fable 5.1 with zero data retention.

Embodied AI

Electric & new energy vehicles Business / China EVs

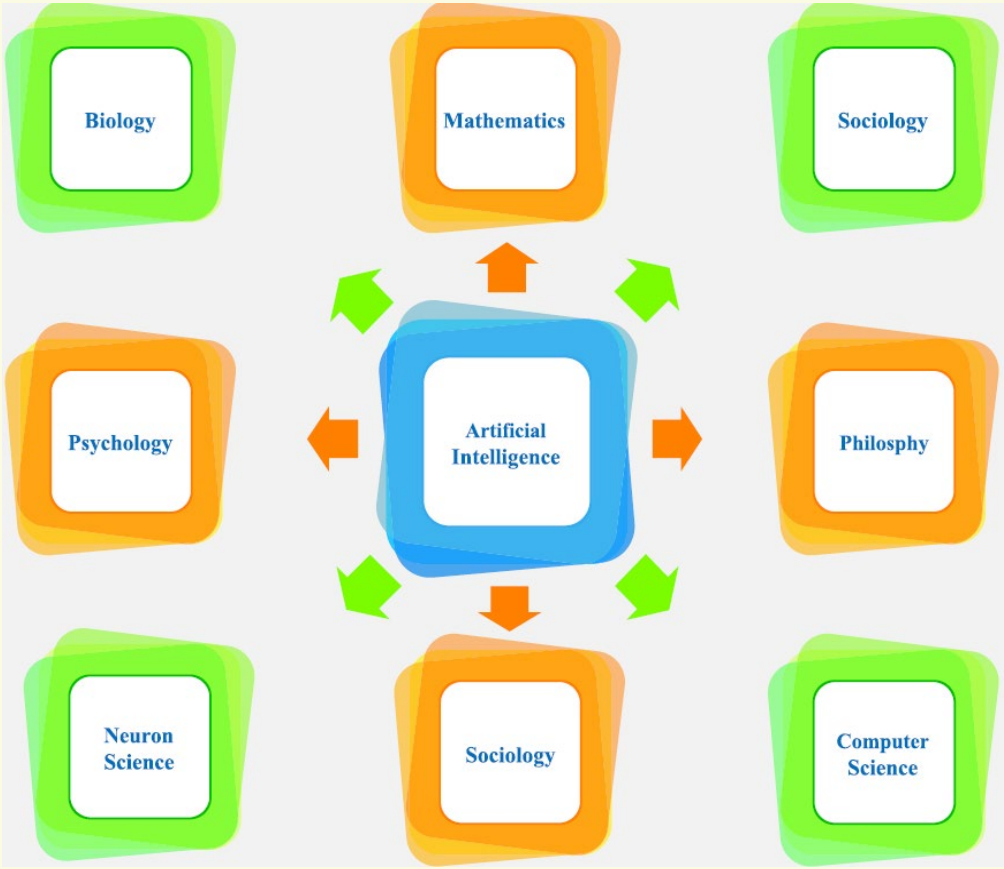
EV maker Xpeng set to challenge Tesla in embodied AI after robotics unit raises US$900m

Backers including Alibaba, Tencent and IDG put up a combined US$600 million while Xpeng itself pledges US$200 million

2-MIN READ Listen

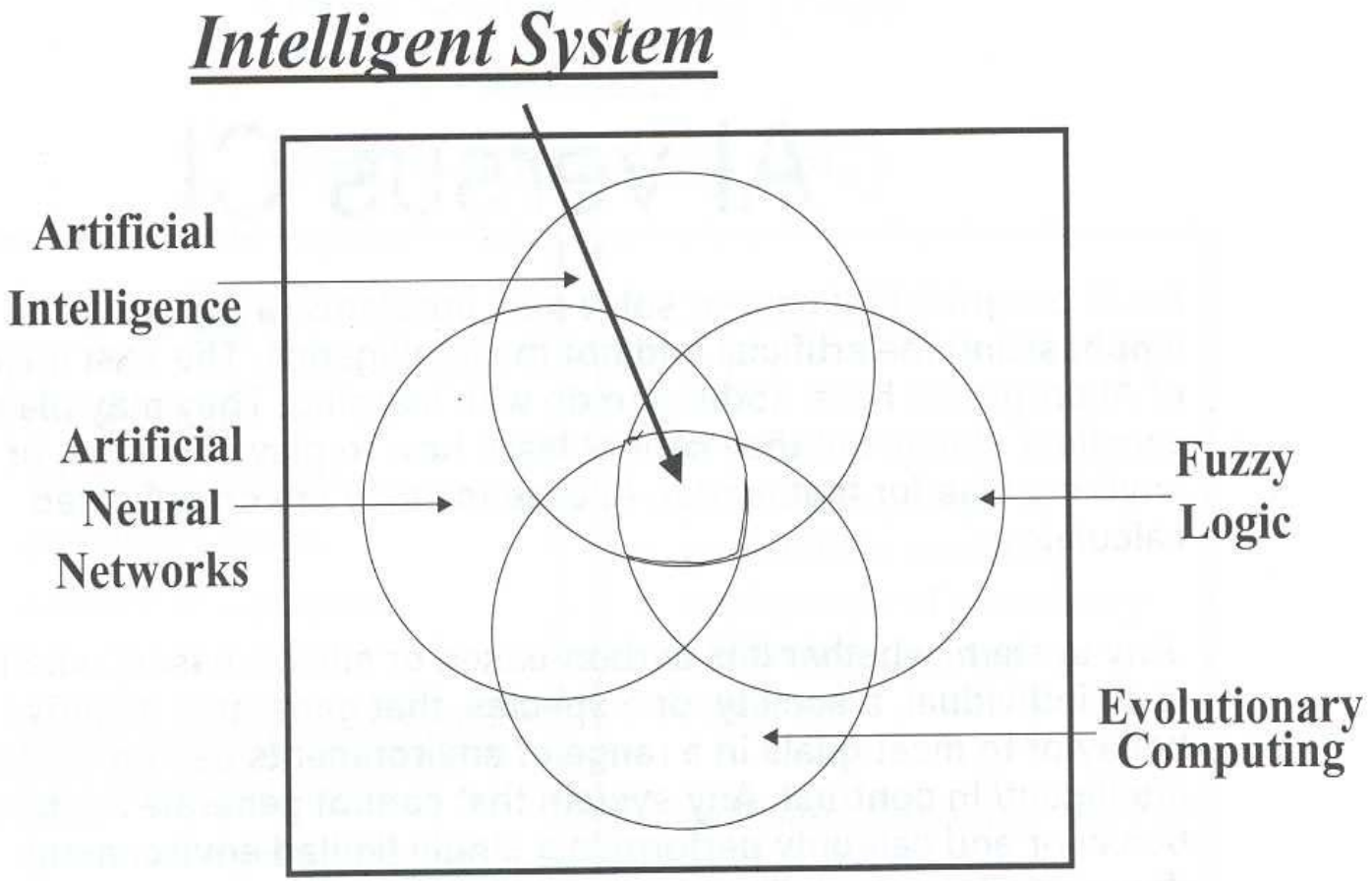


AI and Other Sciences



Computational Intelligence

- Coined by IEEE Neural Networks Council in 1994.
- Represent a new generation of intelligent systems.
- Consist of Neural Networks, Fuzzy Logic, and evolutionary computing techniques (genetic algorithms).

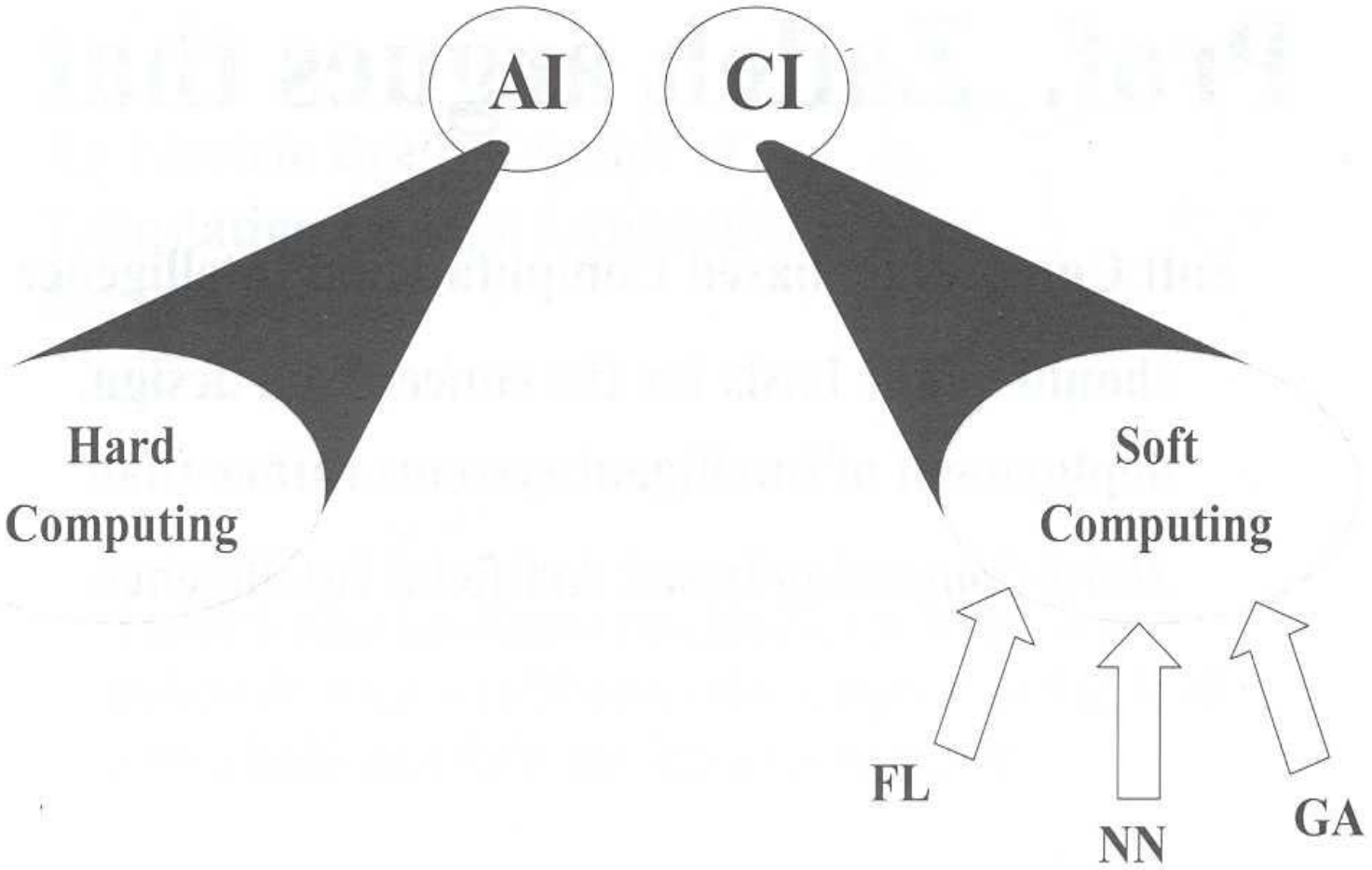

Intelligent System
Artificial
Intelligence
Artificial
Neural
Networks
Fuzzy
Logic
Evolutionary
Computing

AI versus CI

An AI program that cannot solve new problems in new ways is emphasizing the artificial and not the intelligence. The vast majority of AI programs have nothing to do with learning. They may play excellent chess, but they cannot learn how to play checkers, or anything else for that matter. In essence they are complicated calculators.

Any system, whether it is carbon-based or silicon-based, whether it is an individual, a society, or a species, that generates adaptive behavior to meet goals in a range of environments can be said to be intelligent/ In contrast. Any system that cannot generate adaptive behavior and can only perform in a single limited environment demonstrates no intelligence.

(Folger)


AI
CI
Hard
Computing
Soft
Computing
FL
NN
GA

Soft Computing

"Soft computing based on computational intelligence should be the basis for the conception, design, deployment of intelligent systems rather than hard computing based on artificial intelligence."

Lofti Zadeh

Hard Computing (HC) versus Soft Computing (SC)

HC

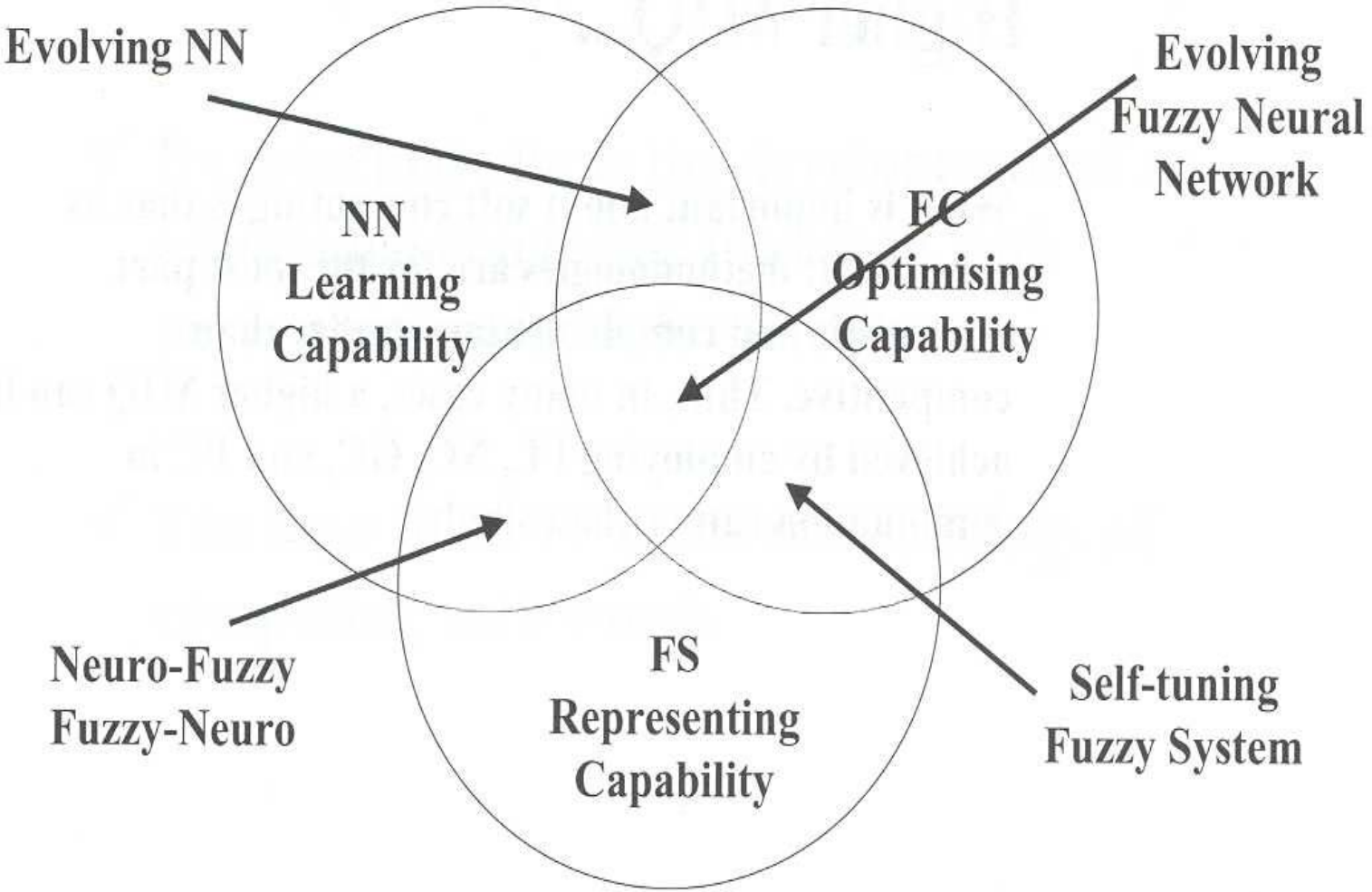
- Bivalent Logic
- Numerical analysis
- probability theory
- differential equations
- functional analysis
- Mathematical programming
- approximation theory

quantitative, precise, formal

SC

- fuzzy Logic
- neurocomputing
- genetic computing
- probabilistic reasoning
- management of uncertainty
- evidential reasoning
- rough sets

qualitative, imprecise, informal


Evolving NN
Evolving
Fuzzy Neural
Network
NN
Learning
Capability
EC
Optimising
Capability
Neuro-Fuzzy
Fuzzy-Neuro
FS
Representing
Capability
Self-tuning
Fuzzy System

What are Neural Networks?

- Composed of a number of interconnected neurons, resembling the human brain.
- Also known as connectionist models, parallel distributed processing (PDP) models, neural computers, and neuromorphic systems.

Components of Neural Networks

- A number of artificial neurons (also known as nodes, processing units, or computational elements)
- Massive inter-neuron connections with different strengths (also known as synaptic weights).
- Input and output channels

CS5486

Formalization of Neural Networks

ANN = (ARCH, RULE)

ARCH: architecture, refers to the combination of components

RULE: rules, refers to the set of rules that relate the components

Architecture of Neural Networks

$$\text{ARCH} = (u, v, w, x, y)$$

- Simple and alike neurons represented by u and v in N-dimensional space
- Inter-neuron connection weights represented by w in M-dimensional space
- External input and outputs represented respectively by x and y in n and m-dimensional space

Model of Neurons

- Biological neurons: 10^{10}-10^{11}
- Highly simplified
- Fire activities are quantified by using state variables (also called activation states)
- Net input to a neuron is usually a weighted sum of state variables from other neurons, input and/or output variables
- Net input to a neuron usually goes thru a nonlinear transformation called activation

Connections between Neurons

- Adaptive: Synaptic connections with adjustable weights
- Excitatory (positive weight) vs. inhibitory (negative weight)
- Distributed knowledge representation, different from digital computers

Rules of Neural Networks

RULE = (*E, F, G, H, L*)

E: Evaluation rule mapped from *v* and/or *y* to a real line; e.g., error function or energy function

F: Activation rule mapped from *u* to *v*; e.g., activation function

G:Aggregation rule mapped from *v, w*, and/or *x* to *u*; e.g., weighted sum

H: output rule mapped from *v* to *y, y* usually is a subset of *v*

L: Learning rule mapped from *v, w*, and *x* to *w*, usually iterative

Learning in Neural Networks

- Goal: To improve performance
- Means: interact with environment
- A process by which the adaptable parameters of an ANN are adjusted thru an iterative process of stimulation by the environment in which the ANN is embedded
- Supervised vs. unsupervised

On Learning

"By three methods we may learn wisdom: First, by reflection, which is the noblest; second, by imitation, which is the easiest, and third by experience, which is the bitterest."

Confucius 孔子

General Incremental Learning Rule

- Discrete-time:

$$w(t+1) = w(t) + L(v, w, x)$$

$$\Delta w(t) = L(v, w, x)$$

- Continuous-time:

$$\frac{dw(t)}{dt} = L(v, w, x)$$

Two-Time Scale Dynamics in Neural Networks

- Faster dynamics in neuron activities represented by *u* and *v*. Also called short-term memory
- Slower dynamics in connection weight activities represented by *w*. Also called long-term memory

Categories of Neural Networks

- Deterministic vs. stochastic, in terms of *F*
- Feedforward vs. recurrent, in terms of *G* and *H*
- Semilinear vs. higher-order, in terms of *G*
- Supervised vs. unsupervised, in terms of *L*

Definition of Neural Networks

Massive parallel distributed processors that have a natural property for storing experiential knowledge and making it available for use

Features of Neural Networks

Resemble the brains in two aspects:

1. Knowledge acquisition: knowledge is acquired by neural networks thru learning processes.
2. Knowledge representation: Inter-neuron connections, known as synaptic weights are used to store acquired knowledge

Properties of Neural Networks

1. Nonlinearity
2. Input-output mapping
3. Adaptivity
4. Contextual information
5. Fault tolerance
6. hardware implementability
7. Uniformity of analysis and design
8. Neurobiological analogy and plausibility

McCulloch-Pitts Neuron

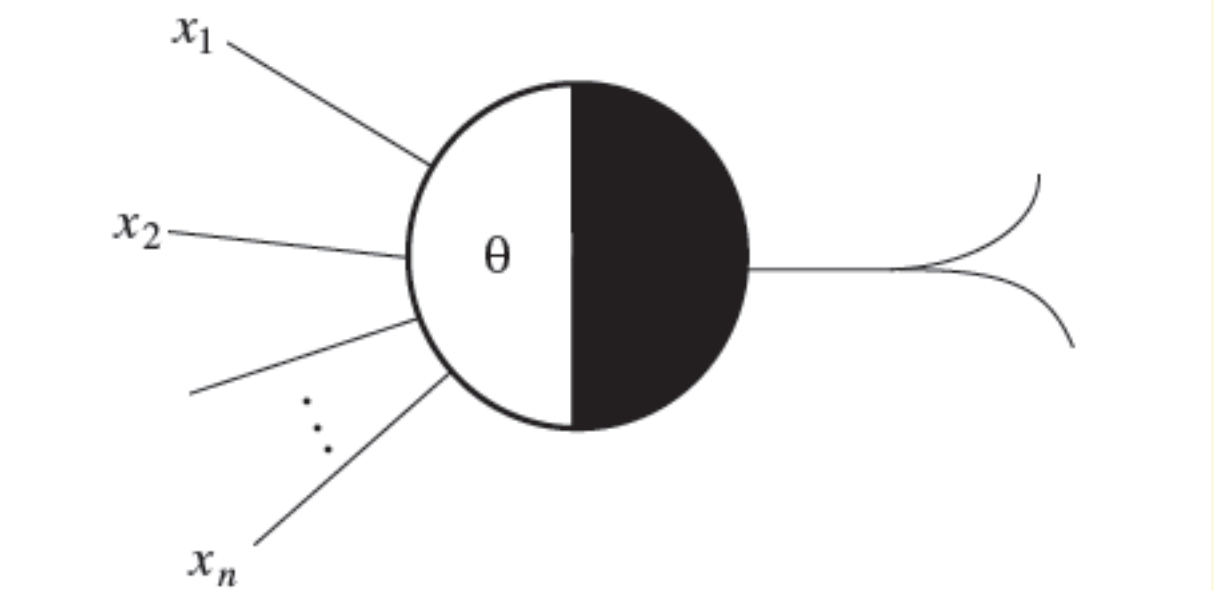


Fig. 2.6. Diagram of a McCulloch–Pitts unit

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

McCulloch-Pitts Neurons

- Binary values {0, 1}
- Unity connection weights of 1 and –1
- If an input to a neuron is 1 and the associated weight is –1, then the output of the neuron is 0
- Otherwise, if the weighted sum of input is not less than a threshold, then the output is 1; or is less than the threshold, then 0.

Threshold Logic

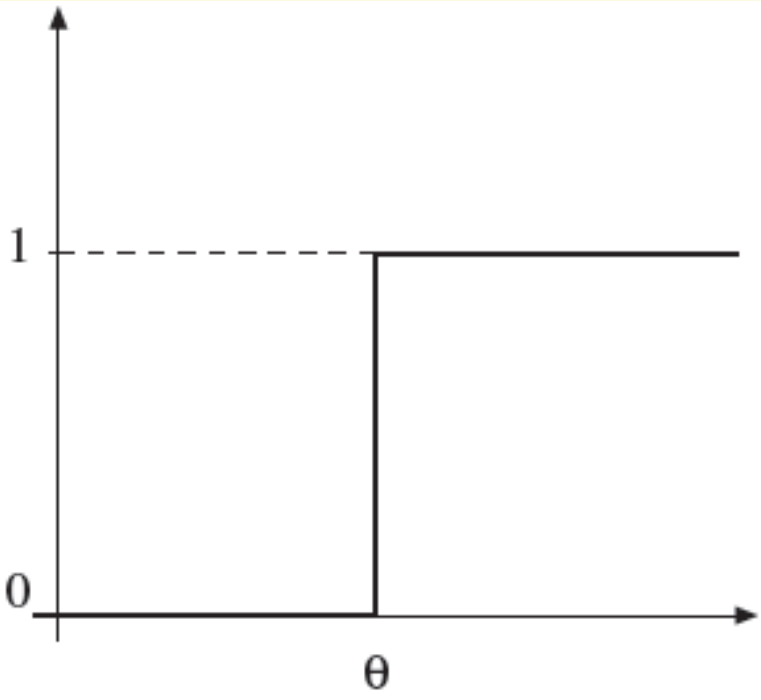


Fig. 2.7. The step function with threshold θ

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

AND & OR Gates

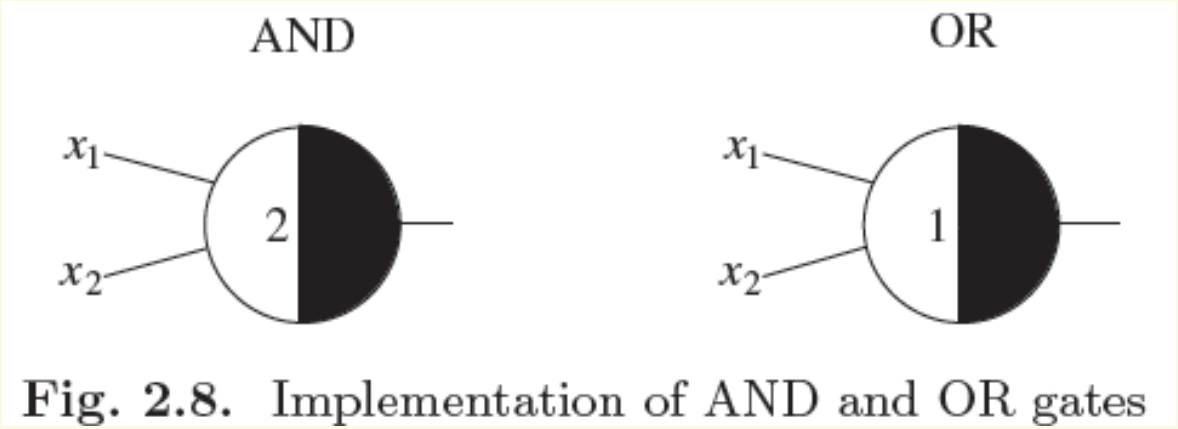


Fig. 2.8. Implementation of AND and OR gates

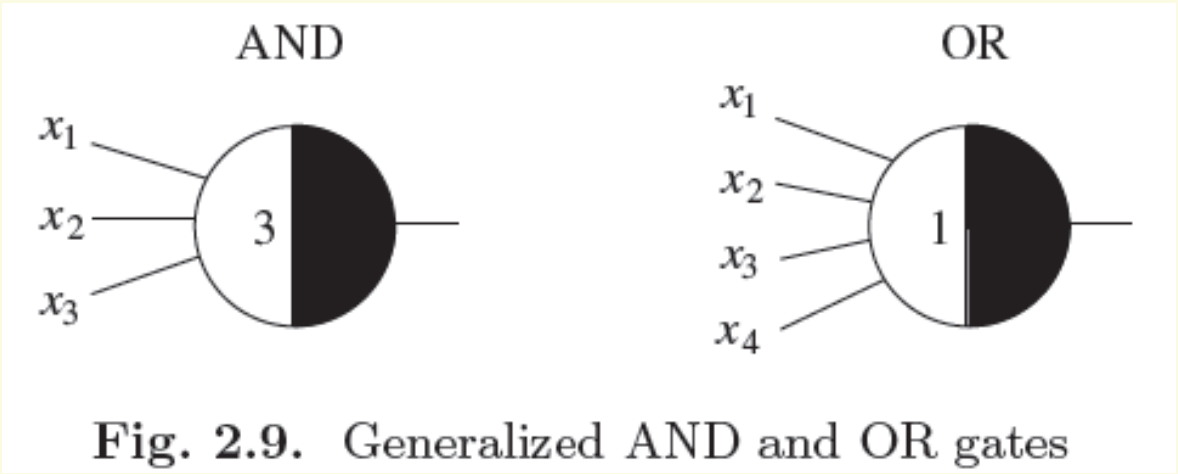


Fig. 2.9. Generalized AND and OR gates

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

AND & OR Gates

x1	x2	z	y
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

x1	x2	z	y
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Other Logic Functions

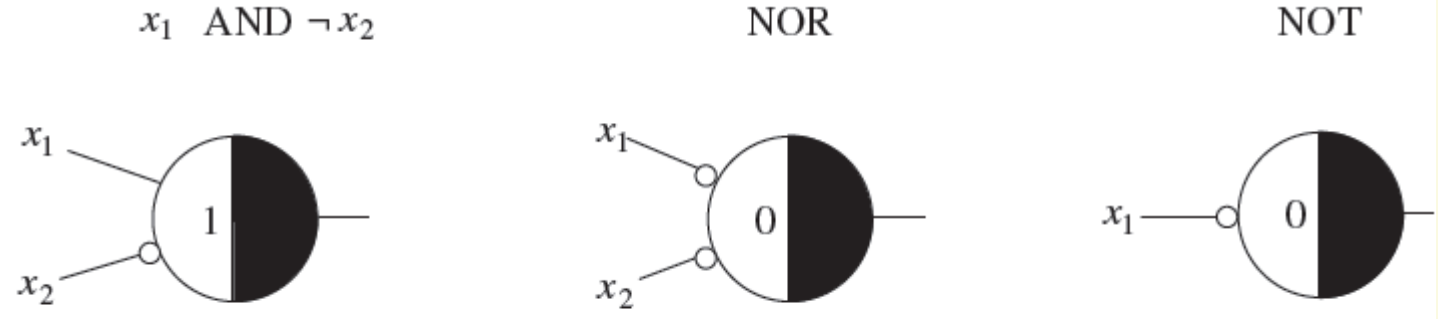


Fig. 2.10. Logical functions and their realization

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

x1	x2	z	y
0	0	0	0
0	1	0	0
1	0	1	1
1	1	0	0

x1	x2	z	y
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

x	z	y
0	1	1
1	0	0

A Decoder

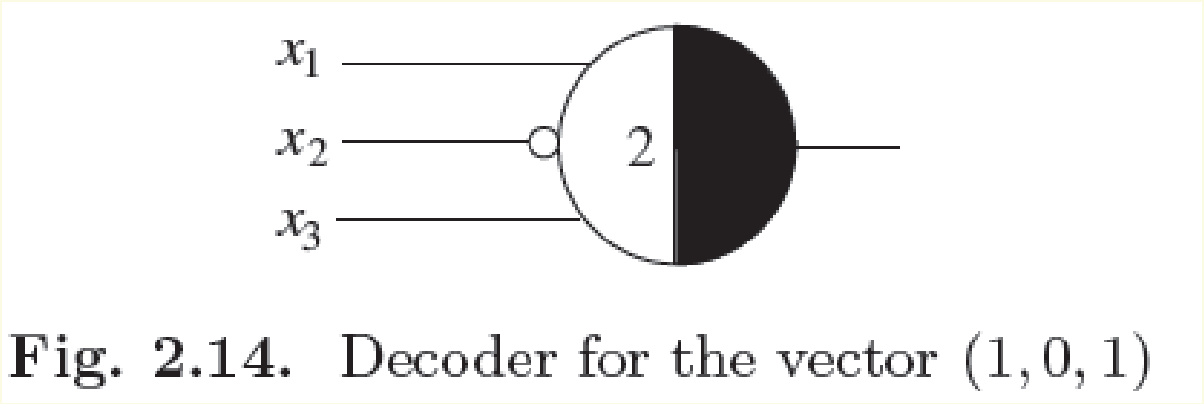


Fig. 2.14. Decoder for the vector $(1, 0, 1)$

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

x1	x2	x3	z	y
1	0	1	1	1
Otherwise			0	0

Another Decoder

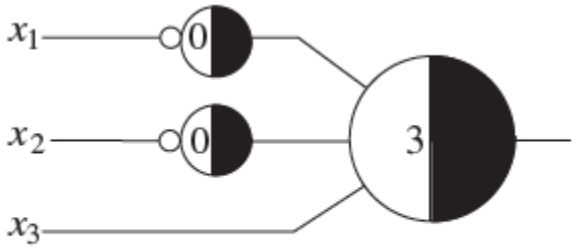


Fig. 2.16. A composite decoder for the vector $(0, 0, 1)$

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

x1	x2	x3	z	y
0	0	1	1	1
Otherwise			0	0

Another Decoder

input vectors	F
(0,0,1)	1
(0,1,0)	1
all others	0

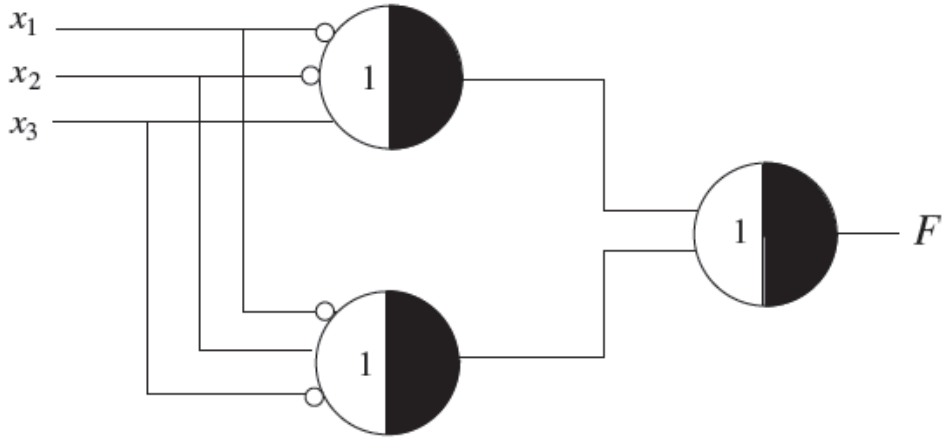


Fig. 2.15. Synthesis of the function F

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Threshold Logic Units

Proposition: Any logical function F: $\{0, 1\}^n$ -> $\{0, 1\}$ can be implemented with a two-layer McCulloch-Pitts network.

Proposition: Uninhibited threshold logic units of McCulloch-Pitts type can only implement monotonic logical functions.

Weighted Connections

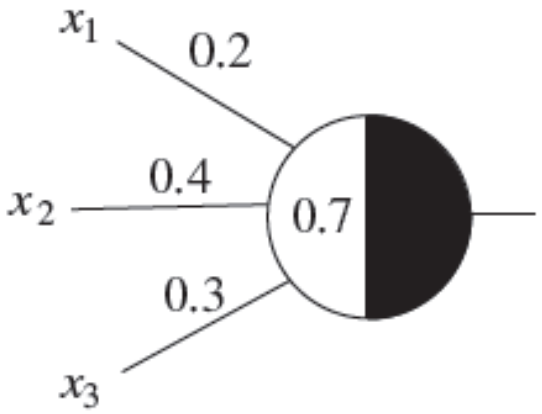


Fig. 2.17. Weighted unit

$0.2x_1 + 0.4x_2 + 0.3x_3 = 0.7$

$2x_1 + 4x_2 + 3x_3 = 7$

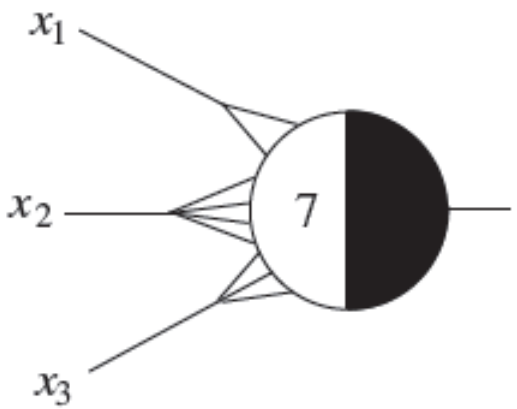


Fig. 2.18. Equivalent computing unit

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

A Recurrent Network

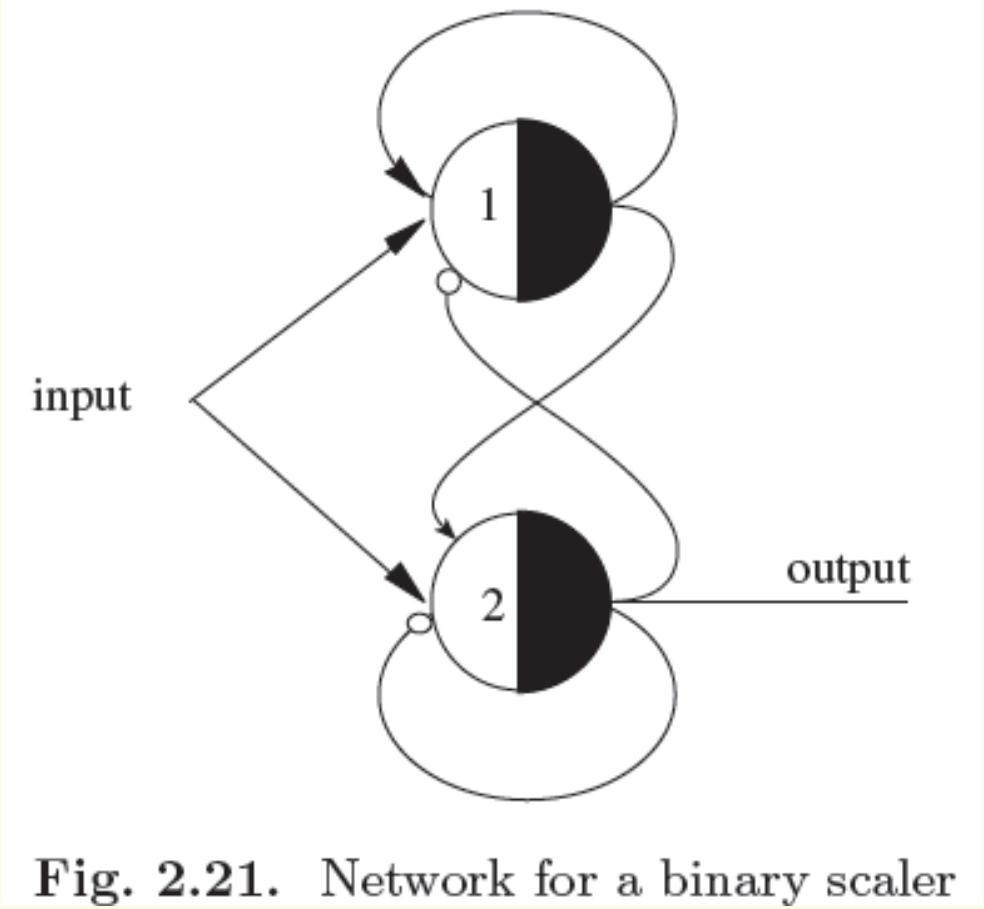


Fig. 2.21. Network for a binary scaler

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Finite Automata

An automaton is an abstract device capable of assuming different states which change according to the received input and previous states.

A finite automaton can take only a finite set of possible states and can react to only a finite set of input signals.

Finite Automata & Recurrent Networks

Proposition: Any finite automaton can be simulated with a recurrent network of McCulloch-Pitts units.

Perceptron

- A single adaptive layer of a feedforward network of pure threshold logic units.
- Developed by Rosenblatt at Cornell University in the late ’50s.
- Trained for pattern classification.
- First working model implemented in electronic hardware.

Perceptron

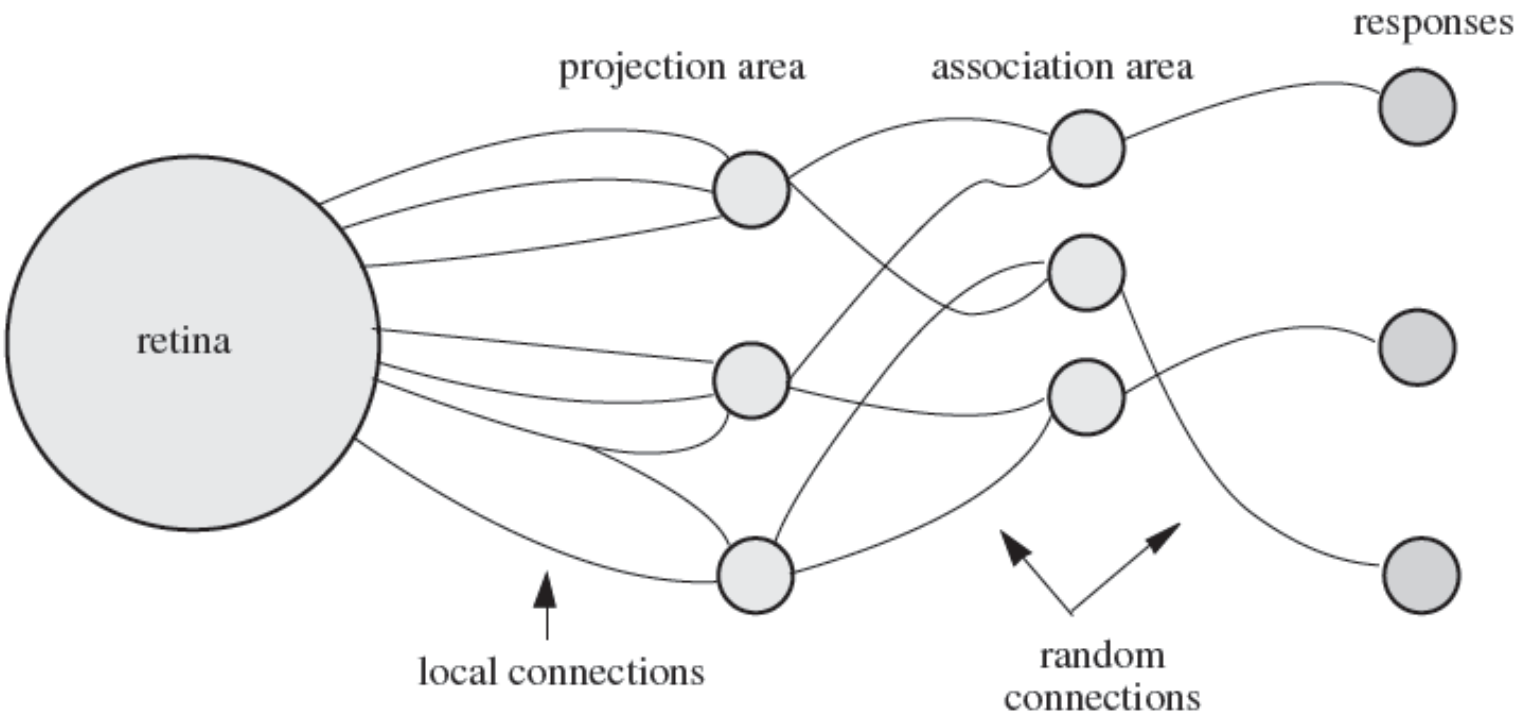


Fig. 3.1. The classical perceptron [after Rosenblatt 1958]

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Perceptron

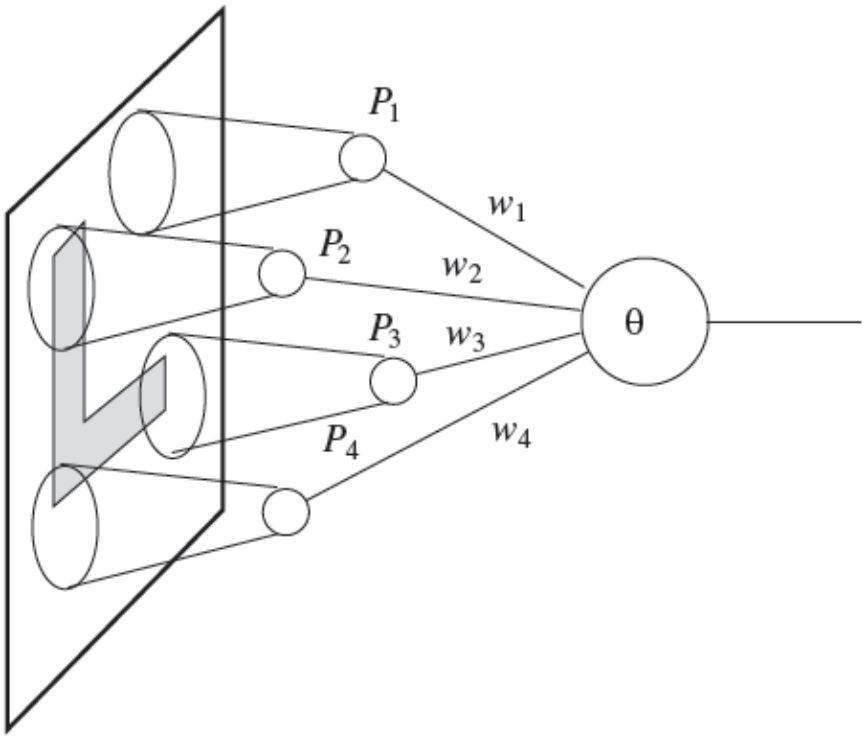


Fig. 3.2. Predicates and weights of a perceptron

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Simple Perceptron

A simple perceptron is a computing device with a threshold logic unit. When receiving *n* real inputs thru connections with *n* associated weights, a simple perceptron outputs 1 if the net input of weighted sum is not less than the threshold, and outputs 0 otherwise.

Linear Separability

Two sets of data in an *n*-dimensional space are said to be (absolutely) linearly separable if *n*+1 real weights (including a threshold) exist such that the weighted sum of a datum in one set is always greater than or equal to (greater than but not equal to) the threshold and that in the other set is always less than the threshold.

Absolute Linear Separability

If two finite sets of data are linearly separable, they are also absolutely linearly separable.

Perceptron Convergence Algorithm

1) Initialize weights and threshold randomly.
2) Calculate actual output of the perceptron:

   For all p

$$y^p = f\left(\sum_{i=1}^{n} w_i x_i^p - \theta\right)$$

1) Adapt weights: for all p

$$w_i(t+1) = w_i(t) + \eta(z^p - y^p)x_i^p, \eta > 0$$

2) Repeat until w converges.

Learning Rule

$$\underline{w}(t+1) = \underline{w}(t) + \eta(z^p - y^p)\underline{x}^p$$

where:

- $\underline{w}(t)$ is the weight vector at iteration t.
- η is the learning rate (set to 1 for simplicity here).
- z^p is the target output for sample p.
- y^p is the perceptron's predicted output for sample p.
- $\underline{x}^p$ is the input feature vector for sample p.

Iterative learning

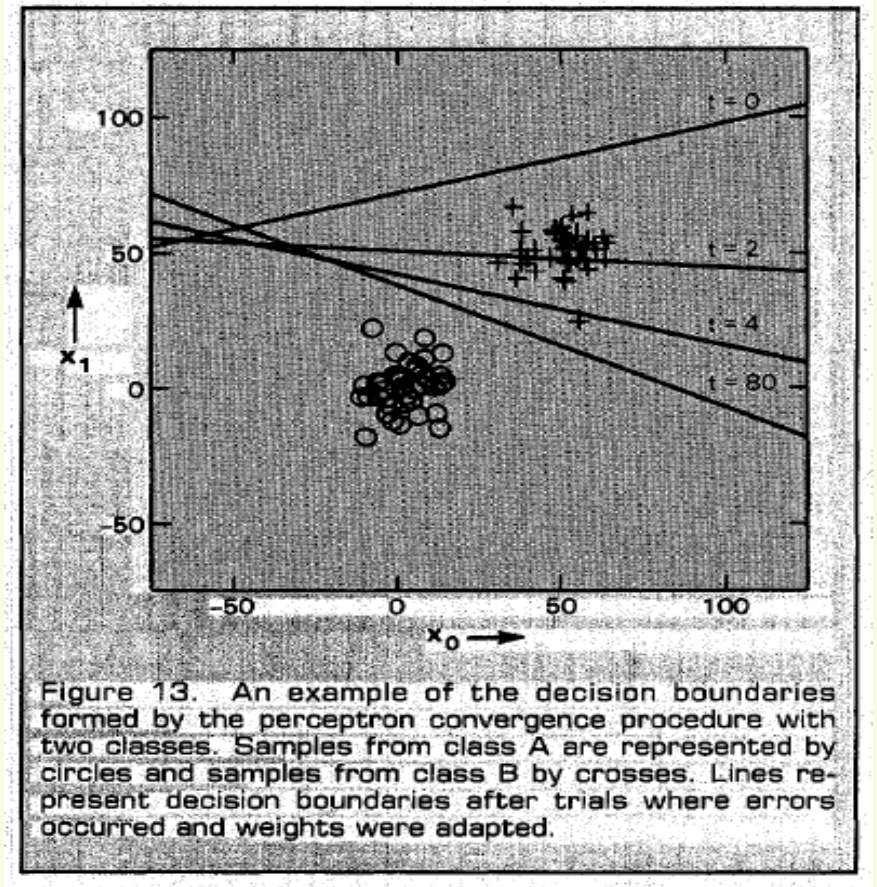


Figure 13. An example of the decision boundaries formed by the perceptron convergence procedure with two classes. Samples from class A are represented by circles and samples from class B by crosses. Lines represent decision boundaries after trials where errors occurred and weights were adapted.

Ref: R. P. Lippmann, "An Introduction to Computing with Neural Nets," *IEEE ASSP Magazine*, vol. 4, no. 2, pp. 4–22, Apr. 1987.

A Simple Case

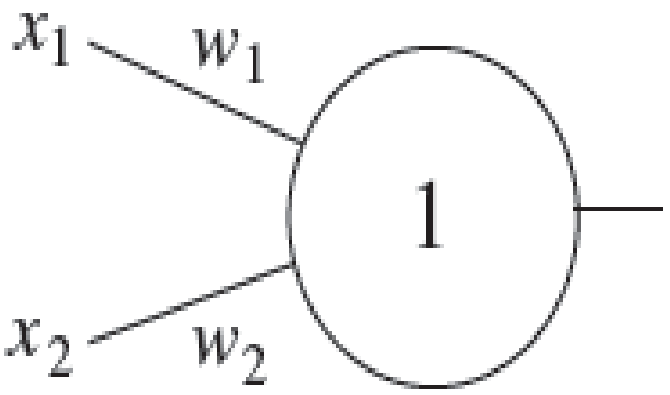


Fig. 4.4. Perceptron with constant threshold

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Error Landscape

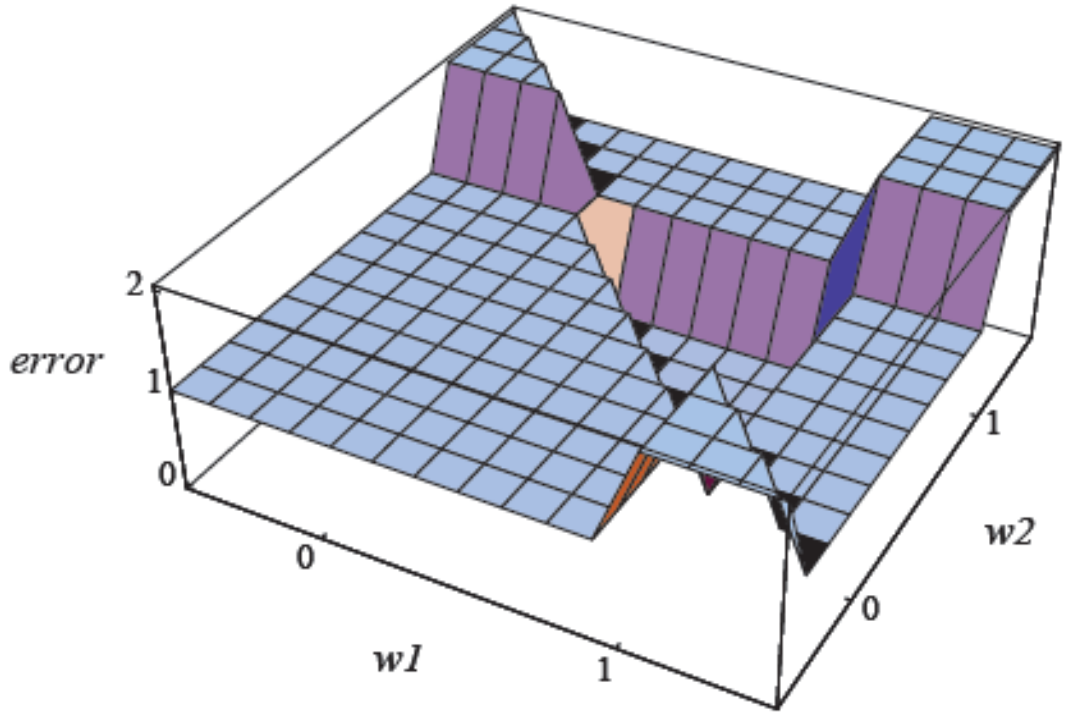


Fig. 4.5. Error function for the AND function

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Learning Process

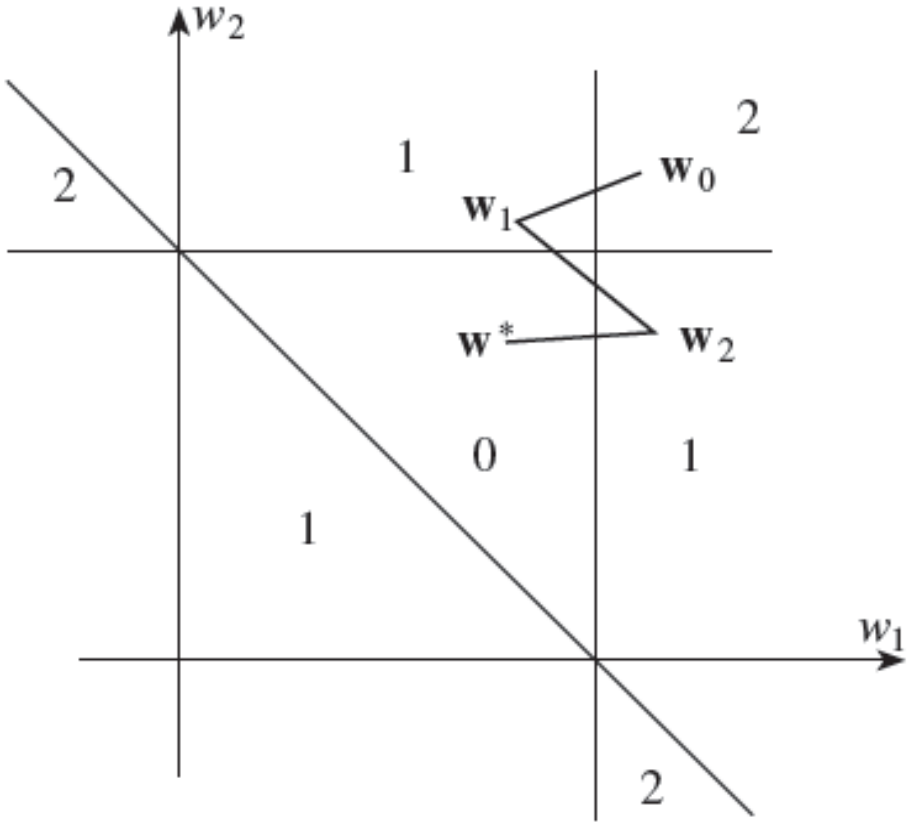


Fig. 4.6. Iteration steps to the region of minimal error

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Threshold

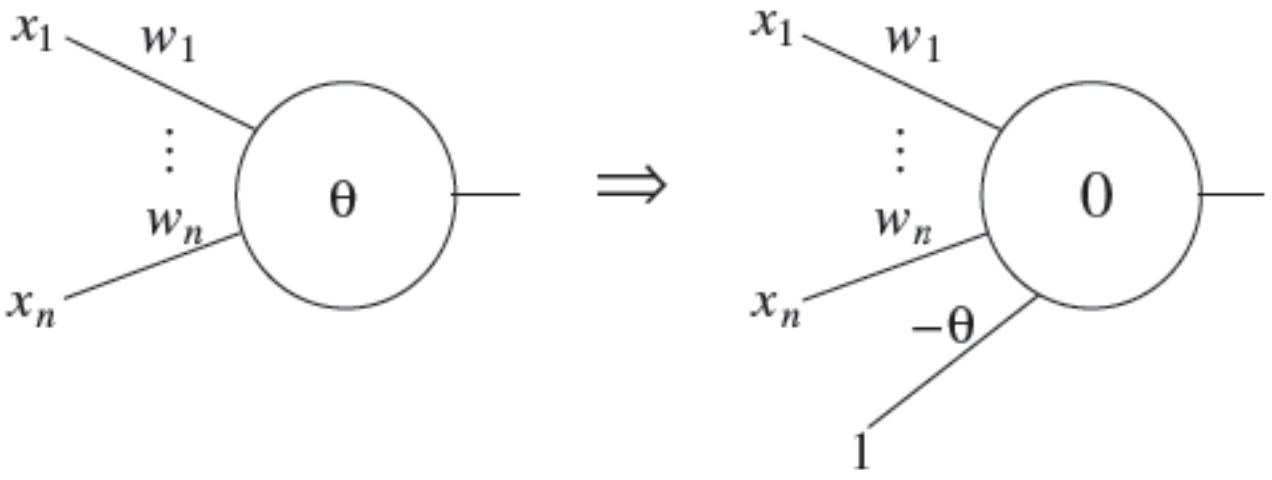


Fig. 3.5. A perceptron with a bias

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Perceptron Convergence Theorem

If two sets of data are linearly separable, the perceptron learning algorithm converge to a set of weights and a threshold in a finite steps.

Truth Table for OR Function

Sample p	x_1^p	x_2^p	Target z^p
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

Iteration 1

Let $\eta = 1$ and initial weights $\underline{w}(0) = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$.

For sample $p = 1, 2, 3, 4$:

- Weighted sum: $u^p = 0 \cdot 0 + 0 \cdot 0 + 0 = 0 \geq 0$

  $\Longrightarrow$ predicted $y^p = 1$.

- Target $z^1 = 0$ (error: $z^1 \neq y^1$).

- Weight update:

$$\underline{w}(1) = \underline{w}(0) + (z^1 - y^1)x^1 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} + (0 - 1) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix}$$

Iteration 2

Weights $\underline{w}(1) = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix}$.

For sample $p = 1, 2, 3, 4$:

- Weighted sum: $u^2 = 0 \cdot 0 + 0 \cdot 1 + (-1) = -1 < 0$

  $\Longrightarrow$ predicted $y^p = 0, p = 1, 2, 3, 4$.

- However, target $z^2 = z^3 = z^4 = 1$ (errors: $z^p \neq y^p, p = 2, 3, 4$).

- Weight update:

$$\underline{w}(2) = \underline{w}(1) + (z^2 - y^2)x^2 = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix} + (1 - 0) \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

Iteration 3

Weights $\underline{w}(2) = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$

For sample $p = 1$:

- Weighted sum: $u^1 = 0 \cdot 0 + 1 \cdot 0 + 0 = 0 \geq 0$

  $\Longrightarrow$ predicted $y^1 = 1$.
- However, target $z^1 = 0$ (error; $z^1 \neq y^1$).
- Weight update:

$$\underline{w}(3) = \underline{w}(2) + (z^1 - y^1)x^1 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} + (0 - 1) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}$$

Iteration 4

Weights $\underline{w}(3) = \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}$

For sample $p = 3$:

- Weighted sum: $u^3 = 0 \cdot 1 + 1 \cdot 0 + (-1) = -1 < 0$

  $\Longrightarrow$ predicted $y^3 = 0$.

- However, target $z^3 = 1$ (misclassification: $z^3 \neq y^3$).

- Weight update:

$$\underline{w}(4) = \underline{w}(3) + (z^3 - y^3)x^3 = \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix} + (1 - 0) \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

Iteration 5

Weights $\underline{w}(4) = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$

For sample $p = 1$:

- Weighted sum: $u^1 = 1 \cdot 0 + 1 \cdot 0 + 0 = 0 \geq 0$
  $\Longrightarrow$ predicted $y^1 = 1$.
- However, target $z^1 = 0$ (misclassification: $z^1 \neq y^1$).
- Weight update:

$$\underline{w}(5) = \underline{w}(4) + (z^1 - y^1)x^1 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} + (0 - 1) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}$$

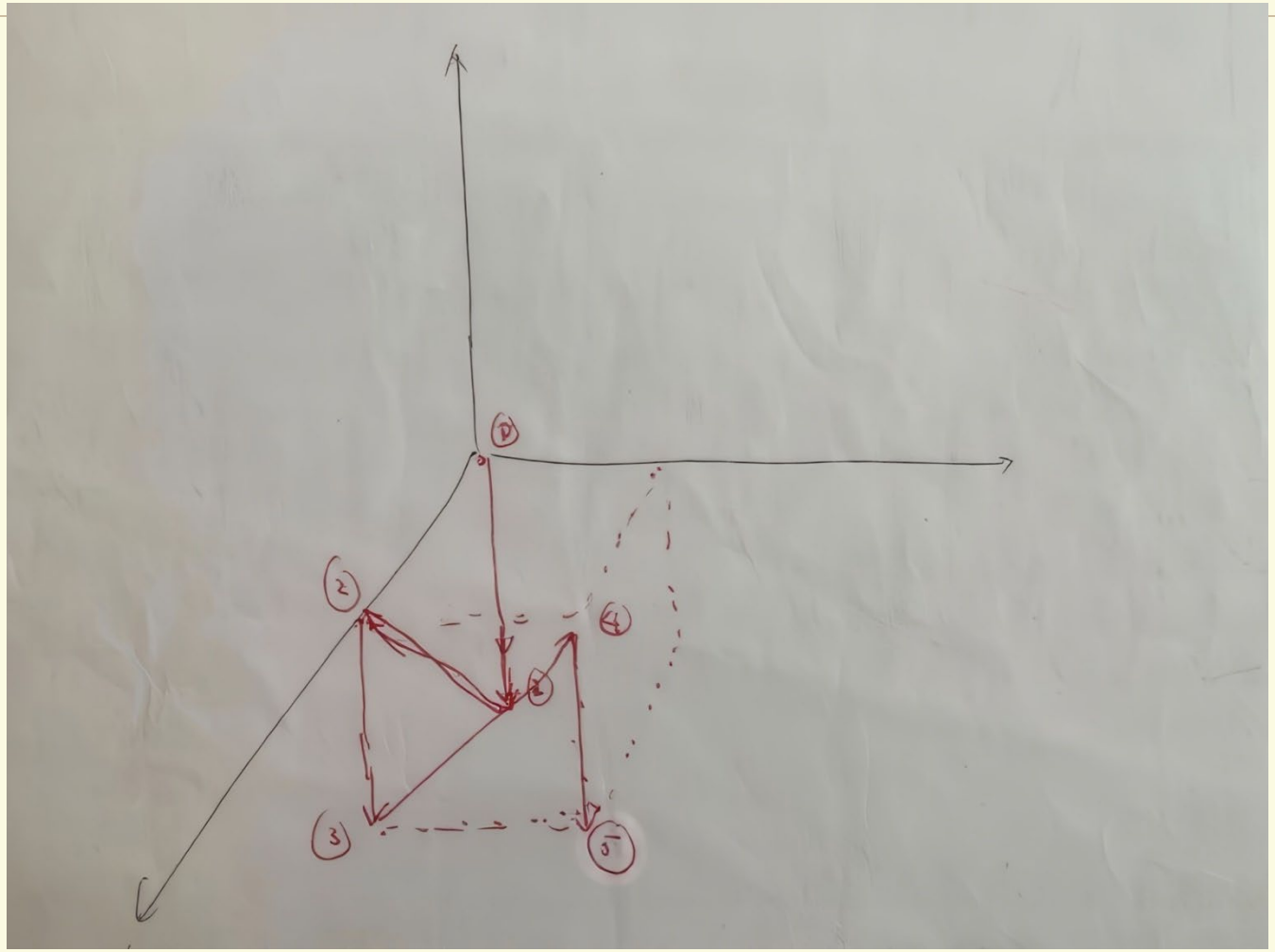
Finally

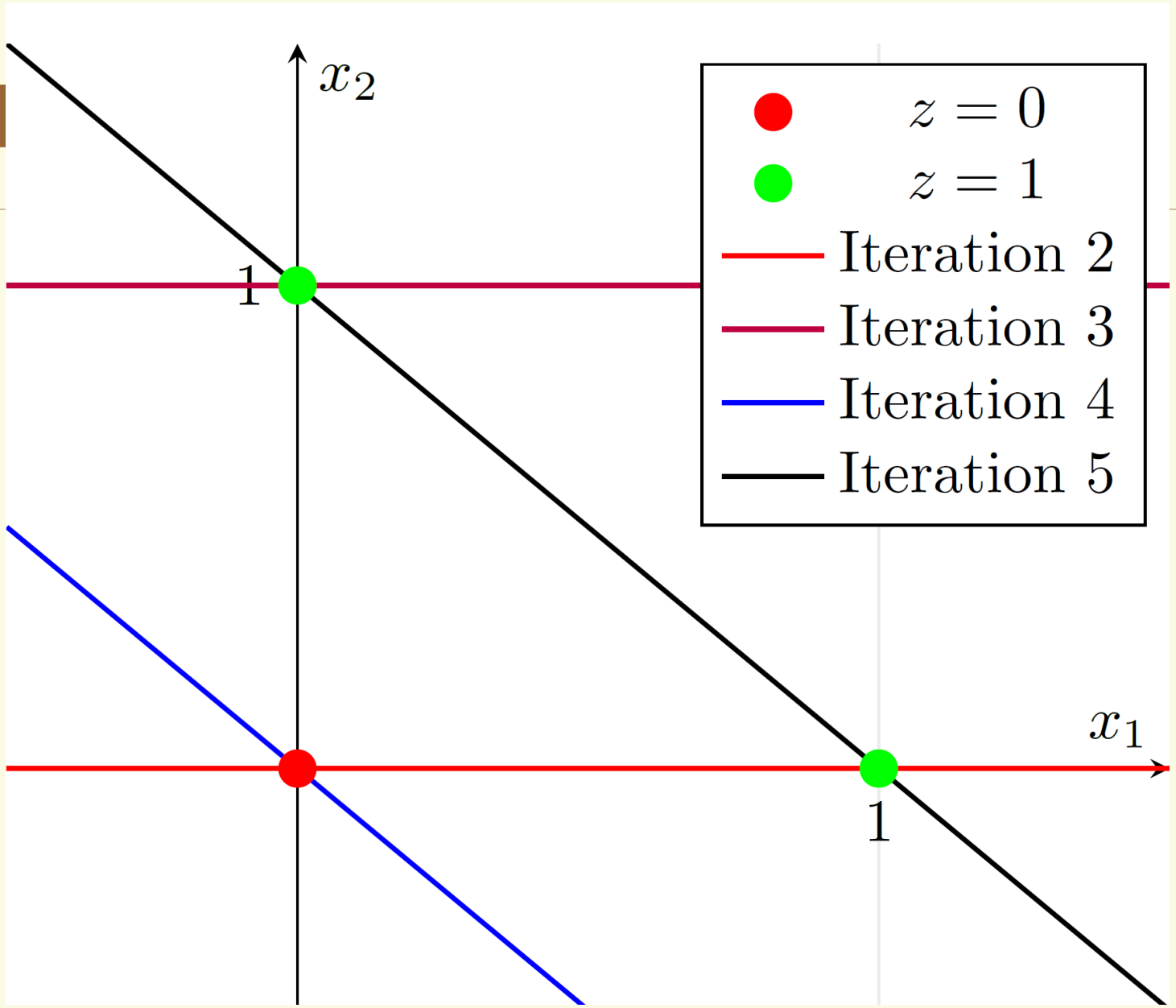
For sample $p = 1, 2, 3, 4$:

* Weighted sum: $u^p = 1 \cdot x_1^p + 1 \cdot x_2^p - 1$
  $\Longrightarrow$ predicted $y^1 = 0, y^2 = 1, y^3 = 1, y^4 = 1$.
* Targets $z^1 = 0, z^2 = 1, z^3 = 1, z^4 = 1$
  (no misclassification: $z^p = y^p, p = 1, 2, 3, 4$).

Converged!

Learning Trajectory



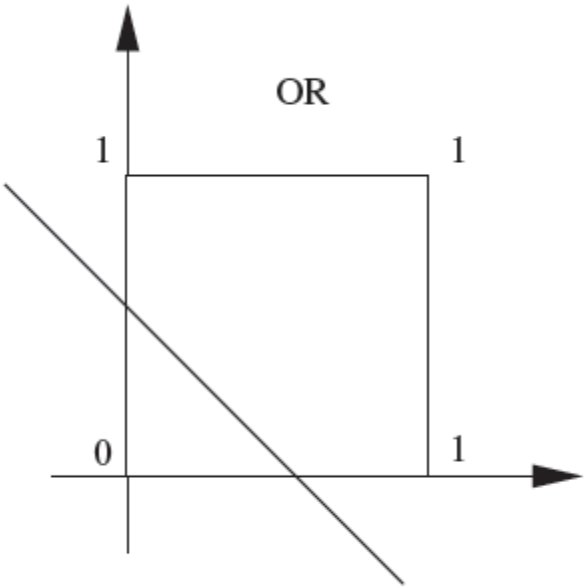

x_2
x_1
1
1
$z = 0$
$z = 1$
Iteration 2
Iteration 3
Iteration 4
Iteration 5

Two-Variable Logic Functions

Table 3.1. The 16 Boolean functions of two variables

x_1	x_2	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
0	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
0	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	0	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

2D OR and AND



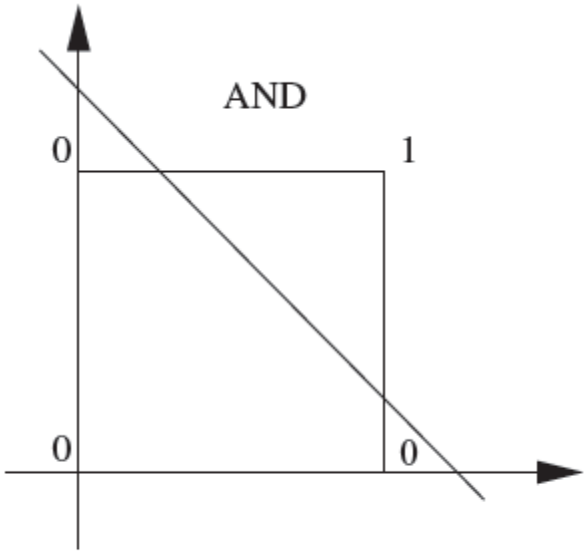


Fig. 3.6. Separations of input space corresponding to OR and AND

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

3D Logic Function

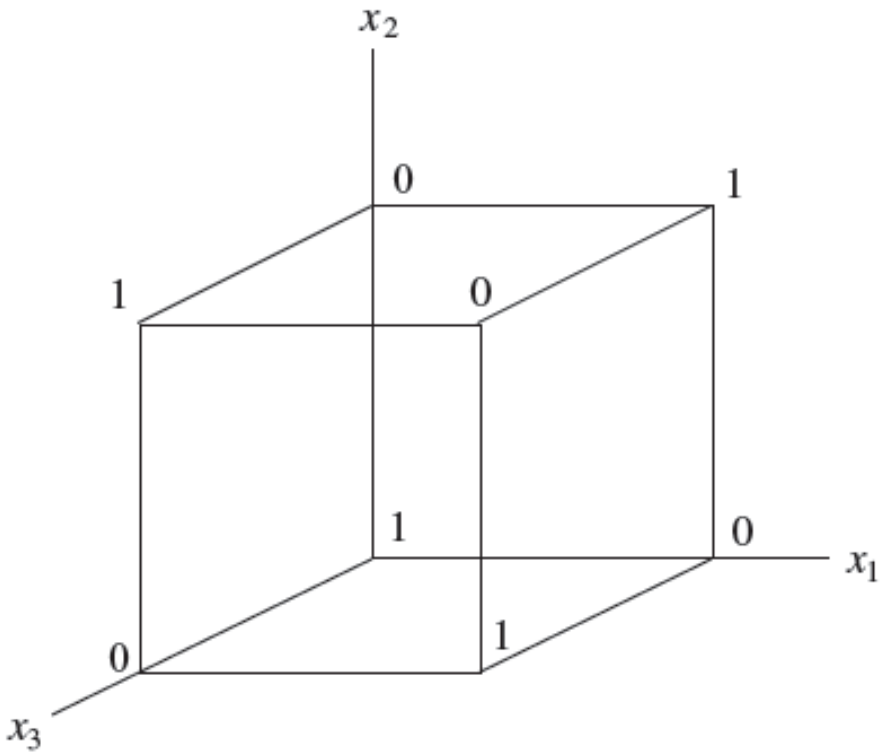


Fig. 2.11. Function values of a logical function of three variables

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

OR Function

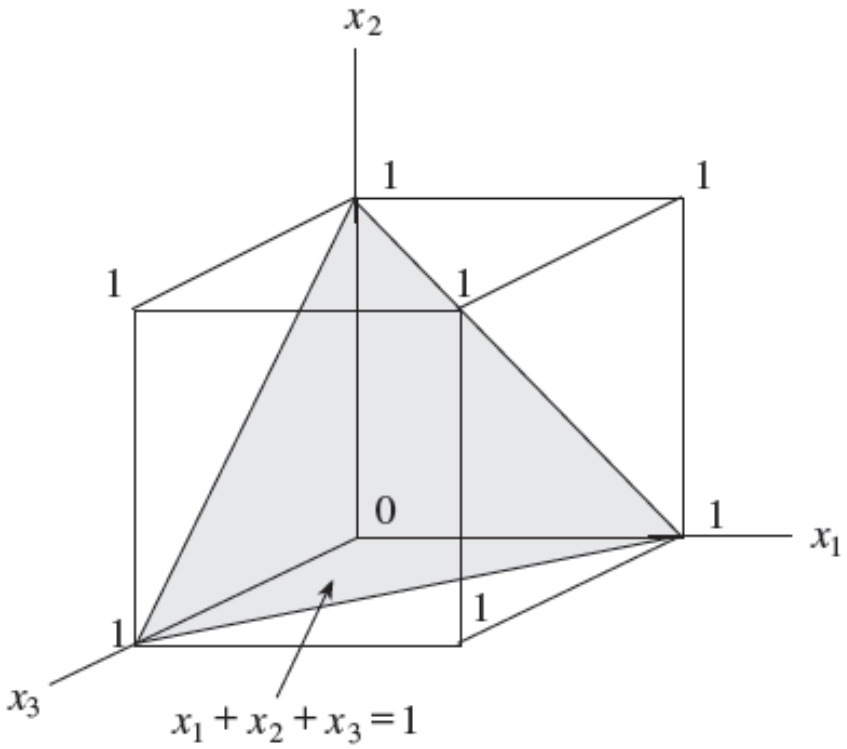


Fig. 2.12. Separation of the input space for the OR function

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Majority Function

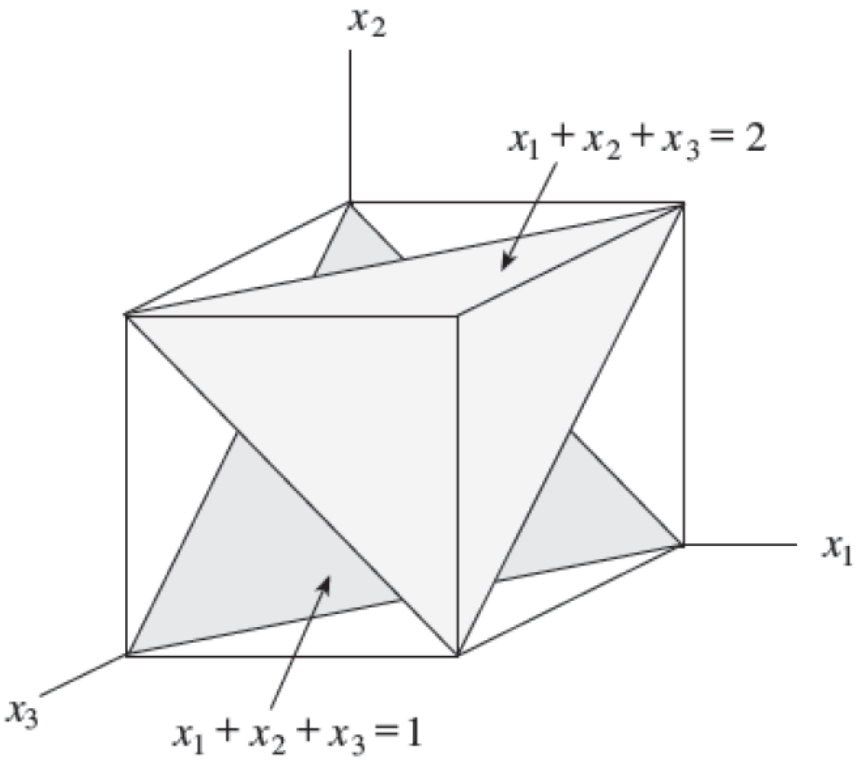


Fig. 2.13. Separating planes of the OR and majority functions

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Duality

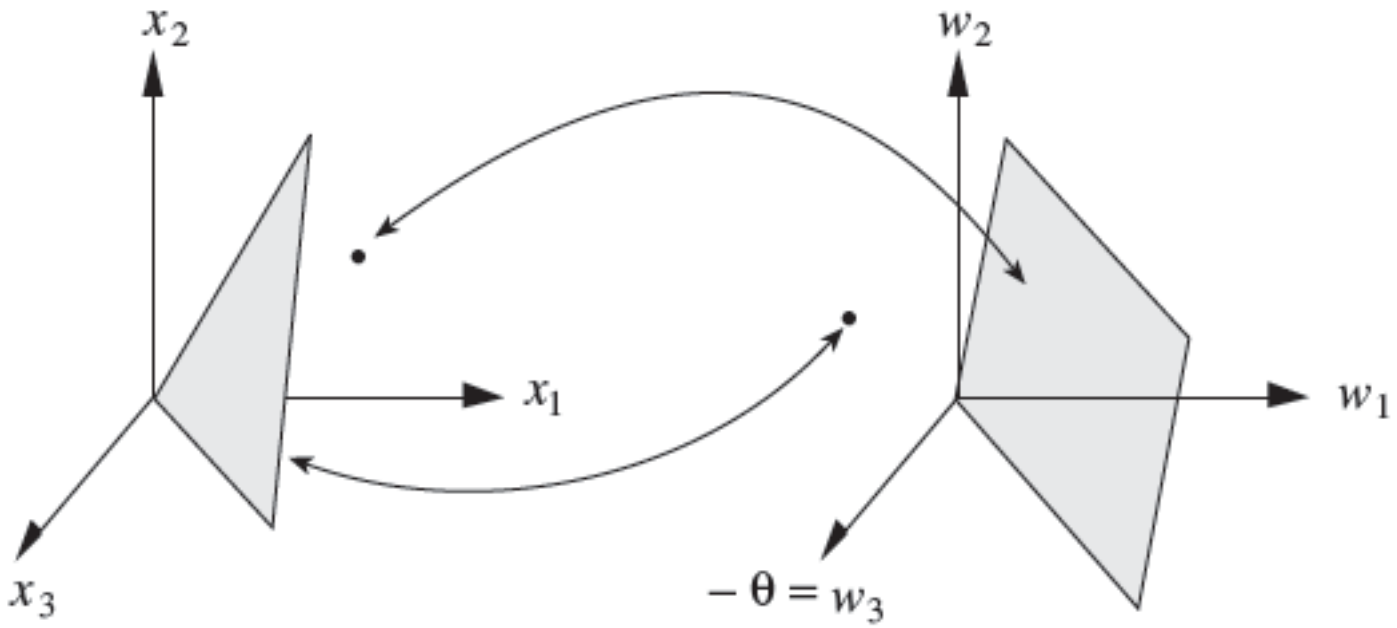


Fig. 3.7. Illustration of the duality of input and weight space

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

2D Weight Space

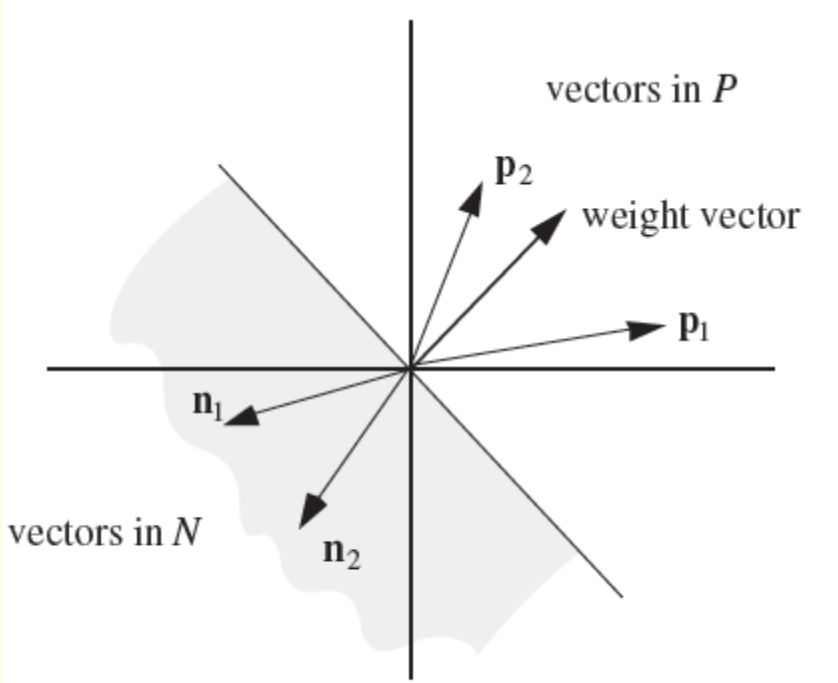


Fig. 4.8. Visualization in input space

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

2D Weight Space

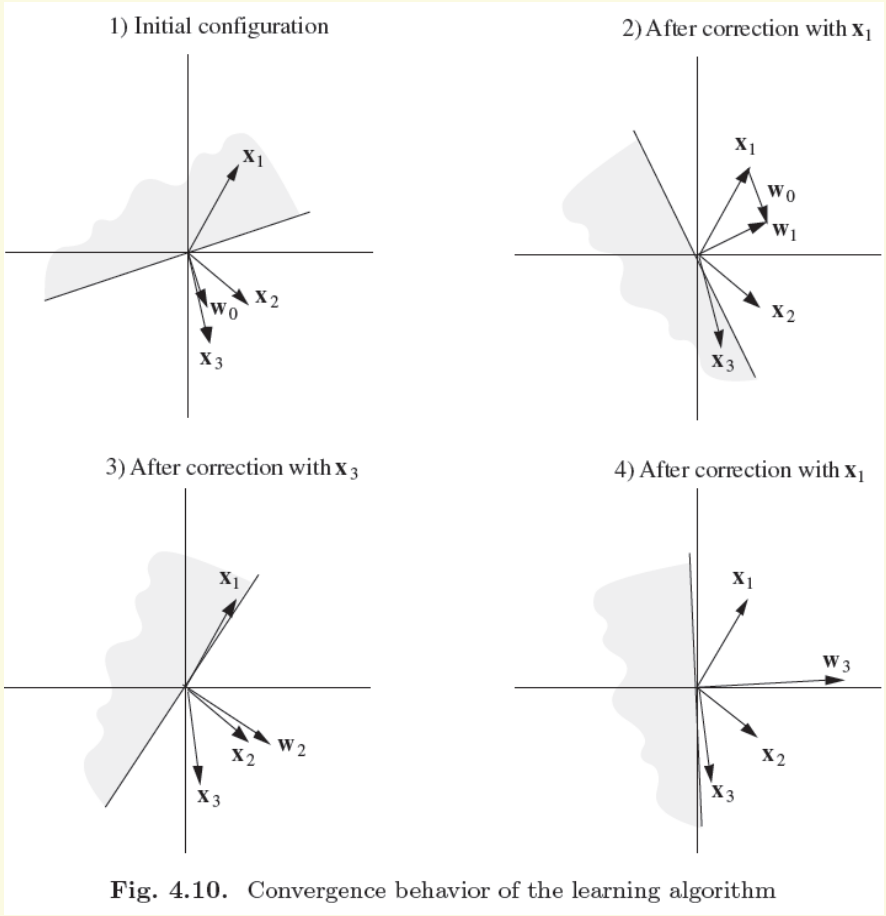


Fig. 4.10. Convergence behavior of the learning algorithm

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

XOR Problem

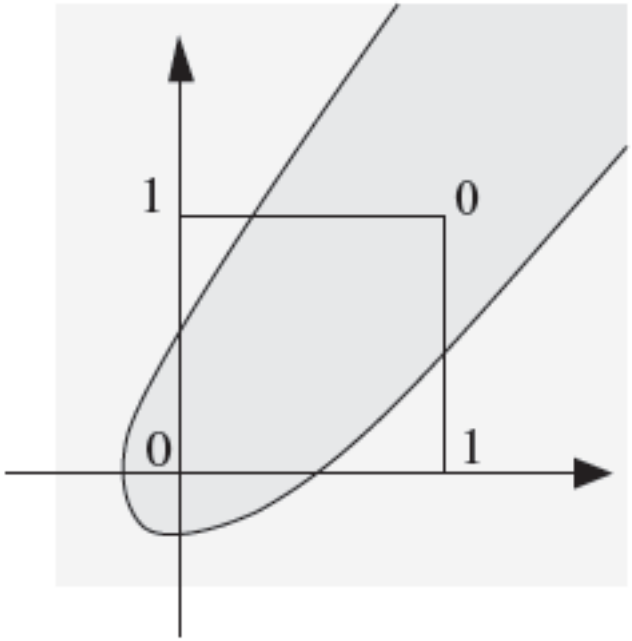


Fig. 3.8. Non-linear separation of input space

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Limitations of Perceptrons

- Only linearly separable data can be classified
- The convergence rate may be low for high-dimensional or large number of data.

Bipolar vs. Unipolar State Variables

- Unipolar: $v \in \{0,1\}$
- Bipolar: $v \in \{-1,1\}$

Bipolar coding of state variables is better than unipolar (binary) one in terms of algebraic structure, region proportion in weight space, etc.

Monte Carlo Tests

Table 6.1. Relative sizes of the regions on the Boolean sphere as percentage of the total surface

Coding	*Boolean function number*							
	0	1	2	3	4	5	6	7
binary	26.83	2.13	4.18	4.13	4.17	4.22	0.00	4.13
bipolar	8.33	6.29	6.26	8.32	6.24	8.36	0.00	6.22

Coding	*Boolean function number*							
	8	9	10	11	12	13	14	15
binary	4.28	0.00	4.26	4.17	4.17	4.14	2.07	27.12
bipolar	6.16	0.00	8.42	6.33	8.27	6.31	6.25	8.23

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

ADALINE

- A single adaptive layer of feedforward network of linear elements.
- Full name: Adaptive linear elements.
- Developed by Widrow and Hoff at Stanford University in early 60's.
- Trained using a learning algorithm called Delta Rule or Least Mean Squares (LMS) Algorithm.

LMS Learning Algorithm

1) Initialize weights and threshold randomly.
2) Calculate actual output of the ADALINE:

$$y = \alpha\left(\sum_{i=1}^{n} w_i x_i - \theta\right), \alpha > 0$$

3) Adapt weights:

$$w_i(t+1) = w_i(t) + \sum_{p} \eta(z^p - y^p)x_i^p, \eta > 0$$

4) Repeat until w converges

Gradient Descent Learning Algorithms

$$E = \sum_p E_p = \sum_p (z^p - y^p)^2$$

$$w(t+1) = w(t) - \eta \nabla E_w$$

$$\Delta w(t) = -\eta \nabla E_w$$

$$\nabla E_w = (\partial E / \partial w_1, \partial E / \partial w_2, \ldots, \partial E / \partial w_M)^T .$$

Training Modes

- Series mode: input training sample pairs one by one orderly or randomly.
- Batch mode: input training sample pairs in the whole training set at each iteration.
- Perceptron learning: either sequential or batch mode.
- ADALINE training: batch mode only.

Perceptron vs. Adaline

- Architecture: Perceptron uses bipolar or unipolar hardlimiter activation function, Adaline uses linear activation function.
- Learning rule: Perceptron learning algorithm is not gradient-descent and can operate in either sequential or batch training mode, whereas Adaline learning (LMS) algorithm is gradient descent, but can only operate in batch mode.

An Example

- Consider the XOR problem in the bipolor coding

p	x_1^p	x_2^p	z^p
1	0	0	-1
2	0	1	1
3	1	0	1
4	1	1	-1

An Example (cont'd)

The convergence implies:

$$\sum_{p=1}^{4}(z^p - y^p)x^p = 0$$

Specifically,

$$(-1 - w^T x^1)x^1 + (1 - w^T x^2)x^2 + (1 - w^T x^3)x^3 + (-1 - w^T x^4)x^4 = 0$$

An Example (cont'd)

$$\text{Substituting } x^1 = \begin{pmatrix}0\\0\\1\end{pmatrix},\ x^2 = \begin{pmatrix}0\\1\\1\end{pmatrix},\ x^3 = \begin{pmatrix}1\\0\\1\end{pmatrix},\ x^4 = \begin{pmatrix}1\\1\\1\end{pmatrix}$$

$$(-1-w_3)\begin{pmatrix}0\\0\\1\end{pmatrix}+(1-w_2-w_3)\begin{pmatrix}0\\1\\1\end{pmatrix}+(1-w_1-w_3)\begin{pmatrix}1\\0\\1\end{pmatrix}+(-1-w_1-w_2-w_3)\begin{pmatrix}1\\1\\1\end{pmatrix}=\begin{pmatrix}0\\0\\0\end{pmatrix}$$

Expanding and combining terms:

$$\begin{pmatrix}0\\0\\-1-w_3\end{pmatrix}+\begin{pmatrix}0\\1-w_2-w_3\\1-w_2-w_3\end{pmatrix}+\begin{pmatrix}1-w_1-w_3\\0\\1-w_1-w_3\end{pmatrix}+\begin{pmatrix}-1-w_1-w_2-w_3\\-1-w_1-w_2-w_3\\-1-w_1-w_2-w_3\end{pmatrix}=\begin{pmatrix}0\\0\\0\end{pmatrix}$$

An Example (cont'd)

It simplifies:

$$\begin{cases} -2w_1 - w_2 - 2w_3 = 0 \\ -w_1 - 2w_2 - 2w_3 = 0 \\ -2w_1 - 2w_2 - 4w_3 = 0 \end{cases}$$

The homogeneous linear equations can be written in a matrix equation $A\mathbf{w} = \mathbf{0}$, where:

$$A = \begin{pmatrix} 2 & 1 & 2 \\ 1 & 2 & 2 \\ 2 & 2 & 4 \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}, \quad \mathbf{0} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

An Example (cont'd)

Solving $A\mathbf{w} = \mathbf{0}$ gives the sole solution:

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Weight Space Regions Separated by Hyperplanes

- Each plane is defined by one training sample.
- One plane separates two (2) half-space.
- Two planes separate up to four (4) regions.
- Three planes separate up to eight (8) regions.
- However, four planes separate up to fourteen (14) regions only.

Number of Weight Space Regions

The number of different regions in weight space defined by m separating hyperplanes in n-dimensional weight space is a polynomial of degree n-1 on m:

$$R(m,n) = 2\sum_{i=0}^{n-1}\binom{m-1}{i} < 2\frac{m^n}{n!}$$

Number of Weight Space Regions

Table 6.2. Comparison of the number of Boolean and threshold functions of n inputs and two different bounds

n	2^{2^n}	$T(2^n, n)$	$R(2^n, n)$	$\lfloor 2^{n^2+1}/n! \rfloor$
1	4	4	4	4
2	16	14	14	16
3	256	104	128	170
4	65,536	1,882	3,882	5,461
5	4.3×10^9	94,572	412,736	559,240

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

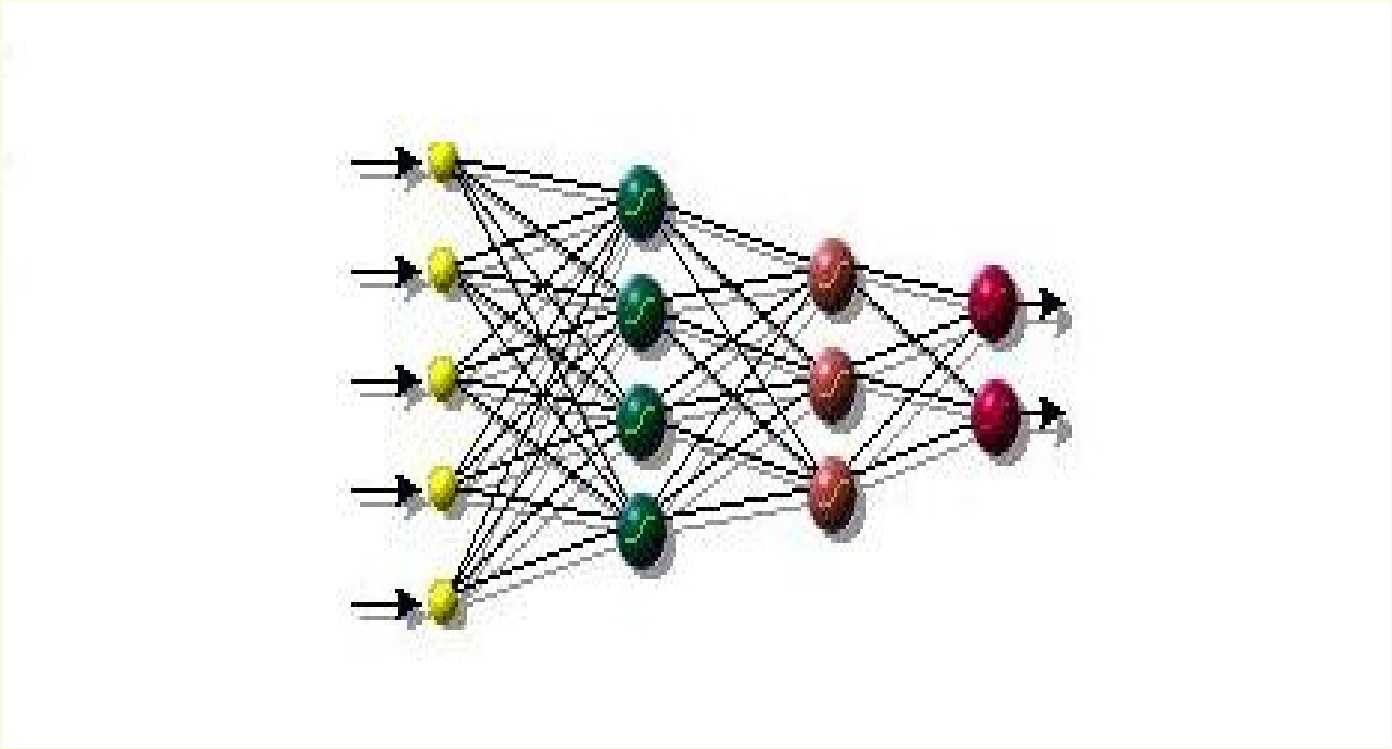
Number of Logic Functions vs. Number of Threshold Functions

- The number of threshold functions defined by hyperplanes is a function of $2^{n(n-1)}$ whereas that of logical functions is 2^{2^n}.
- The learnbability problem: when n is large, there is not enough classification regions in weight space to represent all logical functions.

Learnability Problems

- Solution existence in the weight space? Neither Perceptron nor Adaline can classify patterns with nonlinear distributions such as XOR. But two-layer Perceptron can classify XOR data.
- How to find the solution even though it exists in the weight space? It is known that multilayer Perceptron can classify arbitrary shape of data classes. But how to design learning algorithms to determine the weights?

Multilayer Feedforward Network



Multilayer Feedforward Network

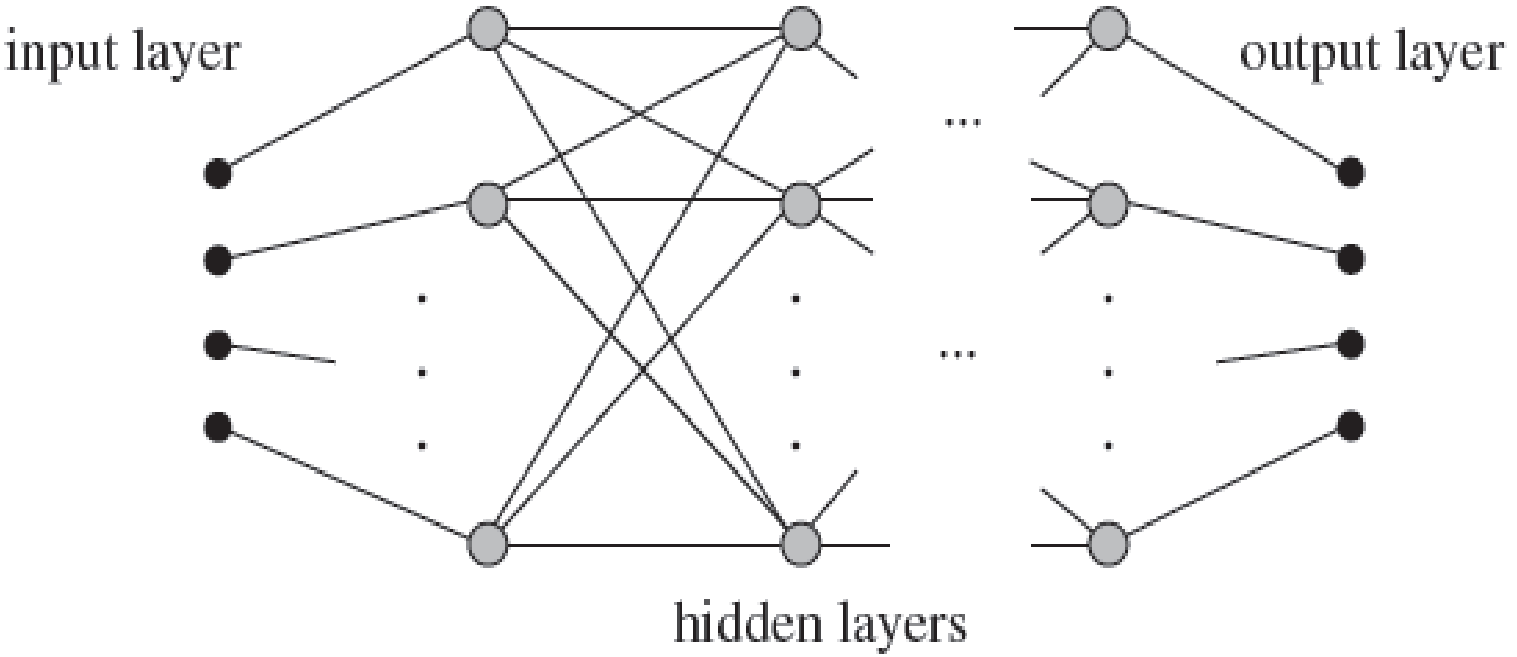


Fig. 6.1. A generic layered architecture

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

XOR Problem Solved

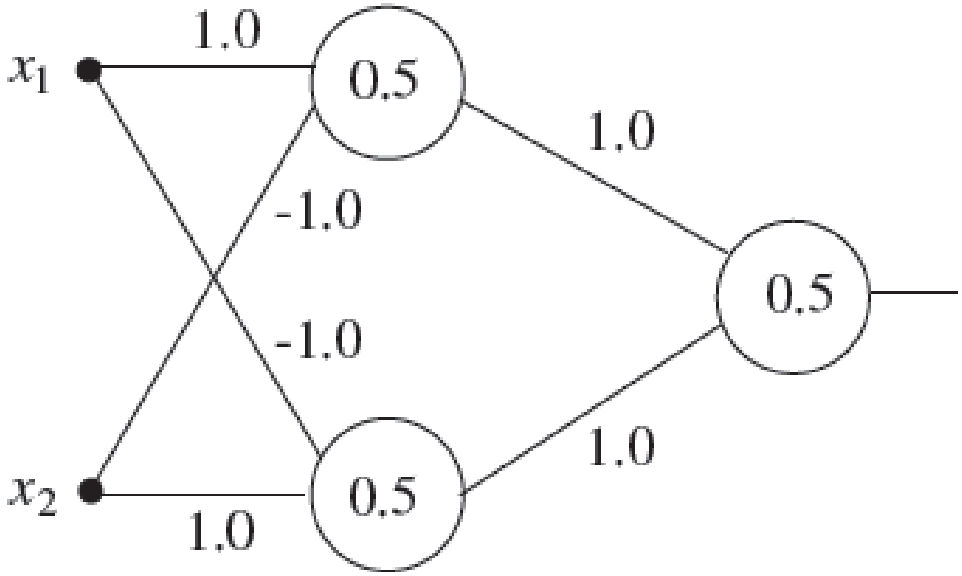


Fig. 6.2. A three-layered network for the computation of XOR

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Alternative Networks

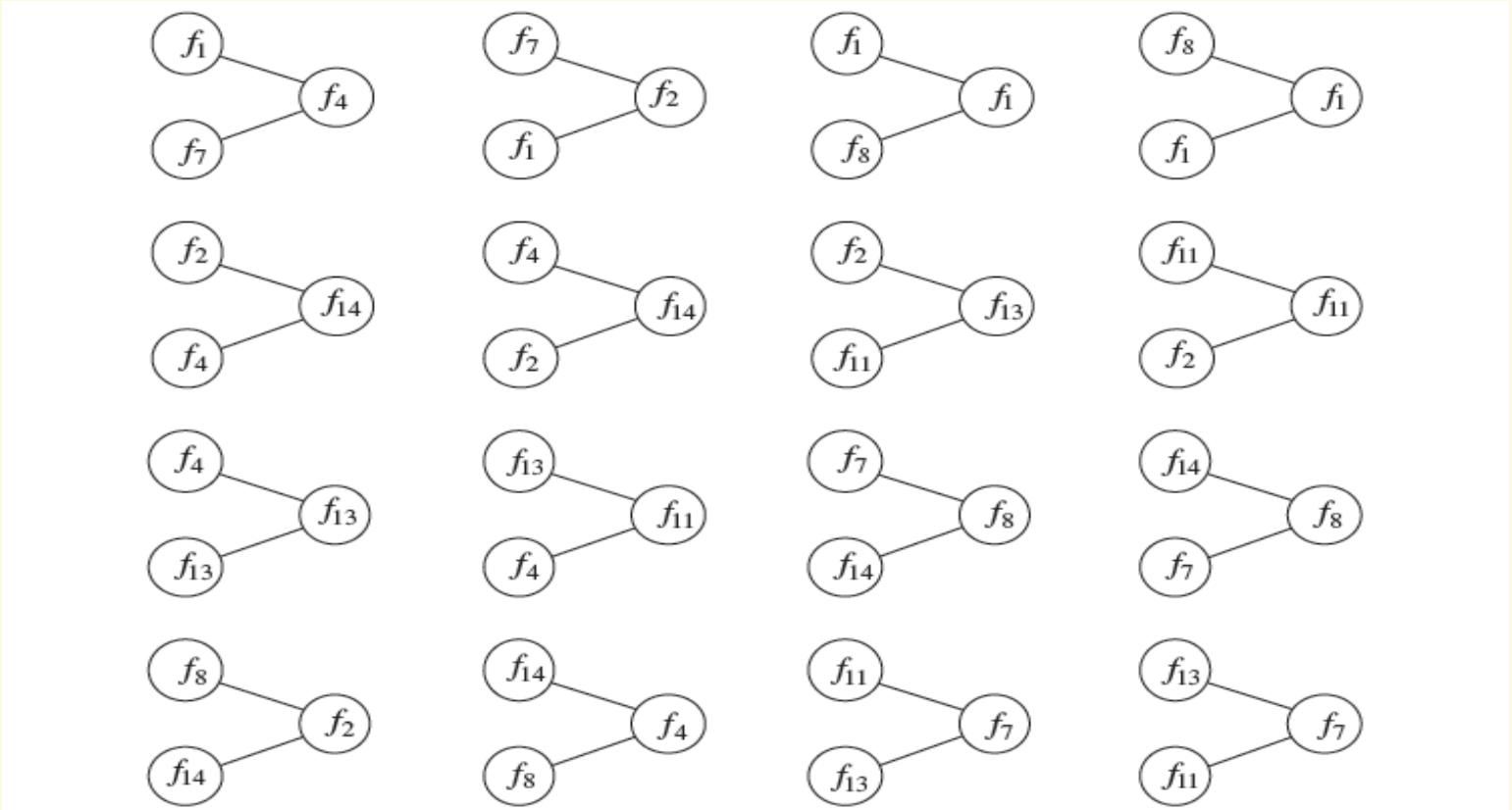


Fig. 6.3. The 16 solutions for the computation of XOR with three computing units

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Learning Process

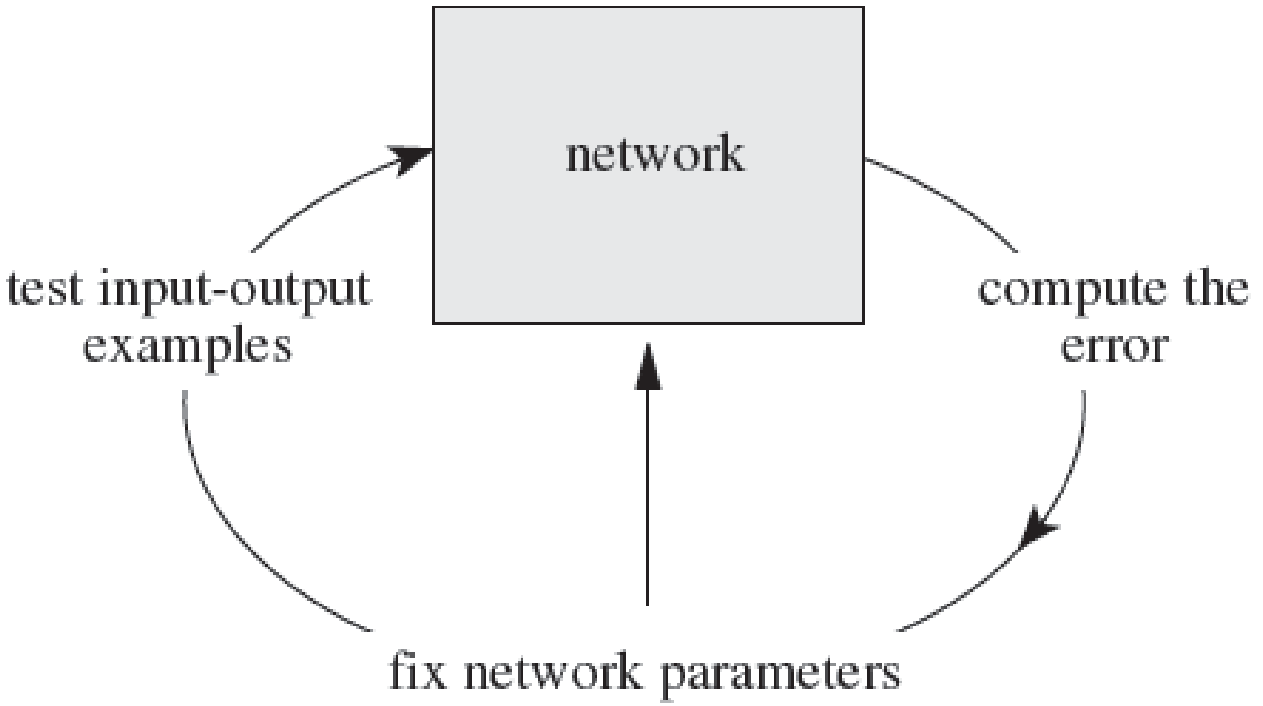


Fig. 4.1. Learning process in a parametric system

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Alternative Network

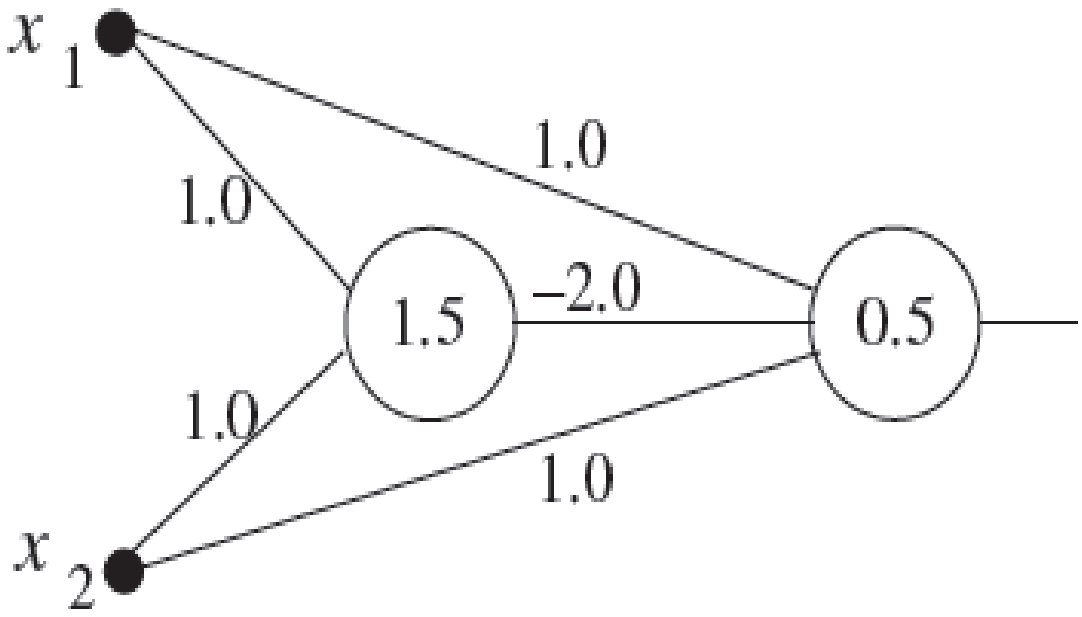


Fig. 6.4. Two unit network for the computation of XOR

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Two-layer Separation

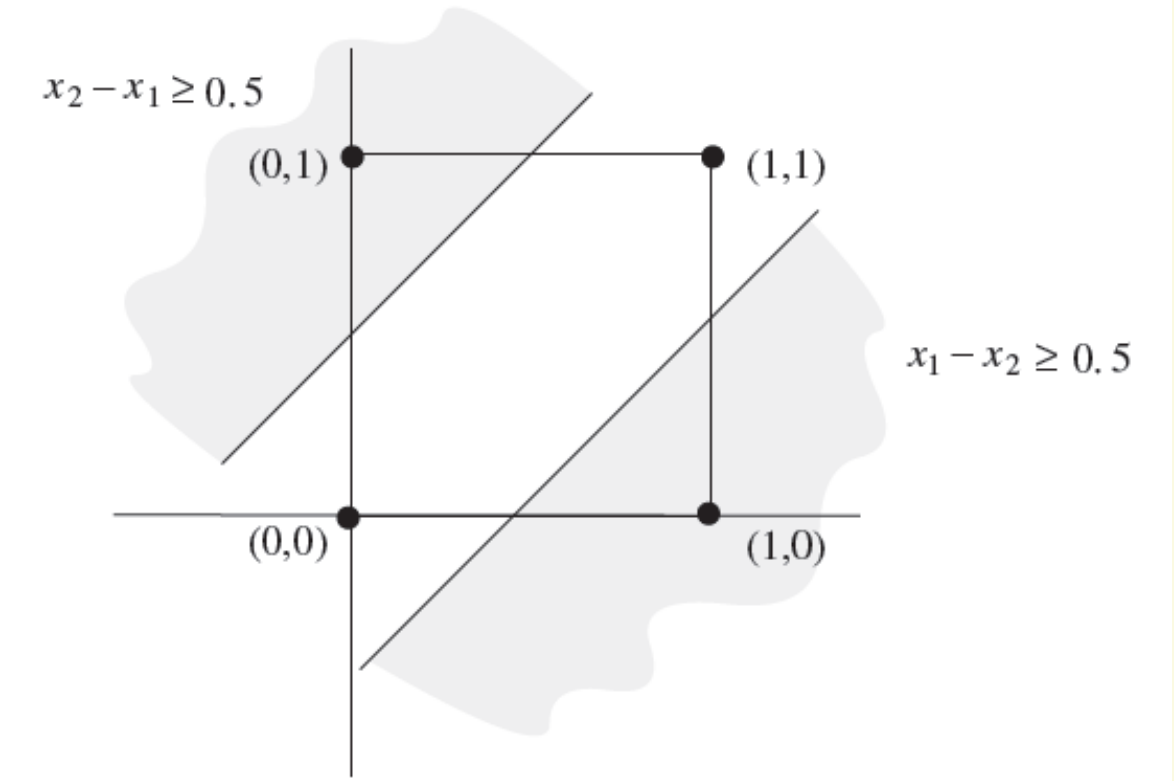


Fig. 6.5. Space separation defined by a two-layered network

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Two-layer Separation

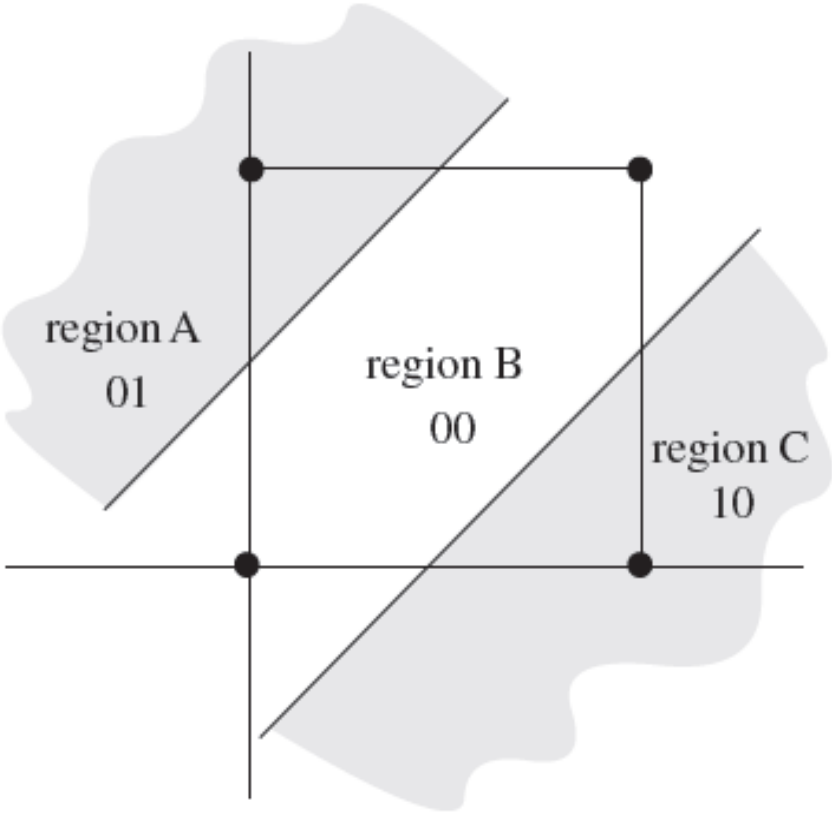


Fig. 6.6. Labeling of the regions in input space

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Backpropagation Algorithm

- Also known as generalized delta rule.
- Invented and reinvented by many researchers, popularized by the PDP group at UC San Diego in 1986.
- A recursive gradient-descent learning algorithm for multilayer feedforward networks of sigmoid activation function.
- Compute errors backward from the output layer to input layer.
- Minimze the mean squares error function.

Sigmoid Activation Functions

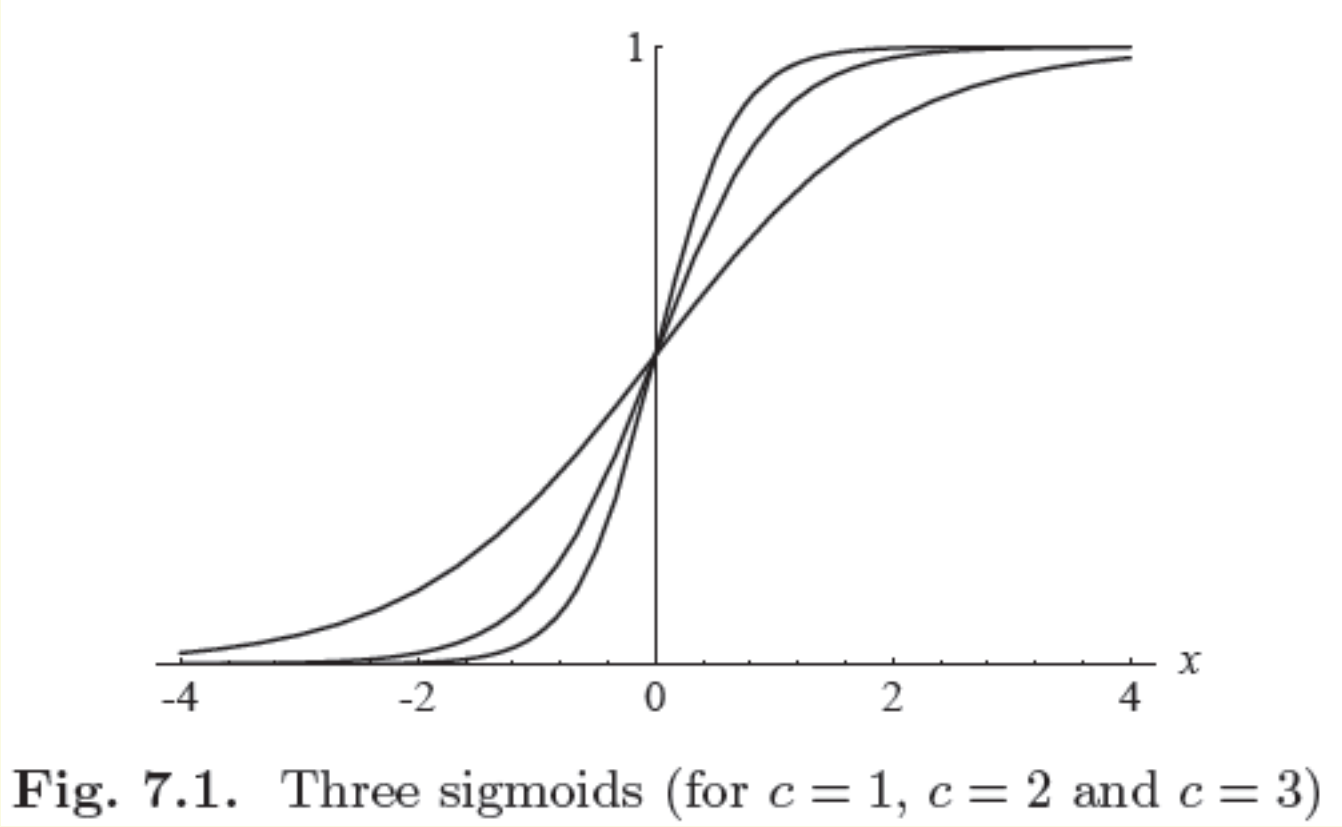


Fig. 7.1. Three sigmoids (for $c = 1$, $c = 2$ and $c = 3$)

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Sigmoid Activation Functions

- Unipolar: $$f(u) = \frac{1}{1+\exp(-u)}$$

$$\frac{df(u)}{du} = \frac{\exp(-u)}{(1+\exp(-u))^2} =$$

$$\frac{1}{1+\exp(-u)} \frac{1+\exp(-u)-1}{1+\exp(-u)} = f(u)[1-f(u)]$$

- Bipolar: $$f(u) = \tanh(u) = \frac{1-\exp(-u)}{1+\exp(-u)}$$

Related Activation Functions

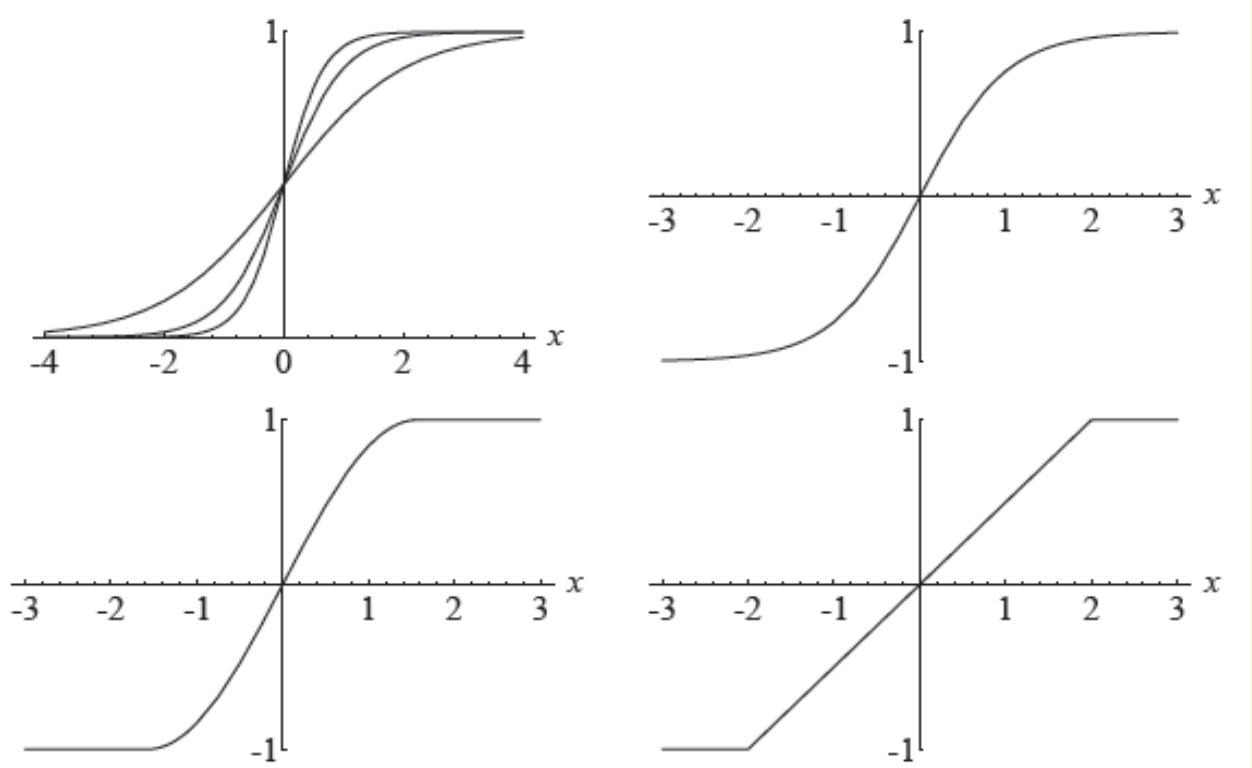
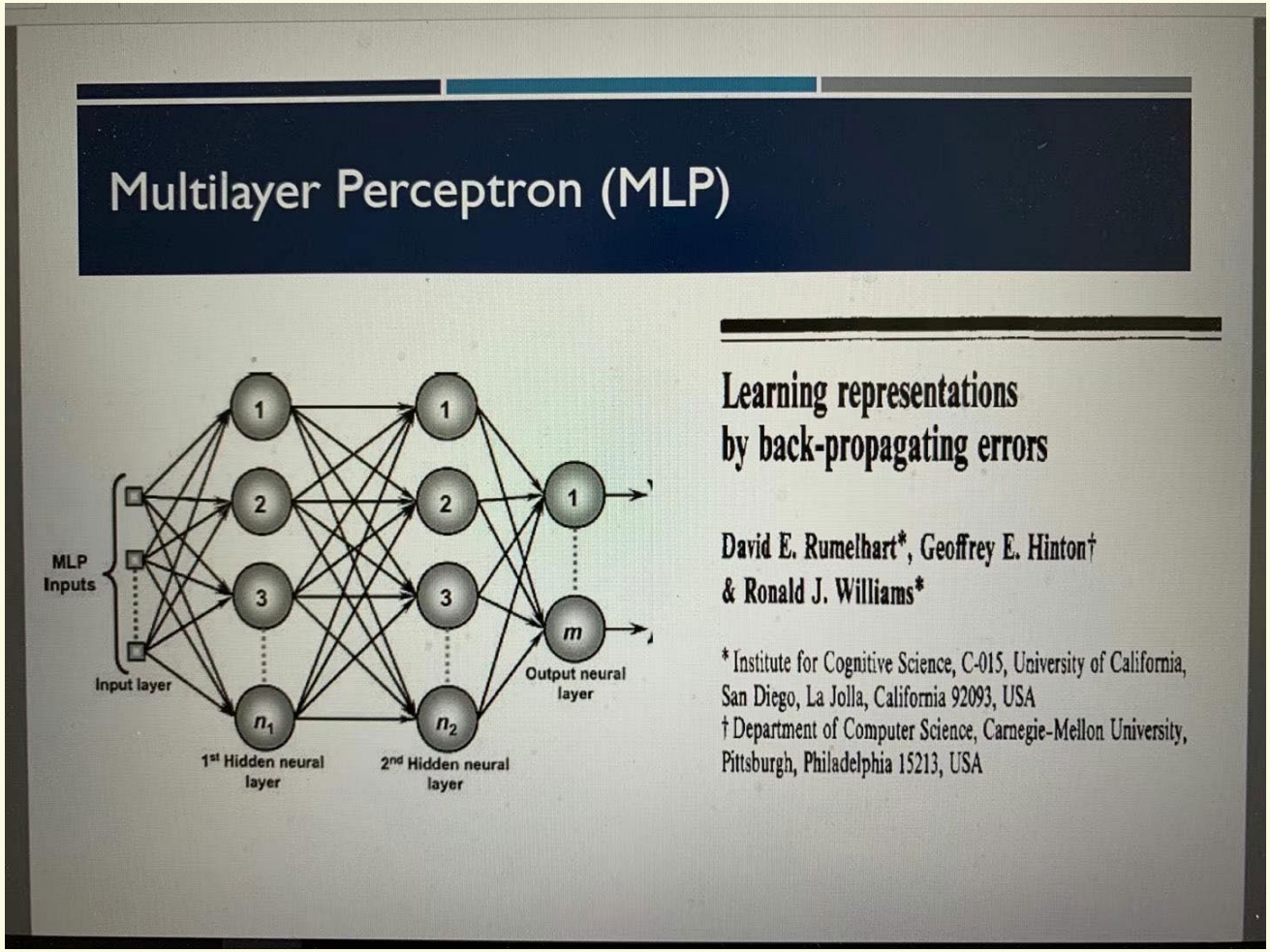


Fig. 7.2. Graphics of some "squashing" functions

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Backpropagation Algorithm



Backpropagation Algorithm (cont'd)

Error function:

$$E = \sum_p E_p = \frac{1}{2} \sum_p \sum_{i=1}^{m} (z_i^p - y_i^p)^2$$

General formula:

$$\Delta w_{ij}(t) = w_{ij}(t+1) - w_{ij}(t) = -\eta \frac{\partial E}{\partial w_{ij}} = -\eta \sum_p \frac{\partial E_p}{\partial w_{ij}}$$

Backpropagation Algorithm (cont'd)

Output layer l:

$$\frac{\partial E_p}{\partial w_{ij}^l} = \frac{\partial E_p}{\partial y_i^p}\frac{\partial y_i^p}{\partial u_i^l}\frac{\partial u_i^l}{\partial w_{ij}^l} =$$

$$-(z_i^p - y_i^p)y_i^p(1-y_i^p)v_j^{l-1} = -\delta_i^l v_j^{l-1}$$

where $\delta_i^l = -\frac{\partial E_p}{\partial u_i^l} = (z_i^p - y_i^p)y_i^p(1-y_i^p).$

Backpropagation Algorithm (cont'd)

Hidden layer l-1:

$$-\frac{\partial E_p}{\partial v_i^{l-1}} = -\sum_j \frac{\partial E_p}{\partial u_j^l}\frac{\partial u_j^l}{\partial v_i^{l-1}} = \sum_j (-\frac{\partial E_p}{\partial u_j^l})\frac{\partial u_j^l}{\partial v_i^{l-1}} = \sum_j \delta_j^l w_{ji}^l$$

$$\delta_i^{l-1} = -\frac{\partial E_p}{\partial u_i^{l-1}} = -\frac{\partial E_p}{\partial v_i^{l-1}}\frac{\partial v_i^{l-1}}{\partial u_i^{l-1}} = (\sum_j \delta_j^l w_{ji}^l) v_i^{l-1}(1-v_i^{l-1})$$

$$\frac{\partial E_p}{\partial w_{ij}^{l-1}} = \frac{\partial E_p}{\partial u_i^{l-1}}\frac{\partial u_i^{l-1}}{\partial w_{ij}^{l-1}} = -\delta_i^{l-1} v_j^{l-2}$$

Backpropagation Algorithm (cont'd)

Input layer 1:

$$\delta_i^1 = (\sum_j \delta_j^2 w_{ji}^2) v_i^1 (1 - v_i^1)$$

$$\frac{\partial E_p}{\partial w_{ij}^1} = \frac{\partial E_p}{\partial u_i^1} \frac{\partial u_i^1}{\partial w_{ij}^1} = -\delta_i^1 x_j^p$$

Backpropagation Algorithm (cont'd)

1) Initialize weights and threshold randomly.
2) Calculate actual output of the MLP:
3) Adapt weights for all layers:

$$w_{ij}(t+1) = w_{ij}(t) - \eta \sum_{p} \frac{\partial E_p}{\partial w_{ij}}$$

4) Repeat until w converges

Local Minima

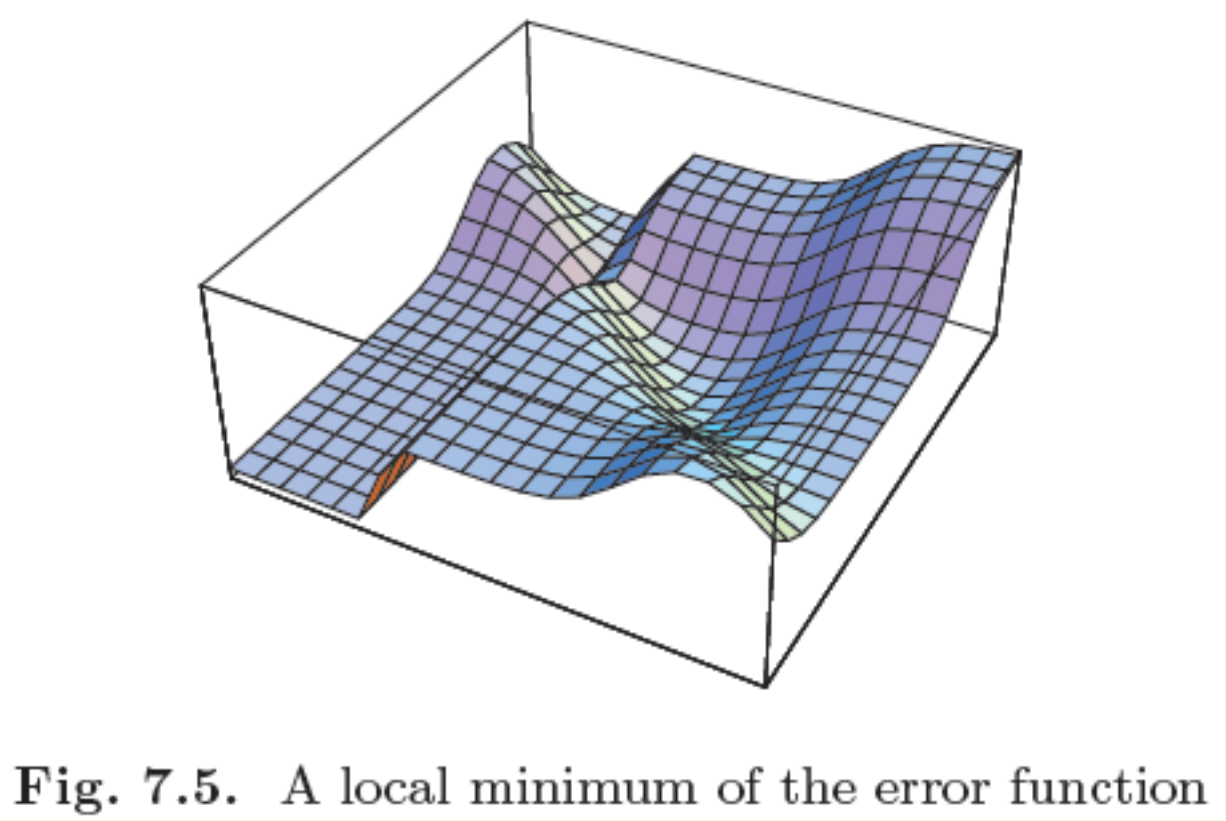


Fig. 7.5. A local minimum of the error function

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Momentum Term

To avoid local oscillation, a momentum term is sometimes added:

$$\Delta w_{ij}(t) = -\eta \frac{\partial E}{\partial w_{ij}} + \alpha \Delta w_{ij}(t-1)$$

$$0 \le \alpha < 1$$

Kolmogorov Theorem

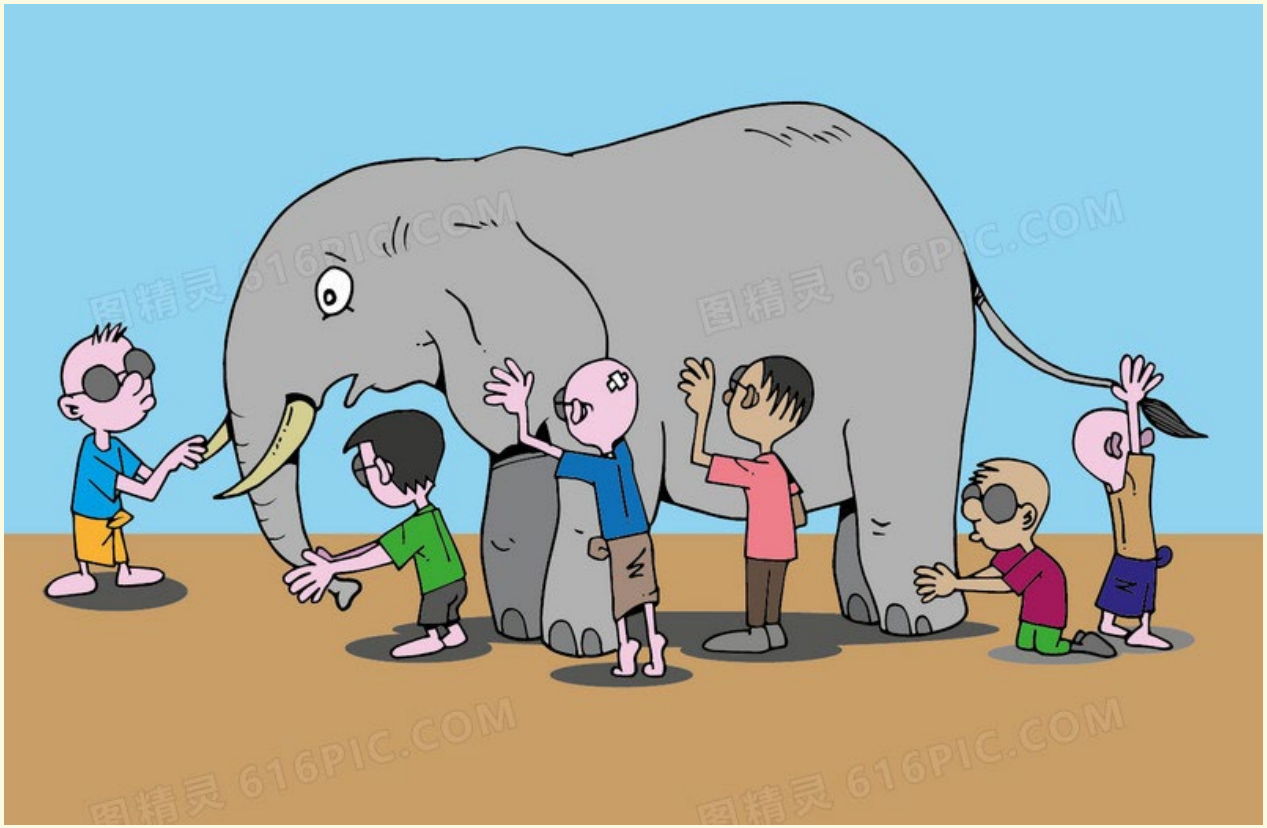
Let f: $[0, 1]^n$ -> $[0, 1]$ be a continuous function. There exist functions of one argument g and h_j for $j=1,2,\ldots,2n+1$ and constant w_i for $i=1,2,\ldots,n$ such that

$$f(x_1, x_2, \ldots, x_n) = \sum_{j=1}^{2n+1} g[\sum_{i=1}^{n} w_i h_j(x_j)].$$

Universal Approximators

- Multilayer feedforward neural networks are universal approximators of continuous functions.
- A set of weights exist such that the approximation errors can be arbitrarily small.
- However, the BP algorithm is not guaranteed to find such a set of weights.

Blind Men and an Elephant



Overfitting Problem

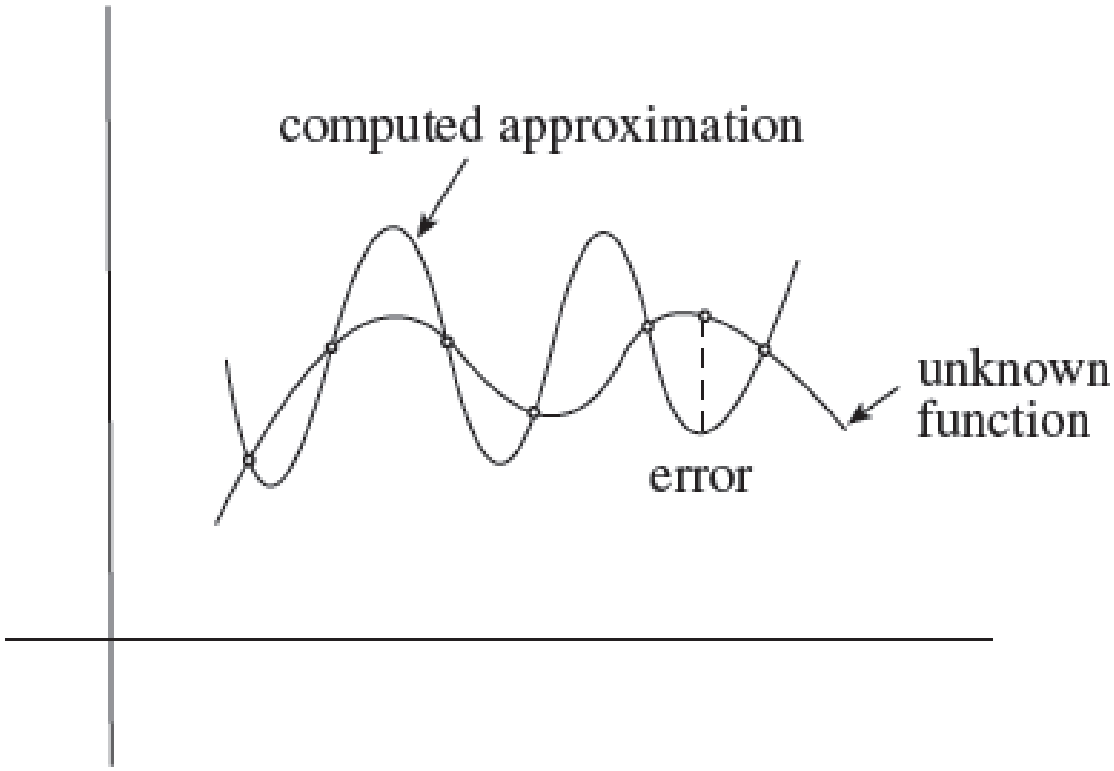


Fig. 6.23. Overfitting a polynomial approximation

© *Neural Networks: A Systematic Introduction*, Springer, 1996.

Radial Basis Functions

- A radial basis function (RBF) is a real-valued function whose value depends on the distance from its origin or center.
- Related to kernel theory in statistical learning.
- Used as the means for approximating or interpolating multivariate functions.

Radial Basis Functions

$$y = \sum_{i=1}^{h} w_i \Phi(\| x - c \|)$$

Polyharmonic spline $\Phi(r) = r^p$, $p = 1,3,5,...$

Gaussian $\Phi(r) = \exp(-\frac{r^2}{\sigma^2})$

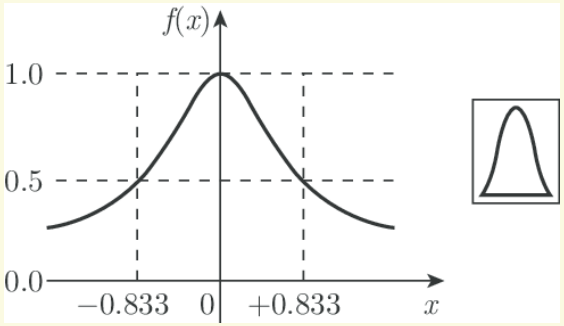
Thin plate spline $\Phi(r) = r^2 \log(r)$

Multiquadric $\Phi(r) = \sqrt{(r^2 + \sigma^2)}$

Inverse multiquadric $\Phi(r) = \frac{1}{\sqrt{(r^2 + \sigma^2)}}$

Radial Basis Functions

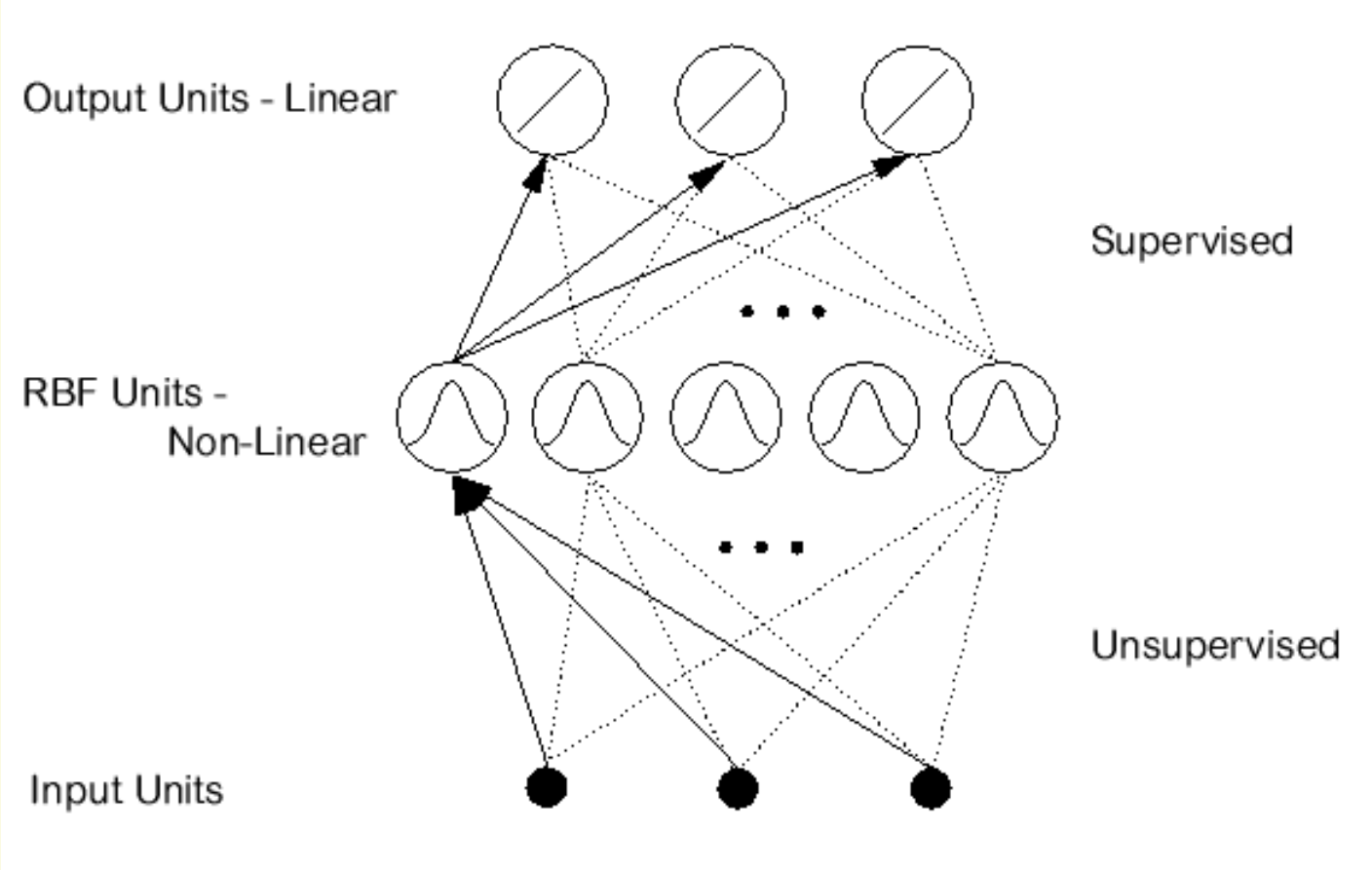
- The most commonly used radial-basis function is a Gaussian function
- In an RBF network, *r* is the distance from the center.
- Graphically, it is a bell-shaped curve.



Radial Basis Function Networks

- Proposed first by Broomhead and Lowe from UK in 1988.
- A linear combination of a number radial basis functions that play the role of hidden neurons.
- Two-layer architecture. Its output layer uses a linear activation function as ADALINE. Its hidden layer uses radial basis activation functions.

Radial Basis Function Networks



Cover's Theorem

- Cover's Theorem (1965):
- A dichotomy $\{X^+, X^-\}$ is said to be φ-separable if there exist an m-dimensional vector w such that
  - $w^T \varphi(x) > 0$, if x in X^+
  - $w^T \varphi(x) < 0$, if x in X^-
  - The hyperplane defined by $w^T \varphi(x) = 0$ is the separating surface between the two classes.

XOR Problem Revisited

An RBF network can transform the linearly inseparable XOR data in the input space to linearly separable data in the hidden state space.

XOR Problem Revisited

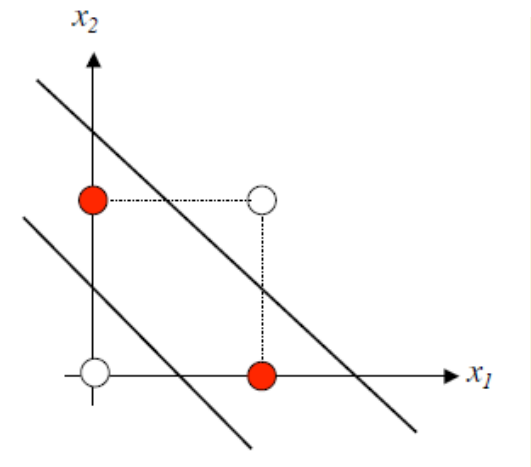
- Using a pair of Gaussian RBFs, the input patterns are mapped onto the φ_1- φ_2 plane and become linearly separable.

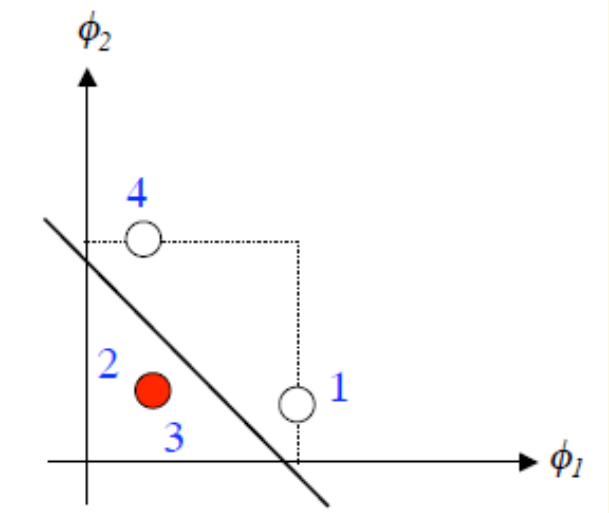
$$\phi_1(x) = e^{-\|x-c^1\|^2} \qquad \phi_2(x) = e^{-\|x-c^2\|^2}$$

Let $c^1 = [1,1]^T$ $c^2 = [0,0]^T$

XOR Problem Revisited

p	x_1	x_2	ϕ_1	ϕ_2
1	0	0	1.0000	0.1353
2	0	1	0.3678	0.3678
3	1	0	0.3678	0.3678
4	1	1	0.1353	1.0000

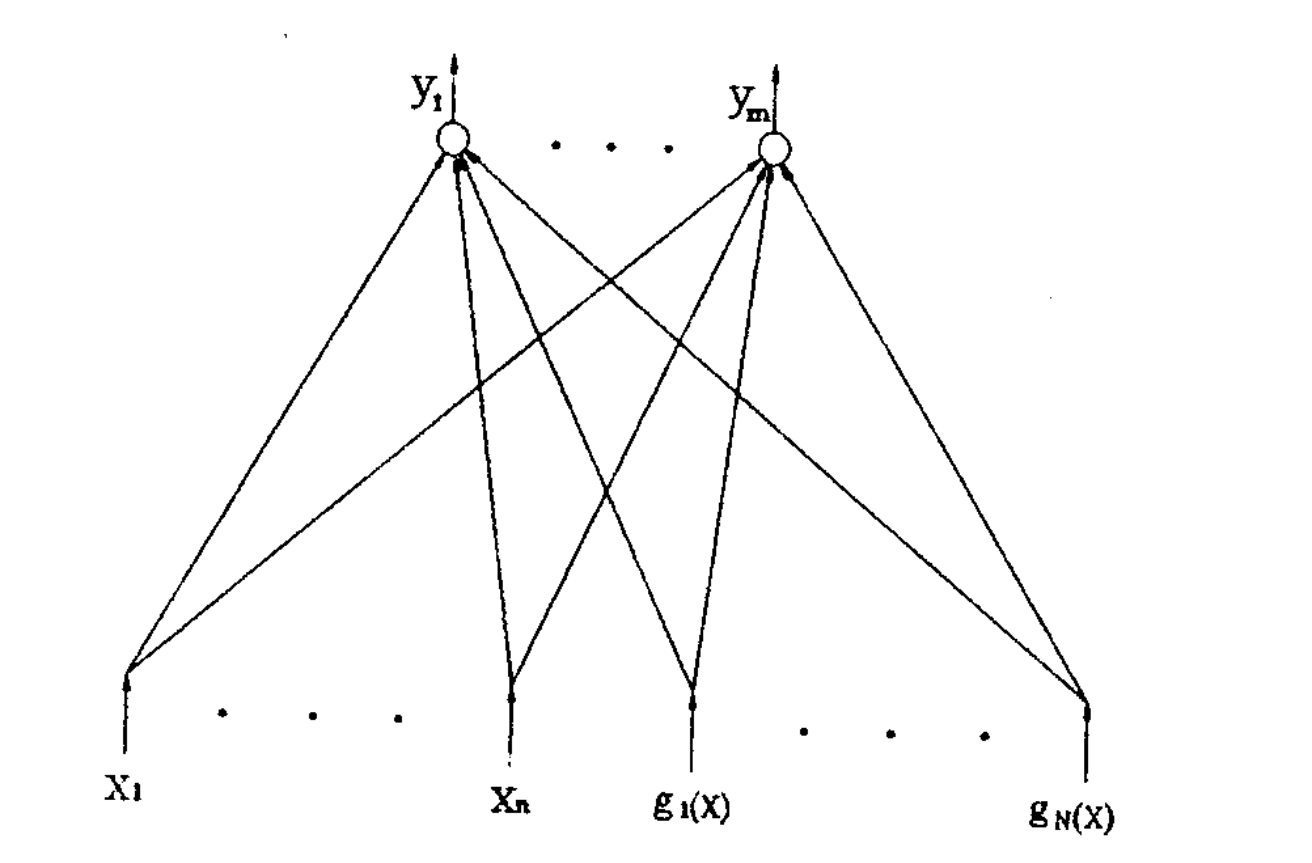




Functional Link Network

- Proposed by Yoh-Han Pao at CWRU in late 80's
- One-layer feedforward architecture
- Higher-order aggregation rule
- Fast learning process
- Local minima could be eliminated
- Many successful stories in applications

Functional Link Network



Extreme Learning Machine

- Proposed by Guangbin Huang at NTU in mid 2000's
- One-layer feedforward architecture
- Random connection weights from inputs to hidden neurons
- Fast learning process for weights in output layer.
- Local minima eliminated

Support Vector Machine

- Proposed by Vladmir Vapnik based on statistical learning and kernel theory in early 1990's.
- Minimization of structural risk.
- Maximal generalization power.



V. Vapnik

Linear Discriminant Function

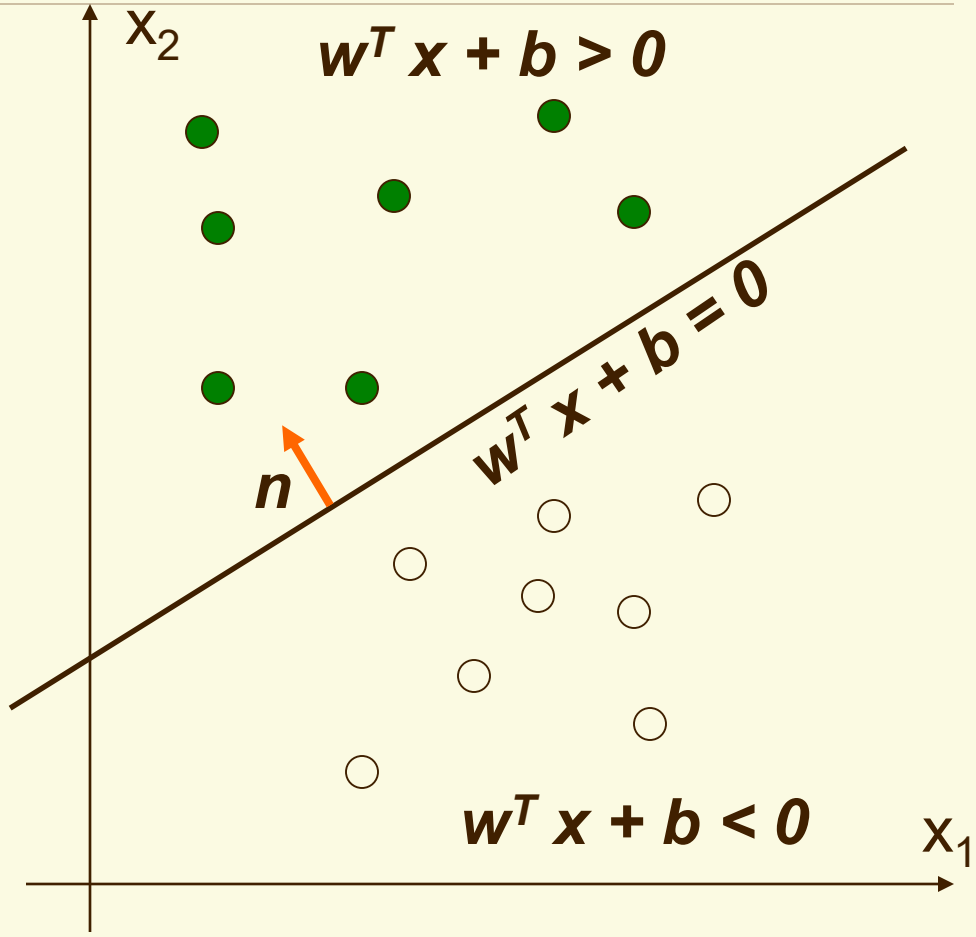
g(x) is a linear function:

$$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

- A hyper-plane in the feature space

(Unit-length) normal vector of the hyper-plane:

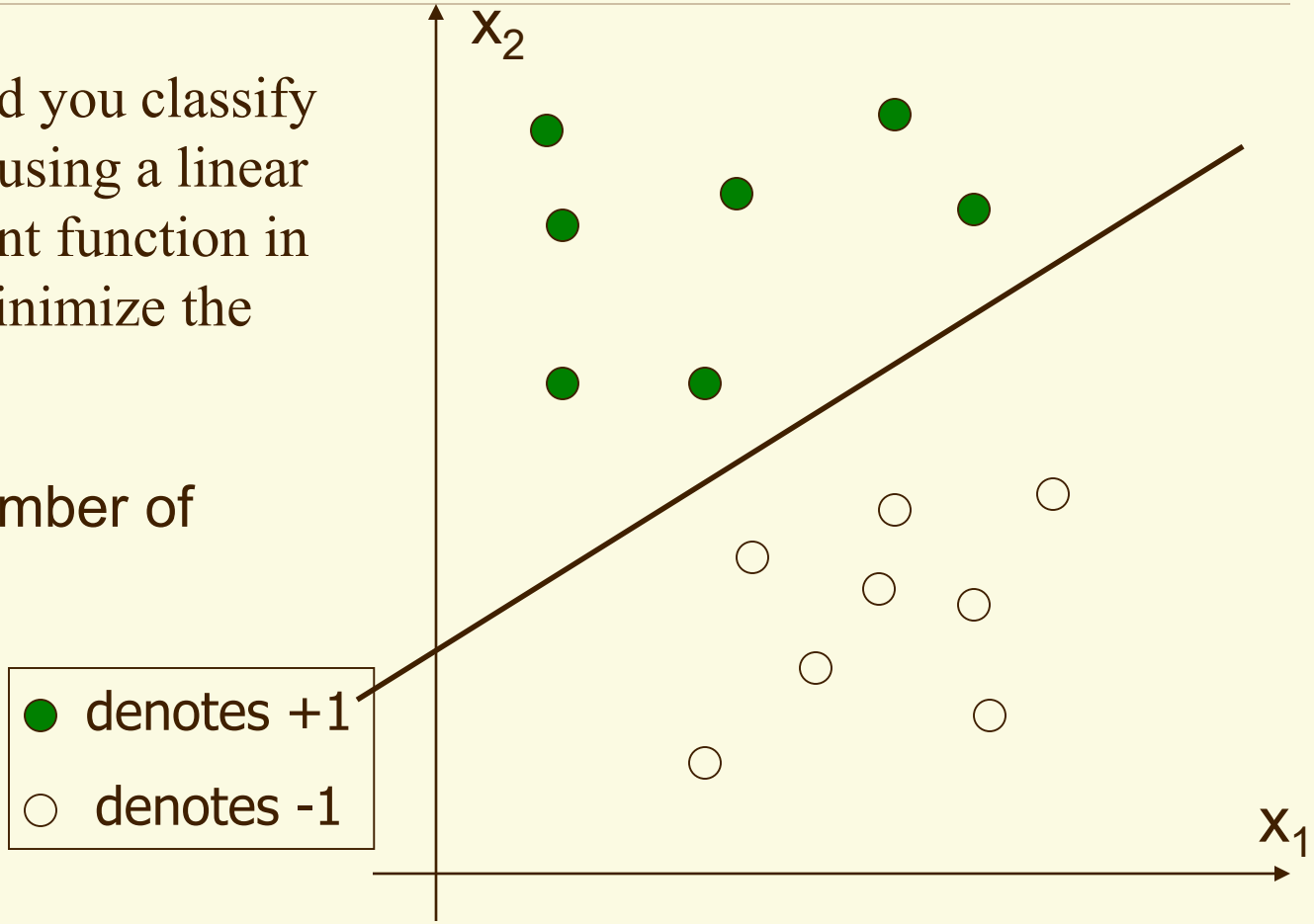
$$\mathbf{n} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$$



Linear Discriminant Function

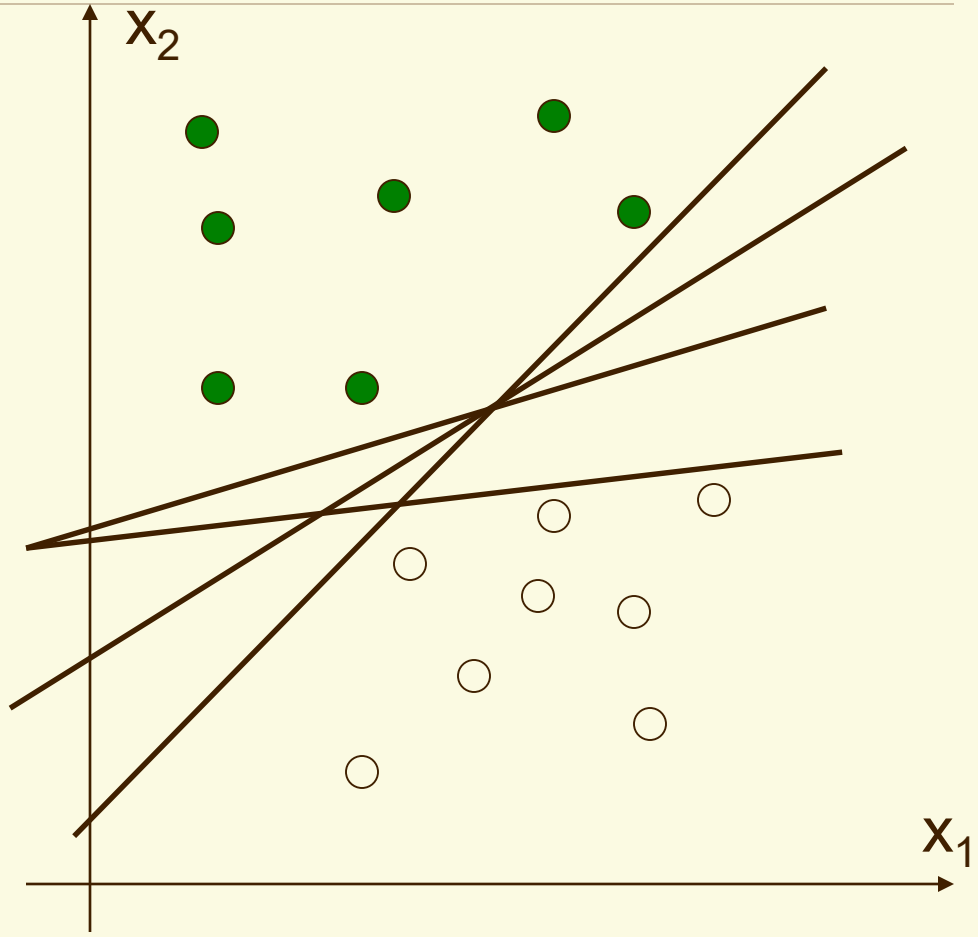
How would you classify these data using a linear discriminant function in order to minimize the error?

Infinite number of answers!



Linear Discriminant Function

Which one is the best with maximal generalization power?



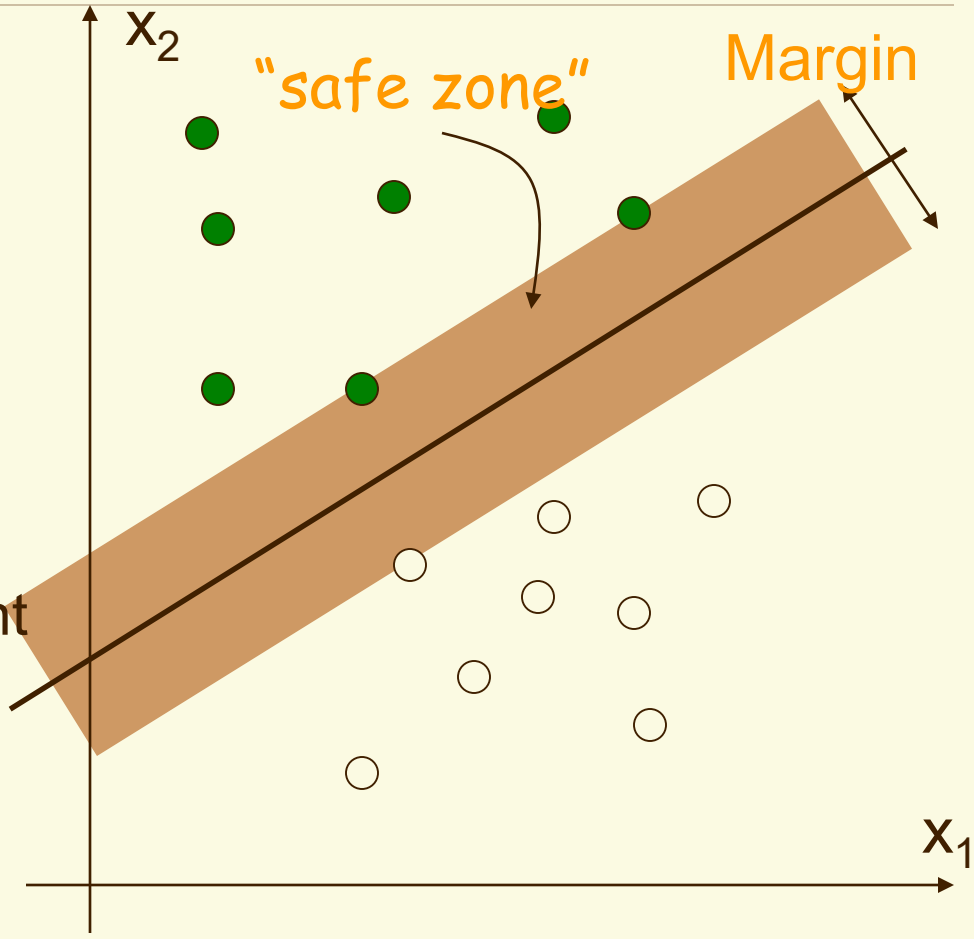
Maximal Margin Classifier

The linear discriminant function (classifier) with the maximum margin is the best.

Margin is defined as the width that the boundary could be increased by before hitting a data point

Why it is the best?

Robust to outliners and thus strong generalization ability



Maximal Margin Classifier

Given a set of data:

$\{(\mathbf{x}_i, y_i)\},\ i = 1, 2, \cdots, n$, where

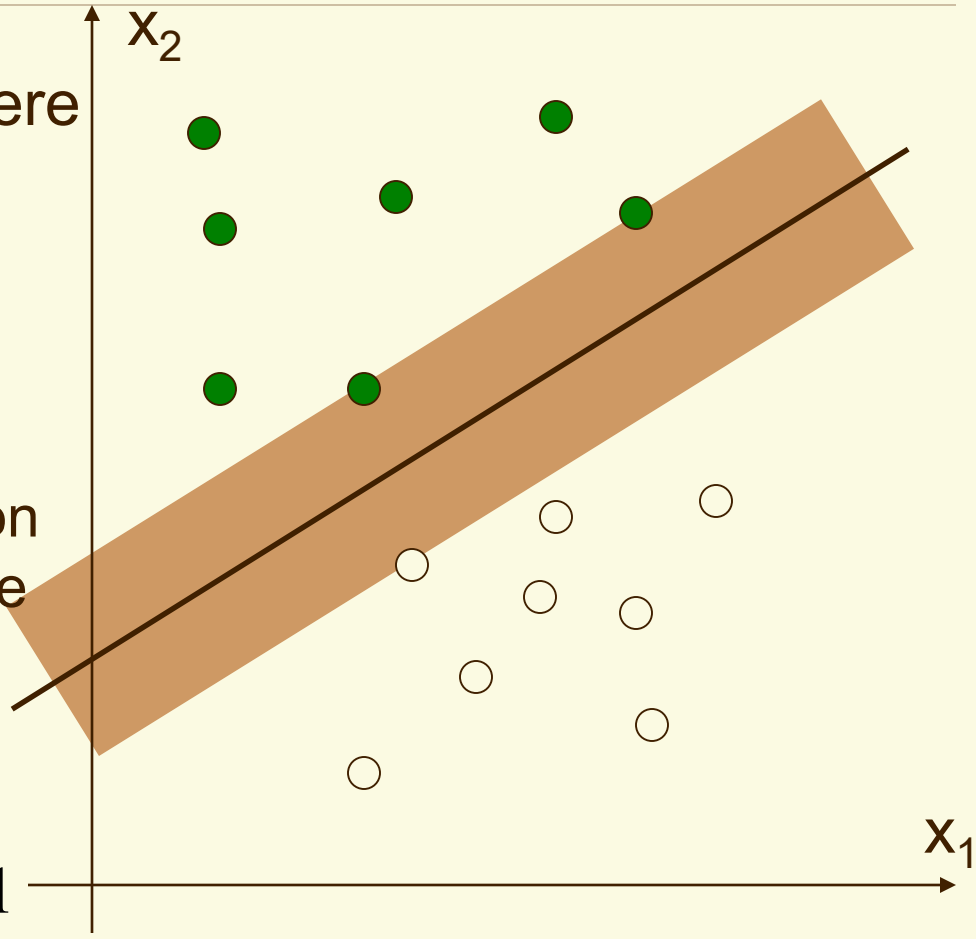
$$\text{For } y_i = +1,\ \ \mathbf{w}^T\mathbf{x}_i + b > 0$$

$$\text{For } y_i = -1,\ \ \mathbf{w}^T\mathbf{x}_i + b < 0$$

With a scale transformation on both *w* and *b*, the above is equivalent to

$$\text{For } y_i = +1,\ \ \mathbf{w}^T\mathbf{x}_i + b \geq 1$$

$$\text{For } y_i = -1,\ \ \mathbf{w}^T\mathbf{x}_i + b \leq -1$$



Maximal Margin Classifier

Let

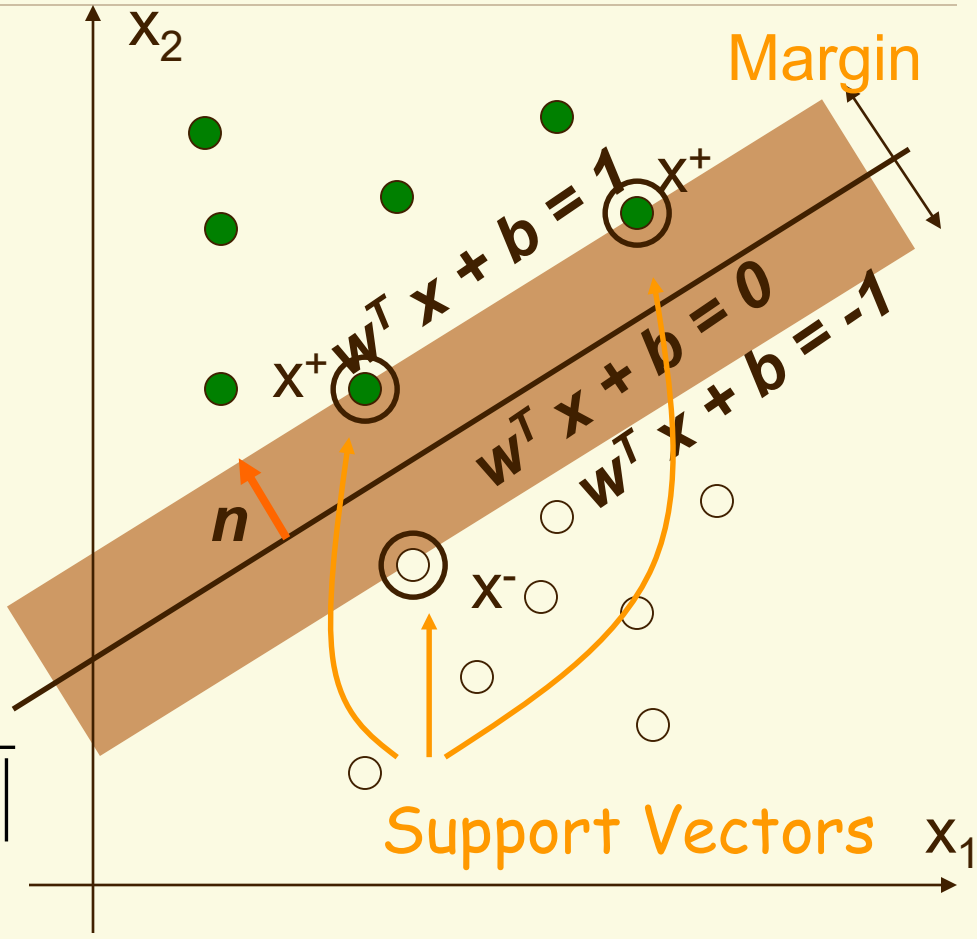
$$\mathbf{w}^T \mathbf{x}^+ + b = 1$$

$$\mathbf{w}^T \mathbf{x}^- + b = -1$$

The margin width is:

$$M = (\mathbf{x}^+ - \mathbf{x}^-) \cdot \mathbf{n}$$

$$= (\mathbf{x}^+ - \mathbf{x}^-) \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|}$$



Maximal Margin Classifier

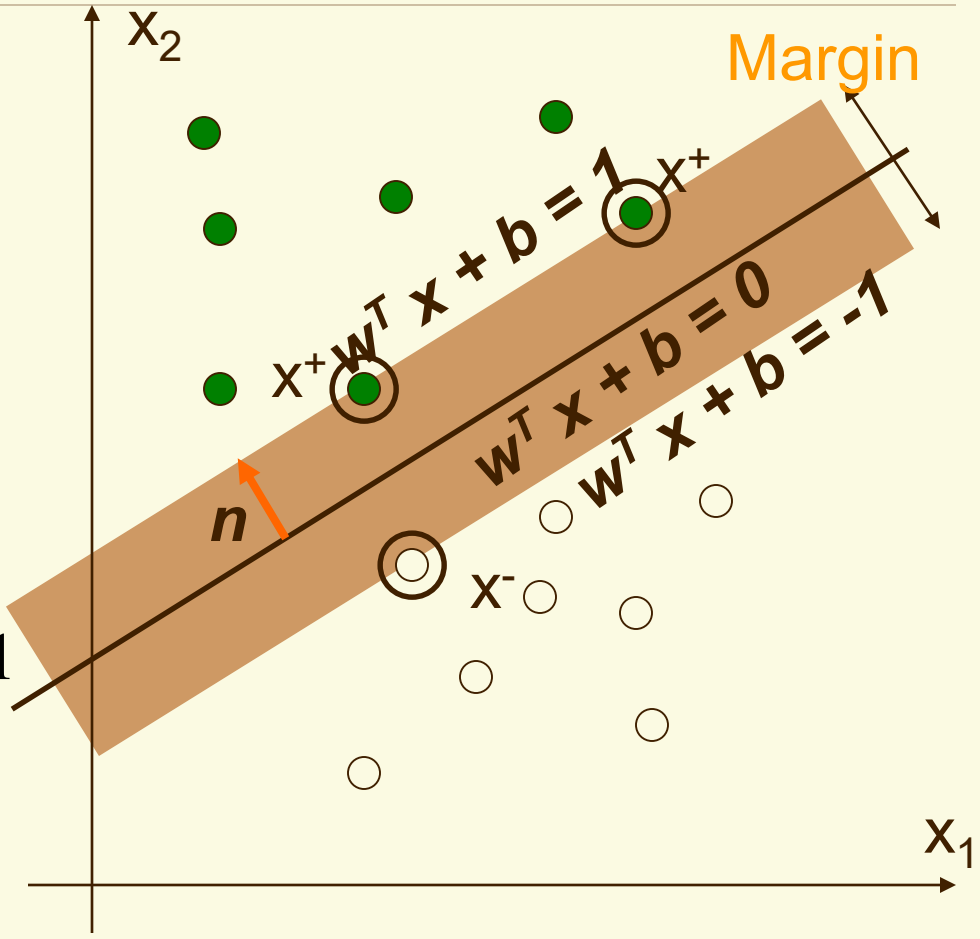
- Formulation:

$$\text{maximize} \quad \frac{2}{\|\mathbf{w}\|}$$

such that

$$\text{For } y_i = +1, \quad \mathbf{w}^T \mathbf{x}_i + b \geq 1$$

$$\text{For } y_i = -1, \quad \mathbf{w}^T \mathbf{x}_i + b \leq -1$$



Maximal Margin Classifier

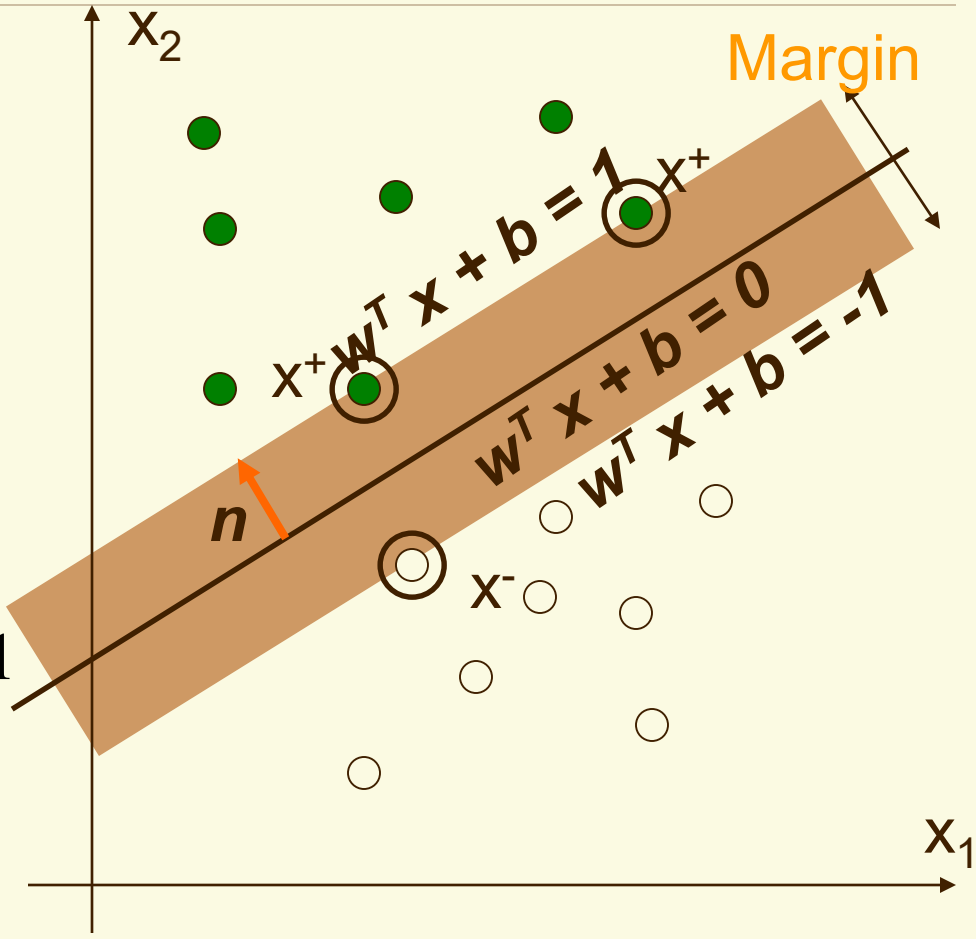
- Formulation:

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

such that

$$\text{For } y_i = +1, \quad \mathbf{w}^T\mathbf{x}_i + b \geq 1$$

$$\text{For } y_i = -1, \quad \mathbf{w}^T\mathbf{x}_i + b \leq -1$$



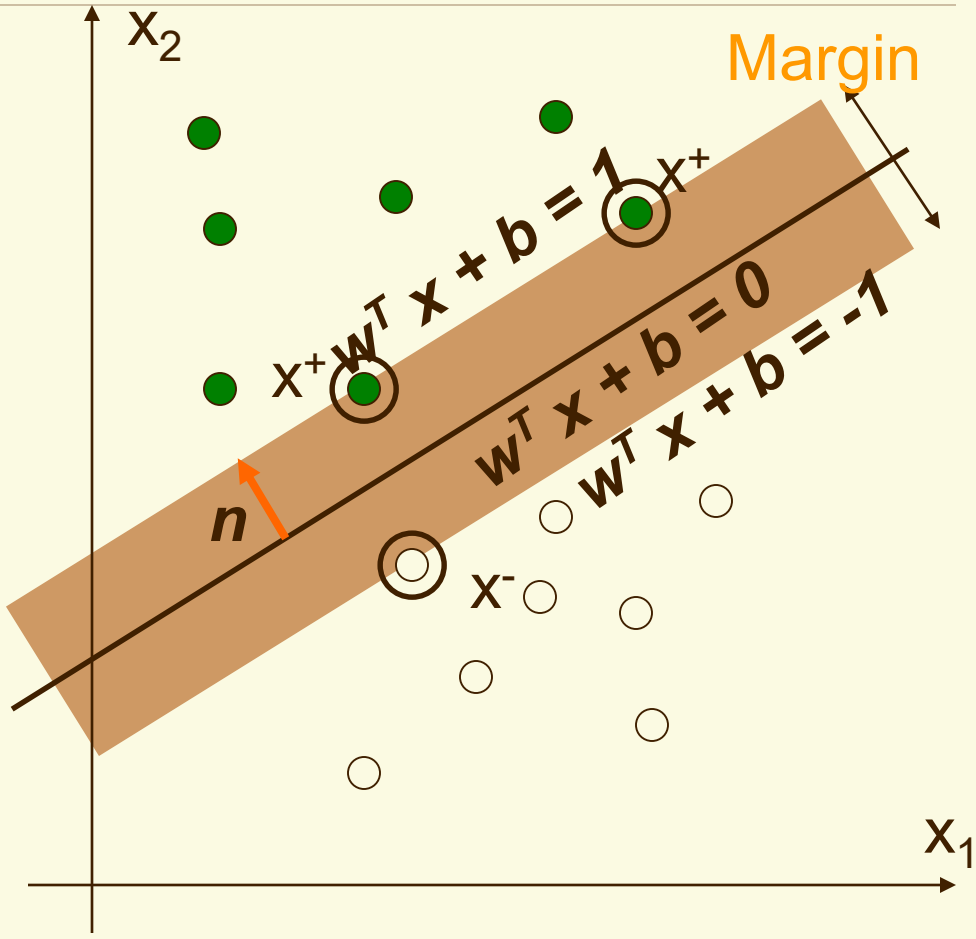
Maximal Margin Classifier

- Formulation:

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

subject to

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$

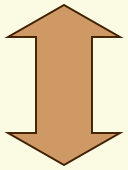


Quadratic Programming Problem

Quadratic programming with linear constraints

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{s.t.} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$



Lagrangian Function

$$\text{minimize} \quad L_p(\mathbf{w}, b, \alpha_i) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \left(y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 \right)$$

$$\text{s.t.} \quad \alpha_i \geq 0$$

Quadratic Programming Problem

$$\text{minimize} \quad L_p(\mathbf{w}, b, \alpha_i) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \left(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \right)$$

$$\text{s.t.} \quad \alpha_i \geq 0$$

$$\frac{\partial L_p}{\partial \mathbf{w}} = 0 \quad \Rightarrow \quad \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i$$

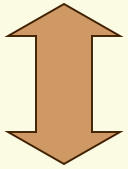
$$\frac{\partial L_p}{\partial b} = 0 \quad \Rightarrow \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

Quadratic Programming Problem

$$\text{minimize} \quad L_p(\mathbf{w}, b, \alpha_i) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \left(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \right)$$

$$\text{s.t.} \quad \alpha_i \geq 0$$

Lagrangian dual problem



$$\text{maximize} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

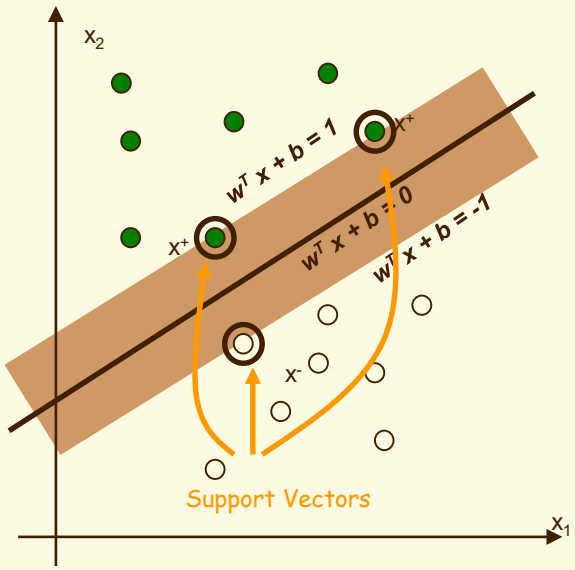
$$\text{s.t.} \quad \alpha_i \geq 0 \text{ , and } \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

Quadratic Programming Problem

From KKT condition, we know:

$$\alpha_i \left(y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \right) = 0$$

Thus, only support vectors $\alpha_i \neq 0$



The solution has the form:

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i = \sum_{i \in \mathrm{SV}} \alpha_i y_i \mathbf{x}_i$$

get b from $y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 = 0,$

where $\mathbf{x}_i$ is support vector

Linear Discriminant Function

The linear discriminant function is:

$$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = \sum_{i \in \mathrm{SV}} \alpha_i \mathbf{x}_i^T \mathbf{x} + b$$

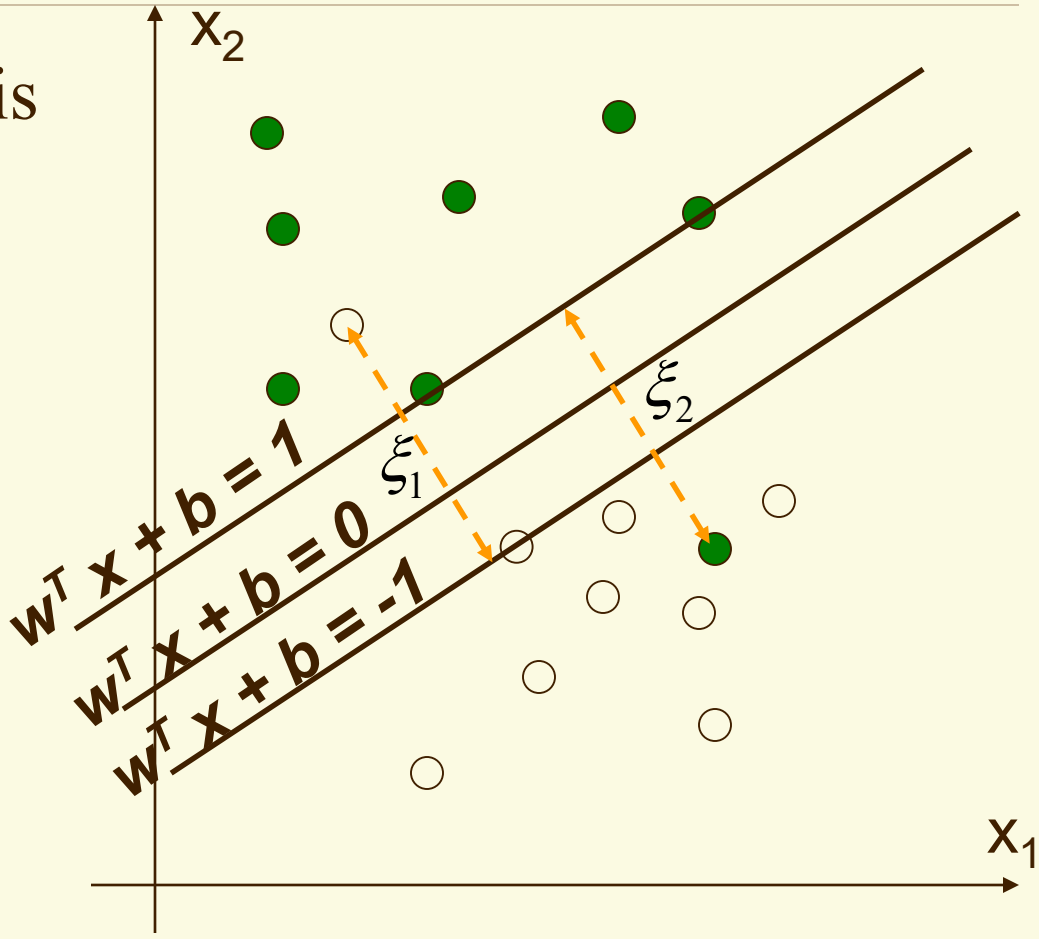
It is a weighted *dot product* between the test point $\boldsymbol{x}$ and the support vectors $\boldsymbol{x_i}$

Solving the optimization problem involved computing the dot products $\boldsymbol{x_i^T x_j}$ between all pairs of training samples

Soft Constraints

What to do if data is not linearly separable? (noisy data, outliers, etc.)

Slack variables ξ_i can be added to allow mis-classification of nonlinearly distributed or noisy data



Soft Constraints

Problem formulation:

$$\text{minimize} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

subject to

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

Parameter *C* is a weight between marginal maximization and misclassification miinimization.

Maximal Margin Classifier

Problem reformulation: (Lagrangian dual problem)

$$\text{maximize} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

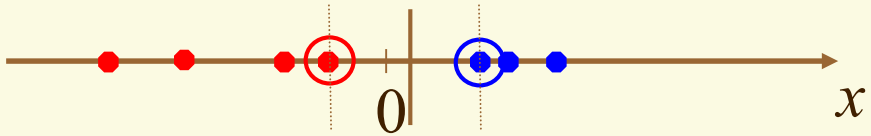
Subject to

$$0 \le \alpha_i \le C$$

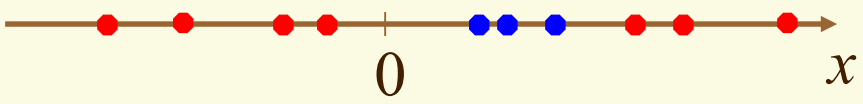
$$\sum_{i=1}^{n} \alpha_i y_i = 0$$

Nonlinear SVM

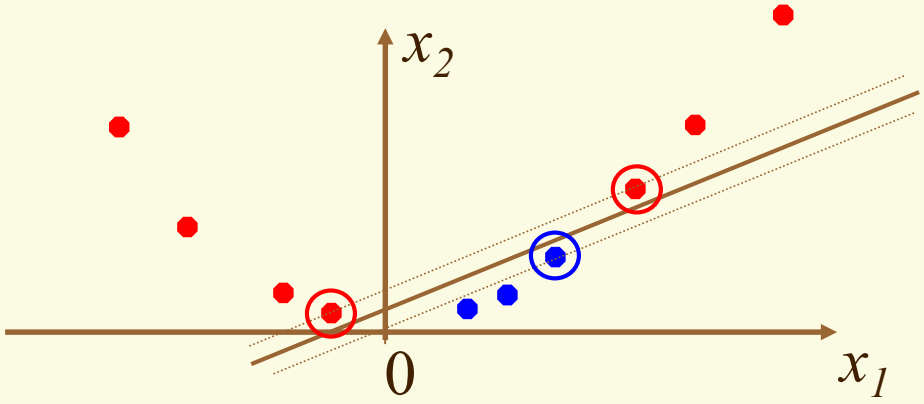
Datasets that are linearly separable with noise work out great:



But what to do if the dataset is nonlinearly distributed?

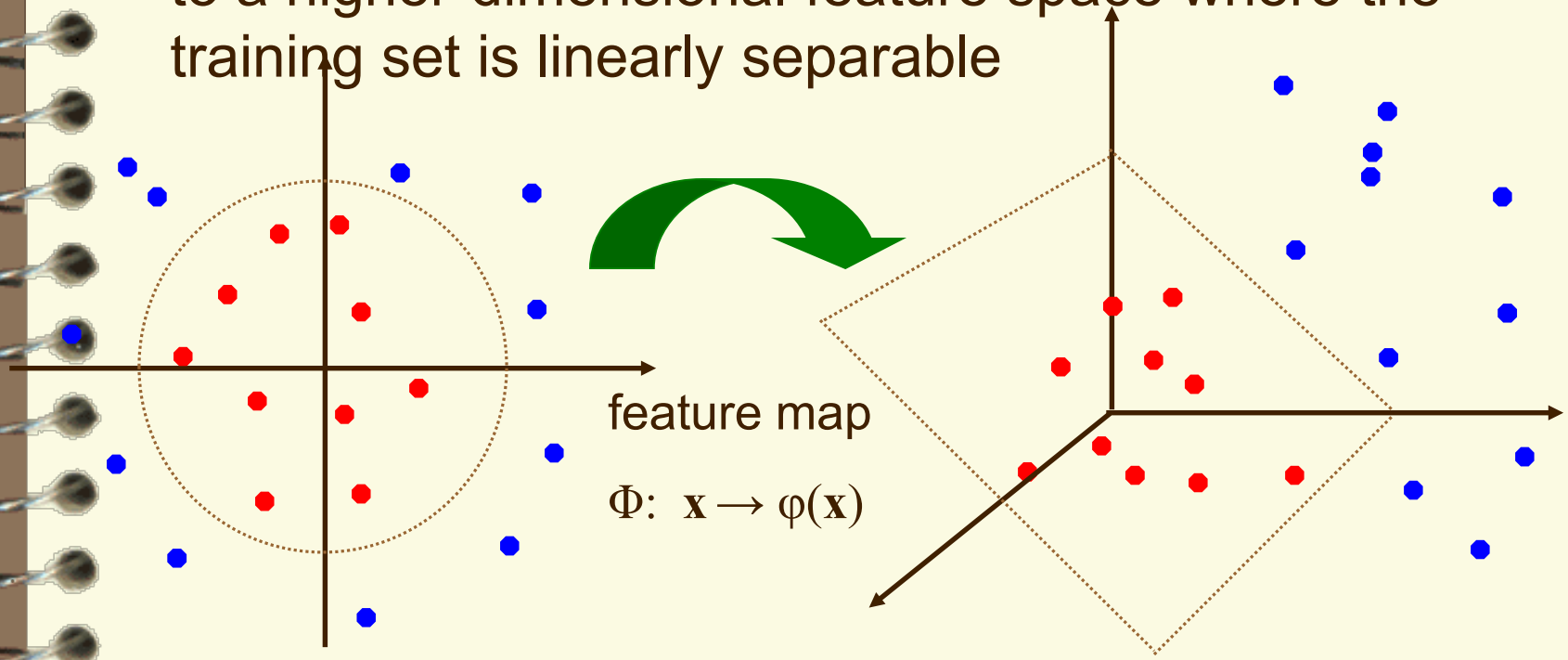


How about… mapping data to a higher-dimensional space:



Feature Space

General idea: the original input space can be mapped to a higher-dimensional feature space where the training set is linearly separable



The Kernel Trick

With this mapping, the discriminant function is now:

$$g(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b = \sum_{i \in \text{SV}} \alpha_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}) + b$$

No need to know this mapping explicitly, because we only use the dot product of feature vectors in both the training and test.

A *kernel function* is defined as a function that corresponds to a dot product of two feature vectors in some expanded feature space:

$$K(\mathbf{x}_i, \mathbf{x}_j) \equiv \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

An Example

2-dimensional vectors $\mathbf{x}=[x_1 \; x_2]$;

let $K(\mathbf{x}_i,\mathbf{x}_j)=(1 + \mathbf{x}_i^T\mathbf{x}_j)^2$,

Need to show that $K(\mathbf{x}_i,\mathbf{x}_j) = \varphi(\mathbf{x}_i)^T\varphi(\mathbf{x}_j)$:

$K(\mathbf{x}_i,\mathbf{x}_j)=(1 + \mathbf{x}_i^T\mathbf{x}_j)^2$,

$= 1+ x_{i1}^2x_{j1}^2 + 2\, x_{i1}x_{j1}\, x_{i2}x_{j2}+ x_{i2}^2x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2}$

$= [1 \;\; x_{i1}^2 \;\; \sqrt{2}\, x_{i1}x_{i2} \;\; x_{i2}^2 \;\; \sqrt{2}x_{i1} \;\; \sqrt{2}x_{i2}]^T \, [1 \;\; x_{j1}^2 \;\; \sqrt{2}\, x_{j1}x_{j2} \;\; x_{j2}^2 \;\; \sqrt{2}x_{j1} \;\; \sqrt{2}x_{j2}]$

$= \varphi(\mathbf{x}_i)^T\varphi(\mathbf{x}_j)$, where $\varphi(\mathbf{x}) = [1 \;\; x_1^2 \;\; \sqrt{2}\, x_1x_2 \;\; x_2^2 \;\; \sqrt{2}x_1 \;\; \sqrt{2}x_2]$

Common Kernel Functions

- Linear kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- Polynomial kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^p$
- Gaussian (Radial-Basis Function (RBF)) kernel:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

- Sigmoid:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta_0 \mathbf{x}_i^T \mathbf{x}_j + \beta_1)$$

Kernel Functions

For some functions $K(x_i,x_j)$ checking that

$$K(x_i,x_j)= \varphi(x_i)^T\varphi(x_j)$$ **can be cumbersome.**

Mercer's theorem:

Every semi-positive definite symmetric function is a kernel

- **Semi-positive definite symmetric functions correspond to a semi-positive definite symmetric Gram matrix:**

$K(x_1,x_1)$	$K(x_1,x_2)$	$K(x_1,x_3)$	…	$K(x_1,x_N)$
$K(x_2,x_1)$	$K(x_2,x_2)$	$K(x_2,x_3)$		$K(x_2,x_N)$
…	…	…	…	…
$K(x_P,x_1)$	$K(x_N,x_2)$	$K(x_N,x_3)$	…	$K(x_N,x_N)$

Nonlinear SVM: Optimization

Problem reformulation: (Lagrangian dual problem)

$$\text{maximize} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

$$\text{subject to} \quad 0 \le \alpha_i \le C$$

$$\sum_{i=1}^{n} \alpha_i y_i = 0$$

The solution of the discriminant function is

$$g(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i K(\mathbf{x}_i, \mathbf{x}) + b$$

The optimization technique is the same.

SVM Learning

1. Choose a kernel function
2. Choose a value for *C*
3. Solve the quadratic programming problem (many software packages available)
4. Construct the discriminant function from the support vectors

Design Issues

- Choice of kernel
  - Gaussian or polynomial kernel is default
  - if ineffective, more elaborate kernels are needed
  - domain experts can give assistance in formulating appropriate similarity measures
- Choice of kernel parameters
  - e.g. σ in Gaussian kernel
  - σ is the distance between closest points with different classifications
  - In the absence of reliable criteria, applications rely on the use of a validation set or cross-validation to set such parameters.
- Optimization criterion – Hard margin v.s. Soft margin
  - a lengthy series of experiments in which various parameters are tested

SVM Summary

- 1. Maximal Margin Classifier
  - Better generalization ability & less over-fitting
- 2. The Kernel Trick
  - Map data points to a higher dimensional space to make them linearly separable.
  - Since only dot product is used, we do not need to represent the mapping explicitly.

Support Vector Regression

Problem formulation

$$\text{minimize } \frac{1}{2}\|w\|^2$$

$$\text{subject to } \|y_i - (w \cdot x_i - b)\| \leq \varepsilon$$

Support Vector Regression

Problem reformulation

$$\text{minimize } \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l}(\xi_i + \xi_i^*)$$

$$\text{subject to } \begin{cases} y_i - w \cdot x_i - b \leq \varepsilon + \xi_i \\ w \cdot x_i + b - y_i \leq \varepsilon + \xi_i^* \\ \xi_i,\ \xi_i^* \geq 0 \end{cases}$$

Support Vector Regression

$$L = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l}(\xi_i + \xi_i^*) - \sum_{i=1}^{l}(\eta_i\xi_i + \eta_i^*\xi_i^*)$$
$$- \sum_{i=1}^{l}\alpha_i(\varepsilon + \xi_i - y_i + w \cdot x_i + b)$$
$$- \sum_{i=1}^{l}\alpha_i^*(\varepsilon + \xi_i^* + y_i - w \cdot x_i - b)$$

$$\min_{w,b,\xi} \max_{\alpha,\eta} L$$

subject to $\alpha, \eta \geq 0$

Support Vector Regression

$$\frac{\partial L}{\partial b} = \sum_{i=1}^{l}(\alpha_i^* - \alpha_i) = 0$$

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{l}(\alpha_i - \alpha_i^*)x_i = 0$$

$$\frac{\partial L}{\partial \xi_i^{(*)}} = C - \alpha_i^{(*)} - \eta_i^{(*)} = 0$$

$$\max_{\alpha,\eta} -\frac{1}{2}\sum_{i,j=1}^{l}(\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*)x_i \cdot x_j - \varepsilon\sum_{i=1}^{l}(\alpha_i + \alpha_i^*) + \sum_{i=1}^{l} y_i(\alpha_i - \alpha_i^*)$$

$$\text{subject to } \sum_{i=1}^{l}(\alpha_i - \alpha_i^*) = 0 \text{ and } \alpha_i, \alpha_i^* \in [0, C]$$

Least-squares SVM

Proposed by Suykens and Vandewalle at KUL in 1999.

Let $y_i = \boldsymbol{w}^T \boldsymbol{\varphi}(\boldsymbol{x}_i) + b + e_i$

Primal problem formulation

$$\min_{\boldsymbol{w}, b, e_i} \quad \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + \gamma\frac{1}{2}\sum_{i=1}^{N} e_i^2$$

$$y_i = \boldsymbol{w}^T \boldsymbol{\varphi}(\boldsymbol{x}_i) + b + e_i, \quad i = 1, \ldots, N.$$

Least-squares SVM

Lagrangian

$$\mathcal{L} = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + \gamma\frac{1}{2}\sum_{i=1}^{N} e_i^2 - \sum_{i=1}^{N}\alpha_i(\boldsymbol{w}^T\boldsymbol{\varphi}(\boldsymbol{x}_i) + b + e_i - y_i)$$

$$\frac{\partial\mathcal{L}}{\partial w} = 0, \frac{\partial\mathcal{L}}{\partial b} = 0, \frac{\partial\mathcal{L}}{\partial e_i} = 0, \frac{\partial\mathcal{L}}{\partial\alpha_i} = 0 \text{ gives}$$

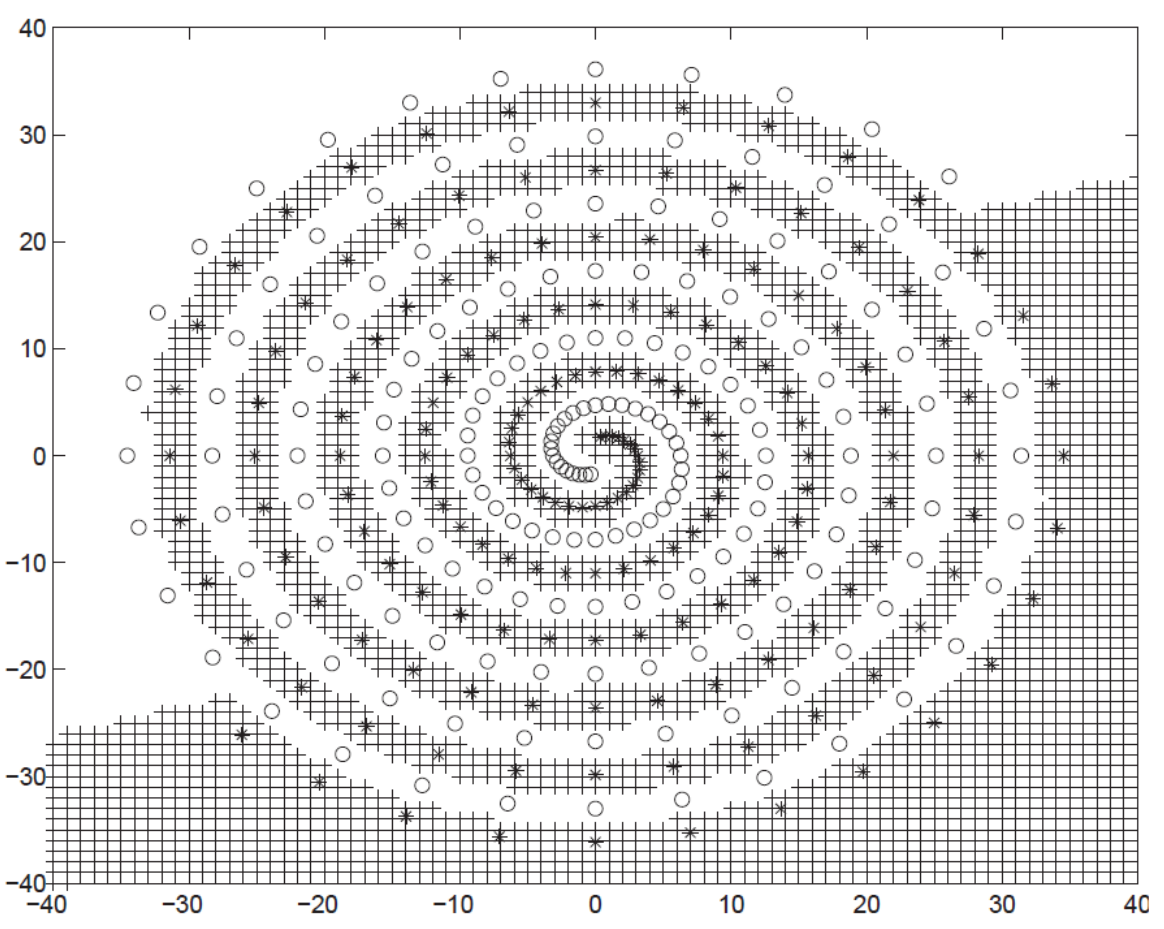
$$\left[\begin{array}{c|c} \boldsymbol{\Omega} + \gamma^{-1}\mathbf{I} & \mathbf{1} \\ \hline \mathbf{1}^T & 0 \end{array}\right] \left[\begin{array}{c} \boldsymbol{\alpha} \\ \hline b \end{array}\right] = \left[\begin{array}{c} \boldsymbol{y} \\ 0 \end{array}\right]$$

Least-squares SVM

Final model

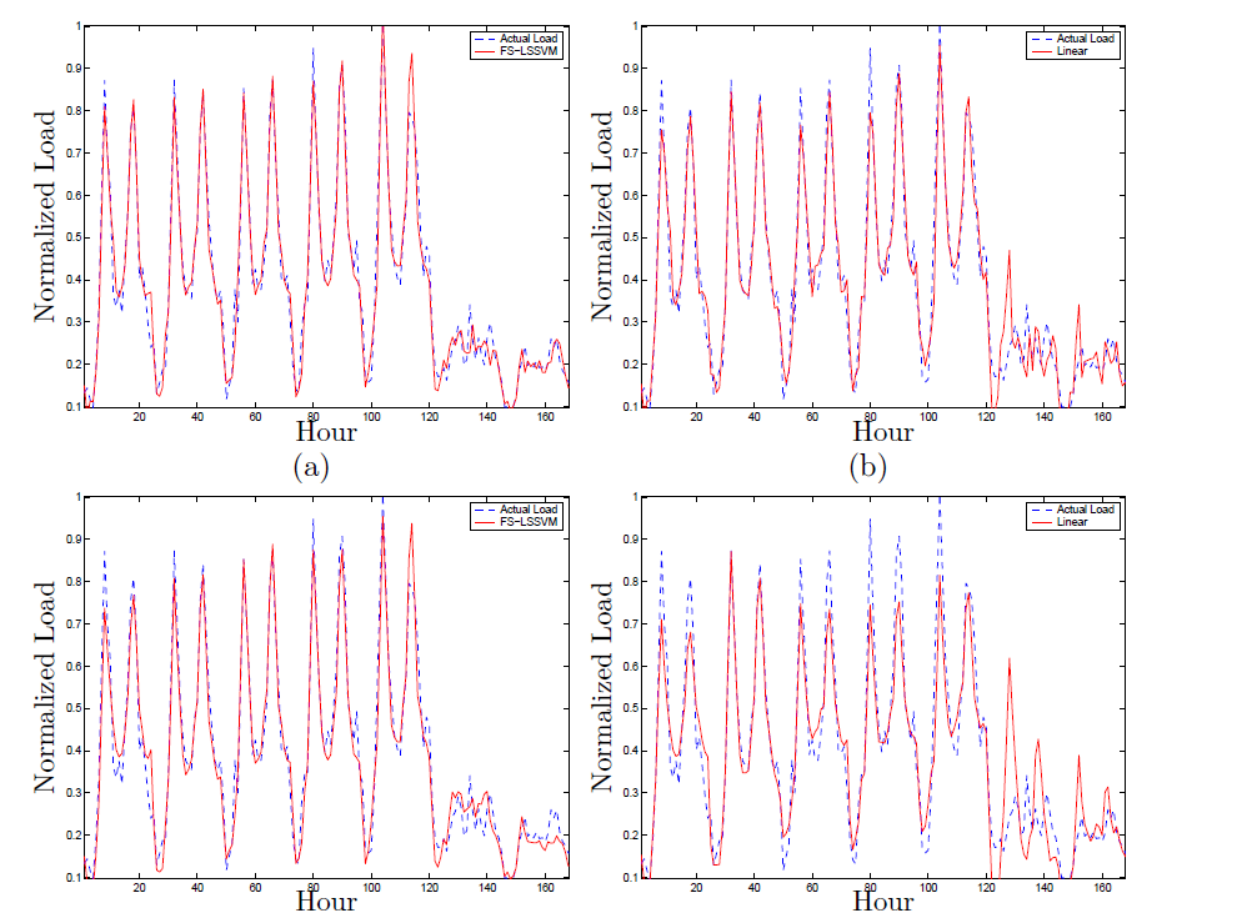
$$y(\boldsymbol{x}) = \sum_{i=1}^{N} \alpha_i K(\boldsymbol{x}_i, \boldsymbol{x}) + b$$

Two-spirial Classification



Ref: J. A. K. Suykens and J. Vandewalle, "Least Squares Support Vector Machine Classifiers," *Neural Processing Letters*, vol. 9, no. 3, pp. 293–300, 1999.

Short-term Prediction



Ref: M. Espinoza, J. A. K. Suykens, R. Belmans, and B. De Moor, "Electric Load Forecasting: Using Kernel-Based Modeling for Nonlinear System Identification," *IEEE Control Systems Magazine*, vol. 27, no. 5, pp. 43–57, 2007.